COMPLETE EDITION · VOLUMES 1–4

Complete Machine Learning Handbook

Theory, Visuals, Python Examples & Interview Preparation

Fundamentals · Ensembles · Preprocessing · Projects & Interviews

Four volumes combined in a single, fully linked PDF

Python · scikit-learn | Simple English + Hinglish | Executed code · Verified outputs

Table of Contents

Click any entry to jump to that chapter. Page numbers match the footer.

Start Here · How to use this handbook 4

Part A · Foundations

1 Machine Learning Foundations 7

Part B · Supervised Learning

2 Linear Regression 10
3 Logistic Regression 14
4 K-Nearest Neighbors (KNN) 18
5 Decision Tree 21
6 Gaussian Naive Bayes 25
7 Support Vector Machine (SVM) 29

Part C · Evaluation Metrics

8 Regression Metrics: R^2, MAE, MSE, RMSE 33
9 Classification Metrics: Accuracy, Precision, Recall, F1 36
10 Confusion Matrix and Classification Report 39

Part D · Model Tuning

11 Cross-Validation 43
12 GridSearchCV 46
13 RandomizedSearchCV 50

Part E · Unsupervised Learning

14 K-Means Clustering 54
15 DBSCAN 58
16 Principal Component Analysis (PCA) 62

Part F · Ensemble Learning

17 Ensemble Learning: The Big Idea 67
18 Stacking 69
19 Bagging 72
20 Random Forest 76
21 AdaBoost 79
22 Gradient Boosting 82

Part G · Data Preprocessing

23 IQR and Outlier Detection 87
24 Pipeline 91
25 ColumnTransformer 94
26 OrdinalEncoder 97
27 OneHotEncoder 100

28 LabelEncoder 103
29 StandardScaler 106
30 Missing Value Handling 109

Part H · Practical Projects

31 Project 1: House Price Prediction (Regression) 114
32 Project 2: Loan Approval Prediction (Classification) 117
33 Project 3: Customer Segmentation (Clustering) 120
34 Project 4: Comparing Multiple ML Models 123

Part I · Interview Preparation

35 Interview Q&A · ML Fundamentals 127
36 Interview Q&A · Linear and Logistic Regression 132
37 Interview Q&A · KNN, Decision Tree and Naive Bayes 137
38 Interview Q&A · Support Vector Machine 142
39 Interview Q&A · Evaluation Metrics 144
40 Interview Q&A · Ensemble Learning 148
41 Interview Q&A · Model Tuning and Cross-Validation 153
42 Interview Q&A · Unsupervised Learning: K-Means, DBSCAN, PCA 156
43 Interview Q&A · Data Preprocessing 160
44 Interview Q&A · Scenario-Based Questions 166
45 Interview Q&A · Mixed Rapid-Fire Round 171

Part J · Reference

46 Formula Cheat Sheet 175
47 References and Further Reading 178

START HERE

How to Use This Handbook

What this handbook is

A beginner-friendly but technically accurate guide to classical Machine Learning with **scikit-learn**. Every topic is explained in simple English *and* in Hinglish, with diagrams, formulas worked out on small numbers, runnable Python, and interview questions.

The four volumes

Volume	Focus	Topics
1	Fundamentals	Linear & Logistic Regression, KNN, Decision Tree, Gaussian Naive Bayes, SVM, evaluation metrics, cross-validation, GridSearchCV, RandomizedSearchCV, K-Means, DBSCAN, PCA
2	Ensemble Learning	Stacking, Bagging, Random Forest, AdaBoost, Gradient Boosting
3	Data Preprocessing	IQR outliers, Pipeline, ColumnTransformer, Ordinal / OneHot / Label encoders, StandardScaler, missing values
4	Projects & Interviews	4 end-to-end projects, 250+ interview questions with Hinglish explanations, formula cheat sheet, references

How every topic chapter is organised

Each topic follows the same 11 parts so you always know where to look: **(1)** definition in simple English, **(2)** Hinglish explanation, **(3)** step-by-step working with a flowchart, **(4)** diagram, **(5)** formulas with numerical examples, **(6)** real-world applications, **(7)** when to use and when not to use, **(8)** advantages, disadvantages and limitations, **(9)** complete scikit-learn code, **(10)** common mistakes and related concepts, **(11)** quick revision notes (the chapter summary).

Colour guide

Orange boxes are **Hinglish** explanations. Green boxes are tips, red boxes are warnings or common mistakes, blue boxes are worked examples, purple boxes are chapter summaries. Green *Output* panels under code are the **real output** captured when this book was built, so numbers you see in the text come from actually running the code.

Running the code

```
pip install numpy pandas scikit-learn matplotlib
```

All examples use datasets that ship with scikit-learn (Iris, Wine, Breast Cancer, Diabetes) or **synthetic data generated in the code itself**, so nothing needs to be downloaded. The synthetic projects use fixed random seeds; swap in your own CSV with `pd.read_csv(...)` to use real data. Exact decimals can differ slightly across scikit-learn versions; this book was built with scikit-learn 1.9.1.

Golden rules used in every example

1. **Split first.** Create the train/test split before looking at statistics of the data.

2. **Fit on train only.** Scalers, imputers, encoders and feature selectors learn *only* from the training data (`fit`), then just apply (`transform`) to validation/test data.
3. **Use a Pipeline.** Putting preprocessing and model in one `Pipeline` makes rules 1 and 2 automatic, including inside cross-validation.
4. **Touch the test set once.** Tune with cross-validation on the training set; report the test score only at the end.
5. **Fix `random_state`.** Results become reproducible and debuggable.

> **Data leakage**
>
> Data leakage means information from outside the training data (usually from the test set or the future) sneaks into model training. It makes scores look great in the notebook and terrible in production. The examples in this book avoid it by design, and each preprocessing chapter shows the leaking version next to the safe version.

Notation

Symbol	Meaning
$x, \mathbf{x}$	a feature / feature vector
y, $\hat{y}$	true target / predicted target
n	number of samples
$\bar{y}$	mean of y
$\beta_0, \beta_1, \mathbf{w}, b$	intercept and weights (model parameters)
λ, C, α	hyper-parameters (set by you, not learned)

PART A

Foundations

1. Machine Learning Foundations

CHAPTER 1

Machine Learning Foundations

1. Definition (Simple English)

Machine Learning

Machine Learning (ML) is teaching a computer to find patterns in data so it can make predictions on new data, instead of us writing every rule by hand.

Three families: **supervised** (data has answers: predict price, spam/not-spam), **unsupervised** (no answers: find groups or compress features) and **reinforcement** (learn by reward; not covered here).

2. Hinglish Explanation

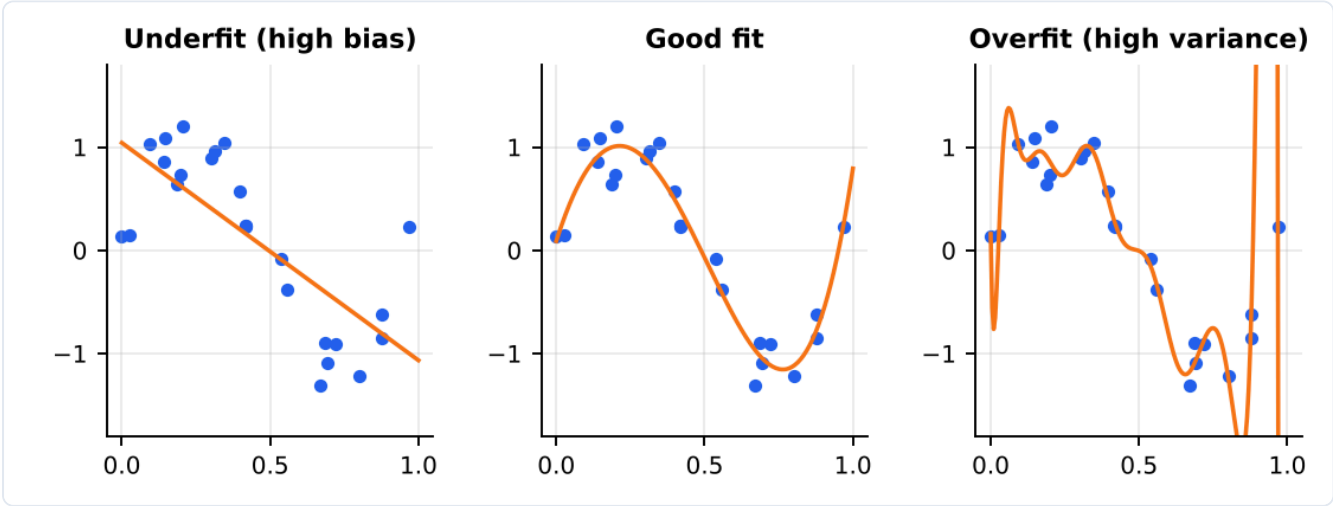
Hinglish mein samjho

Normal programming mein hum rules likhte hain: "agar email mein 'lottery' hai to spam". ML mein hum computer ko hazaaron examples dikhate hain aur woh **khud rules seekh leta hai**. *Supervised* matlab teacher ke saath padhai (har example ka answer diya hua). *Unsupervised* matlab bina teacher ke, computer khud similar cheezein group karta hai.

3. The ML Workflow



4. Bias, Variance, Overfitting



Same data, three models: too simple, balanced, too flexible.

$$\text{Expected error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible noise}$$

Reading the picture

Training error keeps falling as a model gets flexible; test error falls and then rises. If train score is high but test score is low, the model **overfits**. If both are low, it **underfits**.

5. When to Use ML, and Its Limits

Good fit	Poor fit
Rules are hard to write; plenty of representative data	Tiny data, or a simple fixed rule already works
Patterns are stable over time	Data changes meaning quickly and no monitoring exists

6. Python: Train/Test Split Done Right

PYTHON

```python
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.dummy import DummyClassifier
from sklearn.tree import DecisionTreeClassifier

X, y = load_iris(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.25, stratify=y, random_state=0)
print("train/test sizes:", len(X_tr), len(X_te))
base = DummyClassifier(strategy="most_frequent").fit(X_tr, y_tr)
tree = DecisionTreeClassifier(max_depth=3, random_state=0).fit(X_tr, y_tr)
print("baseline accuracy:", round(base.score(X_te, y_te), 3))
print("tree accuracy    :", round(tree.score(X_te, y_te), 3))
```

OUTPUT

```
train/test sizes: 112 38
baseline accuracy: 0.316
tree accuracy    : 0.947
```

7. Common Mistakes

Avoid

Preprocessing before splitting; tuning on the test set; no baseline; judging an imbalanced problem by accuracy; random split on time-ordered data (use time-based split).

8. Quick Revision Notes

Chapter Summary

- Supervised = labelled data (regression: number, classification: class). Unsupervised = no labels.
- Always compare against a dummy baseline.
- Overfit: train good, test bad. Underfit: both bad.
- Split first, fit preprocessing on train only, test set used once.

PART B

Supervised Learning

2. Linear Regression

3. Logistic Regression

4. K-Nearest Neighbors (KNN)

5. Decision Tree

6. Gaussian Naive Bayes

7. Support Vector Machine (SVM)

CHAPTER 2

Linear Regression

1. Definition (Simple English)

In one line

Linear Regression predicts a *number* by drawing the best straight line (or flat plane, when there are many features) through the data. The line is chosen so that the total squared distance between the line and the real points is as small as possible.

It is the simplest *supervised regression* model: you give it examples with known answers (house size → price), and it learns a rule it can apply to new inputs.

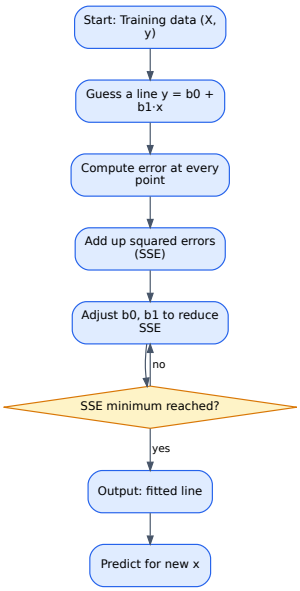
2. Hinglish Explanation

Hinglish mein samjho

Socho aapke paas 5 ghar ka data hai: ghar jitna bada, price utna zyada. Linear Regression bas ek **seedhi line** khinchti hai jo in sab points ke sabse paas se guzre. Phir jab naya ghar aata hai, hum uska size line par dekhte hain aur price padh lete hain.

Line ka equation hota hai $y = \beta_0 + \beta_1 x$. Yahan β_1 **slope** hai (size 1 unit badhne par price kitna badhta hai) aur β_0 **intercept** hai (jab size zero ho tab ki starting value). Model ka kaam bas yeh dono numbers dhoondhna hai, aur "best" ka matlab hai **errors ke squares ka total sabse chhota**. Isse *Ordinary Least Squares (OLS)* kehte hain.

3. How It Works, Step by Step



1. Collect labelled data. 2. Pick the model form $\hat{y} = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p$. 3. Measure each error (residual) $e_i = y_i - \hat{y}_i$. 4. Choose the parameters that minimise $\sum e_i^2$. scikit-learn solves this directly with linear algebra (the *normal equation* / SVD), so no manual loop is needed; for huge data, gradient descent (`SGDRegressor`) does the same job iteratively.

4. Diagram



Left: the best-fit line and its residuals. Right: SSE is a smooth bowl, so there is exactly one best slope.

5. Formulas with a Numerical Example

$$\hat{y} = \beta_0 + \beta_1 x, \qquad \beta_1 = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}, \qquad \beta_0 = \bar{y} - \beta_1 \bar{x}$$

For many features the closed-form solution is the **normal equation** $\hat{\beta} = (X^\top X)^{-1} X^\top \mathbf{y}$. Training minimises $\text{SSE} = \sum_i (y_i - \hat{y}_i)^2$.

Worked example

Data: $x = [1, 2, 3, 4, 5]$, $y = [3, 5, 7, 9, 12]$. Then $\bar{x} = 3$, $\bar{y} = 7.2$, $\sum (x - \bar{x})(y - \bar{y}) = 22$, $\sum (x - \bar{x})^2 = 10$.

$\beta_1 = 22/10 = 2.2$, $\quad \beta_0 = 7.2 - 2.2 \times 3 = 0.6$. Prediction for $x = 6$: $0.6 + 2.2 \times 6 = 13.8$. Residuals are $[0.2, 0, -0.2, -0.4, 0.4]$, so $\text{SSE} = 0.40$ and $R^2 = 1 - 0.40/48.8 \approx 0.992$.

PYTHON

```
import numpy as np
x = np.array([1, 2, 3, 4, 5]); y = np.array([3, 5, 7, 9, 12])
b1 = ((x - x.mean()) * (y - y.mean())).sum() / ((x - x.mean()) ** 2).sum()
b0 = y.mean() - b1 * x.mean()
pred = b0 + b1 * x
sse = ((y - pred) ** 2).sum(); sst = ((y - y.mean()) ** 2).sum()
print(f"slope={b1:.2f}, intercept={b0:.2f}, predict(6)={b0 + b1 * 6:.1f}")
print(f"SSE={sse:.2f}, SST={sst:.1f}, R2={1 - sse / sst:.4f}")
```

OUTPUT

```
slope=2.20, intercept=0.60, predict(6)=13.8
SSE=0.40, SST=48.8, R2=0.9918
```

Regularised variants. *Ridge* adds $\lambda \sum \beta_j^2$ to the loss (shrinks weights, handles correlated features). *Lasso* adds $\lambda \sum |\beta_j|$ (can set weights exactly to 0, so it also selects features). *ElasticNet* mixes both.

Assumptions (remember LINE): **L**inear relationship, **I**ndependent errors, **N**ormally distributed errors (needed only for confidence intervals), **E**qual variance of errors (homoscedasticity). Features should not be perfectly correlated with each other (no multicollinearity).

6. Real-World Applications

- Predicting house prices from area, rooms and location.
- Forecasting sales from advertising spend; estimating demand from price.
- Estimating delivery time from distance and traffic score.
- A quick, explainable **baseline** before trying complex models in almost any regression problem.

7. When to Use and When Not to Use

Use it when	Avoid it when
The relationship is roughly linear	The relationship is strongly curved (unless you add polynomial features)
You need an interpretable model (each coefficient has a meaning)	Data has many extreme outliers (squared error is sensitive to them)
You need a fast baseline or have little data	Features are highly correlated and you use plain OLS (use Ridge/Lasso)
Training must be very cheap	The target is a category (use Logistic Regression instead)

8. Advantages, Disadvantages and Limitations

Advantages	Disadvantages / limitations
Simple, fast, no hyper-parameters in plain OLS	Can only capture linear patterns
Highly interpretable coefficients	Outliers pull the line strongly
Works well as a baseline	Needs scaling for fair coefficient comparison and for regularised versions
Closed-form solution, stable	Extrapolates blindly outside the training range; sensitive to multicollinearity

9. Python Example (scikit-learn)

PYTHON

```python
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import r2_score, mean_absolute_error, root_mean_squared_error

X, y = load_diabetes(return_X_y=True, as_frame=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

models = {
    "OLS":   make_pipeline(StandardScaler(), LinearRegression()),
    "Ridge": make_pipeline(StandardScaler(), Ridge(alpha=10)),
    "Lasso": make_pipeline(StandardScaler(), Lasso(alpha=1.0)),
}
for name, m in models.items():
    m.fit(X_train, y_train)                    # scaler learns from TRAIN only
    p = m.predict(X_test)
    print(f"{name:5s} R2={r2_score(y_test, p):.3f}  MAE={mean_absolute_error(y_test, p):.1f}  "
          f"RMSE={root_mean_squared_error(y_test, p):.1f}")

ols = models["OLS"]
coefs = dict(zip(X.columns, ols[-1].coef_.round(1)))
print("OLS intercept:", round(ols[-1].intercept_, 1)); print("coefficients:", coefs)
print("Lasso zeroed features:", [c for c, w in zip(X.columns, models["Lasso"][-1].coef_) if w == 0])
```

OUTPUT

```
OLS   R2=0.453  MAE=42.8  RMSE=53.9
Ridge R2=0.457  MAE=42.9  RMSE=53.6
Lasso R2=0.467  MAE=42.8  RMSE=53.1
OLS intercept: 153.7
coefficients: {'age': np.float64(1.8), 'sex': np.float64(-11.5), 'bmi': np.float64(25.6), 'bp': np.float64(16.8),
's1': np.float64(-44.4), 's2': np.float64(24.6), 's3': np.float64(7.7), 's4': np.float64(13.1), 's5':
np.float64(35.2), 's6': np.float64(2.4)}
Lasso zeroed features: ['s2']
```

10. Common Mistakes and Related Concepts

Common mistakes

- Scaling with statistics of the *whole* dataset before splitting (leakage). Use a Pipeline.
- Reading a large coefficient as "important" when features are on different scales.
- Trusting a high R^2 on training data; always check the test score.
- Extrapolating far beyond the data range, and ignoring outliers or multicollinearity.

Related: Polynomial regression (`PolynomialFeatures`), Ridge/Lasso/ElasticNet, Logistic Regression (classification cousin), Gradient Descent, Regression metrics (next chapters).

11. Quick Revision Notes

Chapter Summary

- Model: $\hat{y} = \beta_0 + \beta_1 x_1 + \ldots$; fitted by minimising squared errors (OLS).
- Slope $= \mathrm{cov}(x, y)/\mathrm{var}(x)$; intercept $= \bar{y} - \beta_1 \bar{x}$.
- Regularisation: Ridge shrinks, Lasso zeroes out, ElasticNet does both.
- Assumptions: linearity, independent errors, constant variance, (normal errors).
- Scale features inside a Pipeline; judge on test data using R^2, MAE, RMSE.

CHAPTER 3

Logistic Regression

1. Definition (Simple English)

In one line

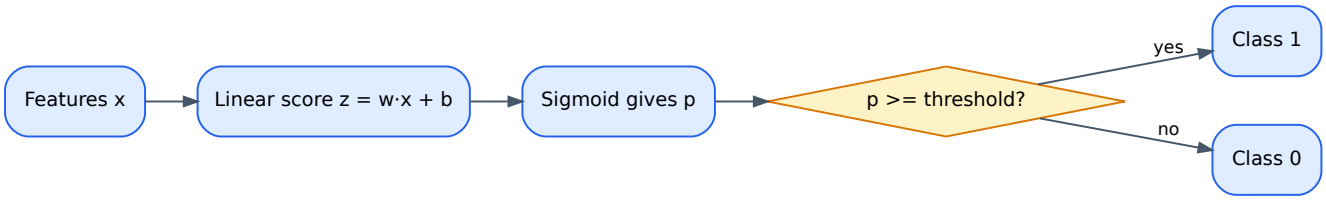
Logistic Regression is a classification model that predicts the *probability* that a sample belongs to a class, by passing a linear score through an S-shaped sigmoid function. Despite the name, it is used for classification.

2. Hinglish Explanation

Hinglish mein samjho

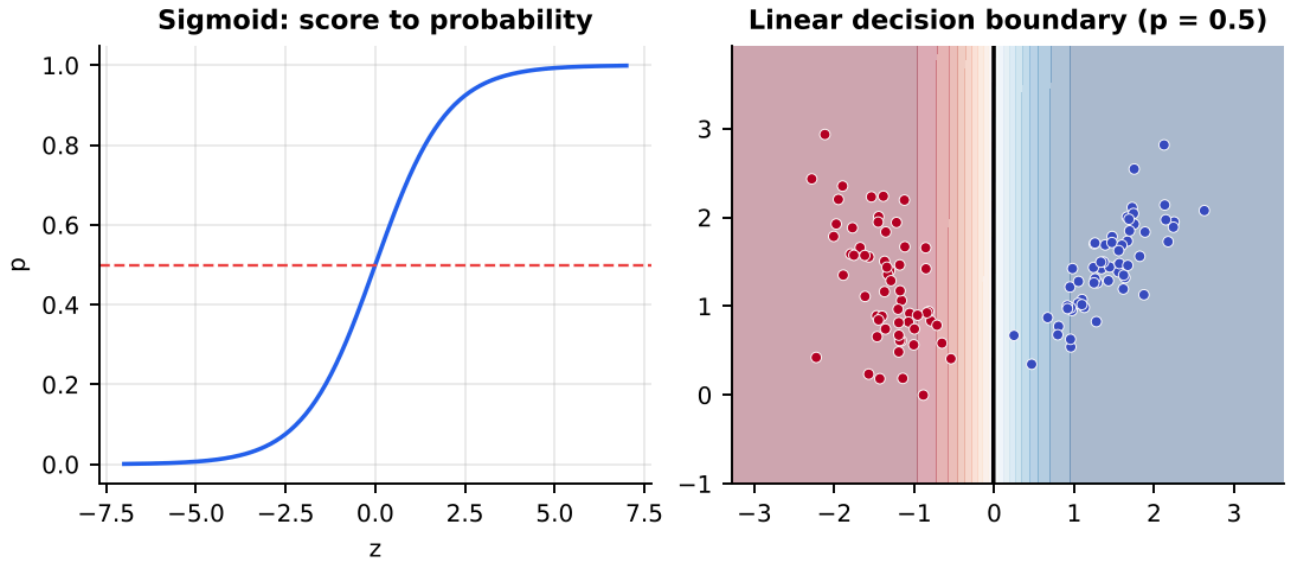
Linear Regression koi bhi number de sakta hai (-5, 300...), lekin probability 0 se 1 ke beech honi chahiye. Isliye hum linear score z ko **sigmoid** se guzarte hain, jo kisi bhi number ko 0-1 mein daba deta hai. Agar probability ≥ 0.5 to class 1, warna class 0. Threshold 0.5 fix nahi hai; fraud jaise cases mein hum ise kam rakh sakte hain taaki zyada fraud pakde jayein.

3. How It Works, Step by Step



Training minimises **log loss** (cross-entropy) using an iterative solver (default `lbfgs`); there is no closed-form solution. scikit-learn applies **L2 regularisation** by default (`C` is the *inverse* of regularisation strength: smaller `C` = stronger penalty).

4. Diagram



Left: the sigmoid. Right: probabilities fade smoothly; the black line is where p = 0.5.

5. Formulas with a Numerical Example

$$z = \mathbf{w}^\top \mathbf{x} + b, \qquad p = \sigma(z) = \frac{1}{1+e^{-z}}, \qquad \text{odds} = \frac{p}{1-p}, \quad \ln\frac{p}{1-p} = z$$

$$\text{Log loss} = -\frac{1}{n}\sum_i \Big[y_i \ln p_i + (1-y_i)\ln(1-p_i)\Big]$$

Worked example

A loan model has $w = 0.8$ (per lakh of income), $b = -4$. For income $= 6$: $z = 0.8 \times 6 - 4 = 0.8$, $p = 1/(1+e^{-0.8}) = 0.69$, so class 1 (approve) at threshold 0.5. Odds $= 0.69/0.31 \approx 2.2$. Each extra lakh multiplies the odds by $e^{0.8} \approx 2.23$. Log loss for one sample with $y = 1$: $-\ln 0.69 = 0.37$; if the model had said $p = 0.1$, loss $= -\ln 0.1 = 2.30$ (confident mistakes are punished hard).

PYTHON

```python
import numpy as np
z = 0.8 * 6 - 4
p = 1 / (1 + np.exp(-z))
print(f"z={z:.2f} p={p:.3f} odds={p/(1-p):.2f} odds-multiplier per unit={np.exp(0.8):.2f}")
print(f"loss(y=1,p={p:.2f})={-np.log(p):.3f}   loss(y=1,p=0.1)={-np.log(0.1):.3f}")
```

OUTPUT

```
z=0.80 p=0.690 odds=2.23 odds-multiplier per unit=2.23
loss(y=1,p=0.69)=0.371   loss(y=1,p=0.1)=2.303
```

6. Real-World Applications

Spam detection, loan default, disease risk screening, customer churn, click-through prediction, and as the default **baseline classifier**. Its probabilities are usually reasonably calibrated, useful when you need a risk score.

7. When to Use and When Not to Use

Use it when	Avoid it when
Classes are roughly linearly separable	Boundary is highly non-linear (use trees, SVM-RBF, boosting) unless you engineer features
You need probabilities and explainable coefficients	Many redundant features and no regularisation
Fast training, high-dimensional sparse data (text)	Features are unscaled and you rely on regularisation

8. Advantages, Disadvantages and Limitations

Advantages	Disadvantages
Fast, simple, interpretable (odds ratios)	Linear decision boundary only
Outputs probabilities; supports L1/L2/ElasticNet	Sensitive to outliers and multicollinearity
Works with multiclass (softmax/one-vs-rest)	Needs feature scaling and encoding of categories
Good baseline for imbalanced data with `class_weight`	Can underperform tree ensembles on complex tabular data

9. Python Example (scikit-learn)

PYTHON

```python
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, roc_auc_score, classification_report

X, y = load_breast_cancer(return_X_y=True, as_frame=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, stratify=y, random_state=42)
clf = make_pipeline(StandardScaler(), LogisticRegression(C=1.0, max_iter=1000))
clf.fit(X_tr, y_tr)
proba = clf.predict_proba(X_te)[:, 1]
pred = (proba >= 0.5).astype(int)
print("accuracy:", round(accuracy_score(y_te, pred), 3), " ROC-AUC:", round(roc_auc_score(y_te, proba), 3))
print(classification_report(y_te, pred, target_names=["malignant", "benign"]))
top = sorted(zip(X.columns, clf[-1].coef_[0]), key=lambda t: -abs(t[1]))[:3]
print("largest |coefficients|:", [(n, round(w, 2)) for n, w in top])
```

OUTPUT

```
accuracy: 0.982  ROC-AUC: 0.995
              precision    recall  f1-score   support

   malignant       0.98      0.98      0.98        42
      benign       0.99      0.99      0.99        72

    accuracy                           0.98       114
   macro avg       0.98      0.98      0.98       114
weighted avg       0.98      0.98      0.98       114

largest |coefficients|: [('worst texture', np.float64(-1.26)), ('radius error', np.float64(-1.08)), ('worst concave
points', np.float64(-0.95))]
```

10. Common Mistakes and Related Concepts

Common mistakes

- Forgetting to scale features (slow convergence, uneven regularisation).
- Treating `C` like regularisation strength (it is the inverse).
- Reporting accuracy on imbalanced data; use precision/recall/ROC-AUC.
- Assuming 0.5 is the right threshold for every business problem.

Related: Softmax regression, LDA, SVM, Naive Bayes, ROC/PR curves, probability calibration, regularisation.

11. Quick Revision Notes

Chapter Summary

- $p = \sigma(\mathbf{w}^\top \mathbf{x} + b)$; trained by minimising log loss.
- Coefficient w = change in log-odds per unit of x; e^w = odds multiplier.
- Decision boundary is linear; threshold is tunable.
- `C` small = more regularisation. Scale features. Prefer ROC-AUC/F1 on imbalanced data.

CHAPTER 4

K-Nearest Neighbors (KNN)

1. Definition (Simple English)

In one line

KNN predicts for a new point by looking at the k closest training points and taking a vote (classification) or an average (regression). It is a *lazy learner*: it stores the data and does the work at prediction time.

2. Hinglish Explanation

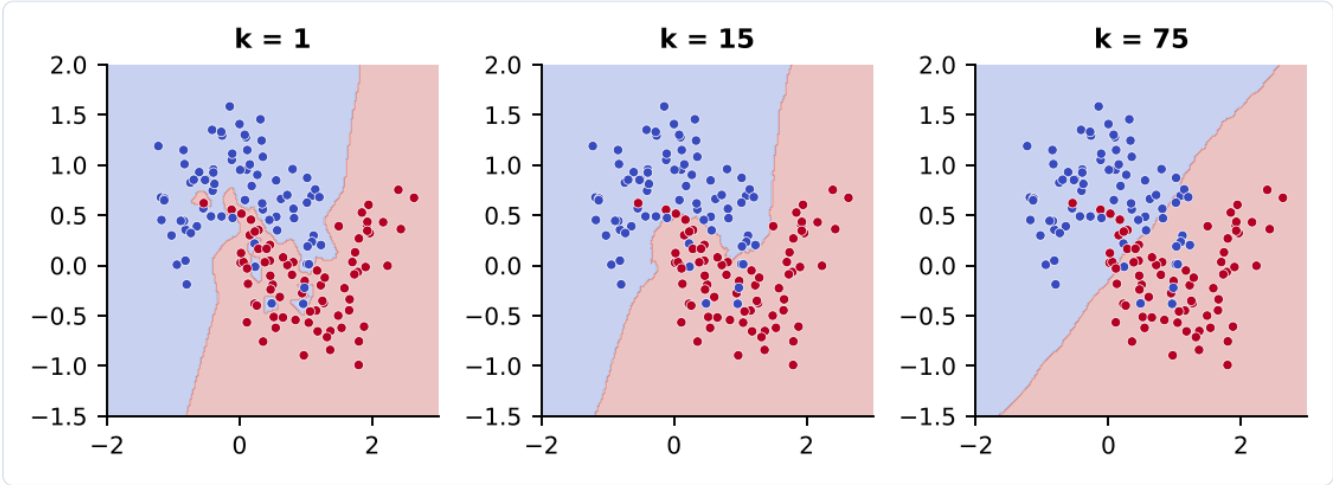
Hinglish mein samjho

Naye shehar mein restaurant chunna ho to aap kya karte ho? Apne 5 sabse paas wale doston se poochte ho, aur jo majority bole woh le lete ho. KNN bhi yahi karta hai: naye point ke k sabse **nazdeeki padosi** dhoondho, unka majority class (ya average value) answer hai. "Nazdeeki" ka matlab distance, isliye **scaling bahut zaroori** hai.

3. How It Works, Step by Step



4. Diagram



k=1 memorises noise (overfit), k=15 is smooth, k=75 is too rigid (underfit).

5. Formulas with a Numerical Example

$$d_{\text{Euclid}}(\mathbf{a},\mathbf{b}) = \sqrt{\sum_j (a_j - b_j)^2}, \qquad d_{\text{Manhattan}} = \sum_j |a_j - b_j|$$

Worked example

Training points: A=(1,1, class 0), B=(2,1, class 0), C=(4,5, class 1), D=(5,5, class 1). New point $q = (2, 2)$. Distances: A $= \sqrt{2} = 1.41$, B $= 1$, C $= \sqrt{13} = 3.61$, D $= \sqrt{18} = 4.24$. For $k = 3$ the neighbours are B, A, C, so votes are 0,0,1 and the prediction is class 0. With *distance weighting* ($w = 1/d$) class 0 gets $1 + 0.71 = 1.71$ vs class 1 gets 0.28, still class 0.

PYTHON

```python
import numpy as np
P = np.array([[1, 1], [2, 1], [4, 5], [5, 5]]); lab = np.array([0, 0, 1, 1]); q = np.array([2, 2])
d = np.sqrt(((P - q) ** 2).sum(axis=1)); order = d.argsort()[:3]
print("distances:", d.round(2), " nearest 3 idx:", order, " votes:", lab[order])
print("prediction:", np.bincount(lab[order]).argmax())
```

OUTPUT

```
distances: [1.41 1.   3.61 4.24]  nearest 3 idx: [1 0 2]  votes: [0 0 1]
prediction: 0
```

6. Real-World Applications

Recommendation by similarity, handwriting digits, anomaly detection (far from all neighbours), imputing missing values (`KNNImputer`), and quick baselines on small datasets.

7. When to Use and When Not to Use

Use it when	Avoid it when
Small/medium data, few features	Huge data (prediction is slow: it scans the training set)
Decision boundary is irregular	Many features (curse of dimensionality: distances lose meaning)
You want a simple, no-training baseline	Features are unscaled or mostly irrelevant

8. Advantages, Disadvantages and Limitations

Advantages	Disadvantages
No training, very intuitive	Slow, memory-hungry prediction
Non-linear boundaries naturally	Sensitive to scale, noise and irrelevant features
Works for classification and regression	Struggles with imbalanced classes (majority dominates)

9. Python Example (scikit-learn)

PYTHON

```python
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier

X, y = load_wine(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.25, stratify=y, random_state=0)
for scale in (False, True):
    for k in (1, 5, 15):
        steps = ([StandardScaler()] if scale else []) + [KNeighborsClassifier(n_neighbors=k)]
        cv = cross_val_score(make_pipeline(*steps), X_tr, y_tr, cv=5).mean()
        print(f"scaled={scale!s:5}  k={k:2d}  CV accuracy={cv:.3f}")
best = make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=5, weights="distance")).fit(X_tr, y_tr)
print("test accuracy:", round(best.score(X_te, y_te), 3))
```

OUTPUT

```
scaled=False  k= 1  CV accuracy=0.736
scaled=False  k= 5  CV accuracy=0.721
scaled=False  k=15  CV accuracy=0.721
scaled=True   k= 1  CV accuracy=0.918
scaled=True   k= 5  CV accuracy=0.940
scaled=True   k=15  CV accuracy=0.932
test accuracy: 0.933
```

10. Common Mistakes and Related Concepts

Common mistakes

- Not scaling (the feature with the largest range dominates distance).
- Picking an even k in binary problems (ties); choosing k on the test set instead of CV.
- Using KNN on 100+ noisy features.

Related: Distance metrics (Minkowski, cosine), KD-tree/Ball-tree, `RadiusNeighbors`, KNNImputer, curse of dimensionality.

11. Quick Revision Notes

Chapter Summary

- Predict from the k nearest training points (vote/average); no training phase.
- Small k = low bias, high variance; large k = the opposite. Tune k by CV.
- Always scale features; use odd k for binary tasks.
- Slow at prediction and weak in high dimensions.

CHAPTER 5

Decision Tree

1. Definition (Simple English)

In one line

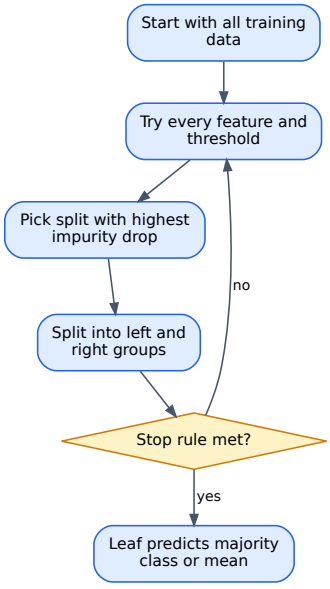
A **Decision Tree** predicts by asking a series of yes/no questions about the features, splitting the data into smaller and purer groups until each leaf gives an answer.

2. Hinglish Explanation

Hinglish mein samjho

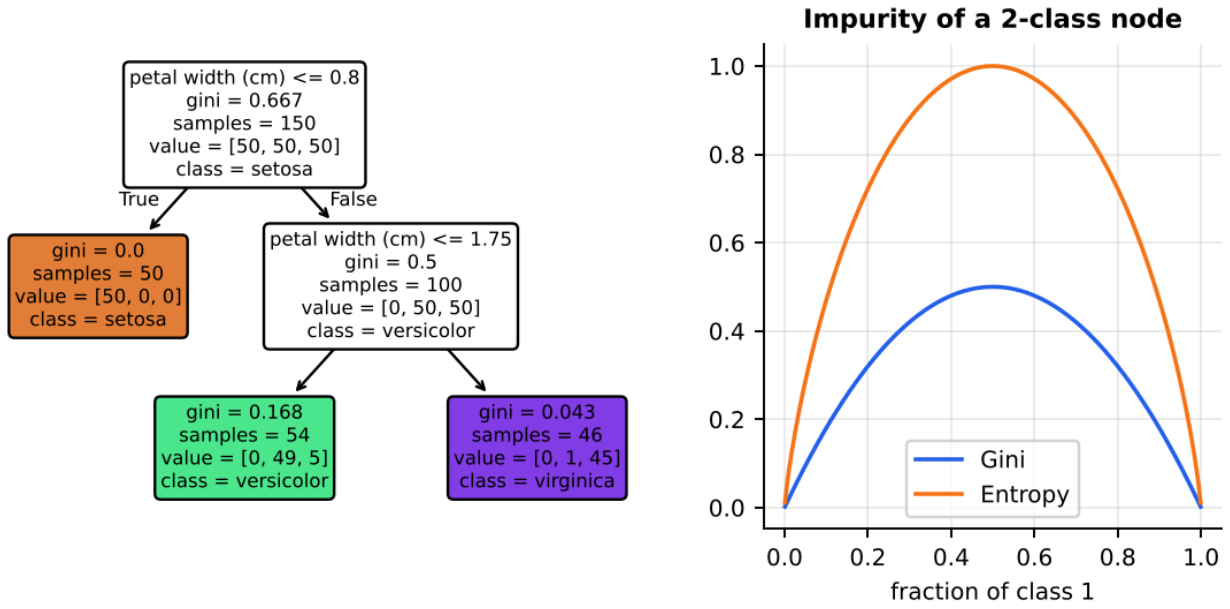
Yeh bilkul "20 questions" game jaisa hai. "Petal length 2.5 se chhoti hai? Haan to setosa. Nahi to agla sawaal..." Tree har step par woh sawaal chunta hai jo data ko sabse **saaf (pure)** groups mein baant de. Purity naapne ke liye **Gini** ya **Entropy** use hota hai. Tree ko bahut gehra chhod do to woh sab kuch ratta maar leta hai (overfit), isliye `max_depth` ya pruning lagate hain.

3. How It Works, Step by Step



Stop rules: `max_depth`, `min_samples_split`, `min_samples_leaf`, or a node that is already pure. scikit-learn uses the CART algorithm (binary splits).

4. Diagram



A depth-2 tree on Iris (left) and the two impurity measures (right): both are 0 for pure nodes and maximal at 50/50.

5. Formulas with a Numerical Example

$$\text{Gini} = 1 - \sum_k p_k^2, \qquad \text{Entropy} = -\sum_k p_k \log_2 p_k, \qquad \text{Gain} = I(\text{parent}) - \sum_c \frac{n_c}{n} I(c)$$

Worked example

A node has 10 samples: 6 "yes", 4 "no". Gini $= 1 - (0.6^2 + 0.4^2) = 0.48$. A split sends 5 samples left (5 yes, 0 no: Gini 0) and 5 right (1 yes, 4 no: Gini $= 1 - (0.2^2 + 0.8^2) = 0.32$). Weighted child impurity $= 0.5 \times 0 + 0.5 \times 0.32 = 0.16$. Gain $= 0.48 - 0.16 = 0.32$. The tree picks the split with the largest gain.

PYTHON

```python
import numpy as np
def gini(c): p = np.array(c) / sum(c); return 1 - (p ** 2).sum()
def entropy(c): p = np.array(c) / sum(c); p = p[p > 0]; return -(p * np.log2(p)).sum()
parent, left, right = [6, 4], [5, 0], [1, 4]
w = lambda f: 0.5 * f(left) + 0.5 * f(right)
print(f"Gini parent={gini(parent):.2f} children={w(gini):.2f} gain={gini(parent) - w(gini):.2f}")
print(f"Entropy parent={entropy(parent):.3f} children={w(entropy):.3f} gain={entropy(parent) - w(entropy):.3f}")
```

OUTPUT

```
Gini parent=0.48 children=0.16 gain=0.32
Entropy parent=0.971 children=0.361 gain=0.610
```

6. Real-World Applications

Credit approval rules, medical triage, customer churn reasons, fault diagnosis, and as the **building block** of Random Forest and Gradient Boosting.

7. When to Use and When Not to Use

Use it when	Avoid it when
You need human-readable rules	You need top accuracy alone (use ensembles)
Mixed numeric/categorical, no scaling wanted	Data is small and noisy (tree changes a lot)
Non-linear patterns and interactions	Smooth linear relationships (trees fit them in steps)

8. Advantages, Disadvantages and Limitations

Advantages	Disadvantages
Interpretable, visualisable	Overfits easily; high variance
No scaling needed	Unstable: small data change, different tree
Handles non-linearity and interactions	Cannot extrapolate beyond training range (regression)
Gives feature importances	Biased toward features with many split points

9. Python Example (scikit-learn)

PYTHON

```python
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.metrics import accuracy_score

data = load_breast_cancer()
X_tr, X_te, y_tr, y_te = train_test_split(data.data, data.target, test_size=0.25, stratify=data.target,
random_state=1)
for depth in (1, 3, 5, None):
    t = DecisionTreeClassifier(max_depth=depth, random_state=1).fit(X_tr, y_tr)
    print(f"max_depth={str(depth):4}  train={t.score(X_tr, y_tr):.3f}  test={accuracy_score(y_te,
t.predict(X_te)):.3f}")
t = DecisionTreeClassifier(max_depth=2, random_state=1).fit(X_tr, y_tr)
print(export_text(t, feature_names=list(data.feature_names)))
imp = sorted(zip(data.feature_names, t.feature_importances_), key=lambda z: -z[1])[:3]
print("top importances:", [(n, round(v, 2)) for n, v in imp])
```

OUTPUT

```
max_depth=1     train=0.923  test=0.902
max_depth=3     train=0.972  test=0.944
max_depth=5     train=0.998  test=0.930
max_depth=None  train=1.000  test=0.930
|--- worst concave points <= 0.15
|   |--- worst area <= 929.80
|   |   |--- class: 1
|   |--- worst area >  929.80
|   |   |--- class: 0
|--- worst concave points >  0.15
|   |--- worst radius <= 12.80
|   |   |--- class: 1
|   |--- worst radius >  12.80
|   |   |--- class: 0

top importances: [(np.str_('worst concave points'), np.float64(0.81)), (np.str_('worst area'), np.float64(0.15)),
(np.str_('worst radius'), np.float64(0.03))]
```

10. Common Mistakes and Related Concepts

Common mistakes

- Leaving the tree unlimited and trusting 100% train accuracy.
- Reading `feature_importances_` as causal (it is biased and unstable).
- Label-encoding unordered categories (tree may imply an order); prefer one-hot.

Related: Pruning (`ccp_alpha`), Random Forest, Gradient Boosting, CART vs ID3/C4.5, regression trees.

11. Quick Revision Notes

Chapter Summary

- Greedy, recursive splitting that maximises impurity reduction (Gini/Entropy; MSE for regression).
- Deep tree = low bias/high variance. Control with `max_depth`, `min_samples_leaf`, `ccp_alpha`.
- No scaling needed; interpretable; unstable alone, strong inside ensembles.

CHAPTER 6

Gaussian Naive Bayes

1. Definition (Simple English)

In one line

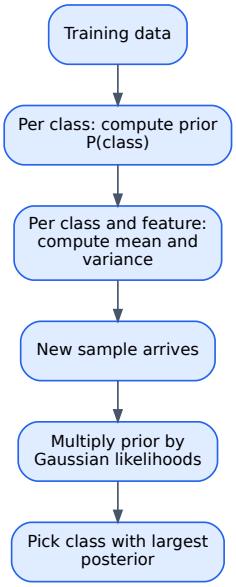
Gaussian Naive Bayes classifies by using Bayes' theorem, assuming that features are independent given the class and that each feature follows a bell-curve (Gaussian) distribution inside each class.

2. Hinglish Explanation

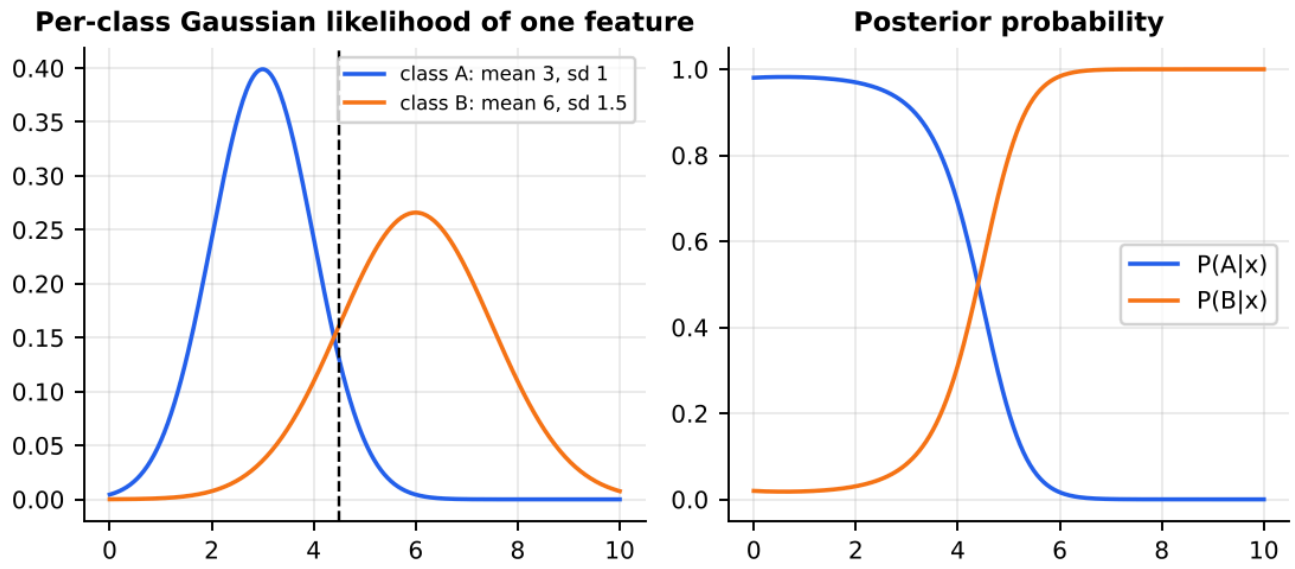
Hinglish mein samjho

Bayes theorem kehta hai: "Naya evidence dekhne ke baad kisi class ki probability kitni hai?" Naive Bayes har class ke liye hisaab lagata hai: *is class mein aise feature values kitne common hain* x *class kitni common hai*. "Naive" isliye kyunki hum maan lete hain ki saare features ek doosre se **independent** hain (jo asal mein aksar sach nahi hota), phir bhi model kamaal chalta hai. "Gaussian" matlab har feature ko class ke andar normal distribution maana gaya hai.

3. How It Works, Step by Step



4. Diagram



Left: each class models the feature as a bell curve. Right: Bayes' rule converts them into posteriors; the boundary is where they cross.

5. Formulas with a Numerical Example

$$P(c \mid \mathbf{x}) \propto P(c) \prod_{j} P(x_j \mid c), \qquad P(x_j \mid c) = \frac{1}{\sqrt{2\pi\sigma_{cj}^2}} \exp\Big(- \frac{(x_j - \mu_{cj})^2}{2\sigma_{cj}^2} \Big)$$

Worked example

One feature, priors $P(A) = P(B) = 0.5$. Class A: $\mu = 3, \sigma = 1$. Class B: $\mu = 6, \sigma = 1.5$. For $x = 4.5$: likelihood A $= 0.1295$, likelihood B $= 0.1613$. Unnormalised posteriors: A $= 0.0648$, B $= 0.0807$. Normalised: $P(A) = 0.445$, $P(B) = 0.555$, so predict **B**.

PYTHON

```
from scipy.stats import norm
x = 4.5; la = norm.pdf(x, 3, 1); lb = norm.pdf(x, 6, 1.5)
pa, pb = 0.5 * la, 0.5 * lb
print(f"likelihoods A={la:.4f} B={lb:.4f}")
print(f"posterior A={pa/(pa+pb):.3f}  B={pb/(pa+pb):.3f}")
```

OUTPUT

```
likelihoods A=0.1295 B=0.1613
posterior A=0.445  B=0.555
```

6. Real-World Applications

Fast baselines for medical screening, real-time classification, text/spam (the multinomial variant), and problems with very little training data.

7. When to Use and When Not to Use

Use it when	Avoid it when
Small data, many features, need speed	Features are strongly correlated
Features are roughly bell-shaped per class	You need well-calibrated probabilities
You need an instant baseline	Strongly non-Gaussian features (transform them or choose another model)

8. Advantages, Disadvantages and Limitations

Advantages	Disadvantages
Extremely fast; trains in one pass	Independence assumption rarely true
Works with little data	Probabilities are over-confident
Few hyper-parameters (`var_smoothing`)	Cannot learn feature interactions

9. Python Example (scikit-learn)

PYTHON

```python
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score

X, y = load_iris(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, stratify=y, random_state=7)
nb = GaussianNB().fit(X_tr, y_tr)
print("priors:", nb.class_prior_.round(3))
print("class means (rows=class):\n", nb.theta_.round(2))
print("test accuracy:", round(accuracy_score(y_te, nb.predict(X_te)), 3))
print("5-fold CV on train:", cross_val_score(GaussianNB(), X_tr, y_tr, cv=5).mean().round(3))
print("probabilities for first test row:", nb.predict_proba(X_te[:1]).round(3))
```

OUTPUT

```
priors: [0.333 0.333 0.333]
class means (rows=class):
 [[5.   3.43 1.44 0.23]
 [5.96 2.76 4.28 1.33]
 [6.71 3.02 5.68 2.06]]
test accuracy: 0.978
5-fold CV on train: 0.943
probabilities for first test row: [[0. 1. 0.]]
```

10. Common Mistakes and Related Concepts

Common mistakes

- Using GaussianNB on count data (use MultinomialNB) or binary data (BernoulliNB).
- Trusting its probabilities for risk decisions without calibration.
- Believing scaling is needed (it is not; each feature is modelled separately).

Related: Bayes' theorem, MultinomialNB, BernoulliNB, LDA/QDA, probability calibration.

11. Quick Revision Notes

Chapter Summary

- Posterior $\propto$ prior $\times$ product of per-feature Gaussian likelihoods.
- "Naive" = features independent given class. Still works well surprisingly often.
- Fast, good for small data; over-confident probabilities. `var_smoothing` stabilises variances.

CHAPTER 7

Support Vector Machine (SVM)

1. Definition (Simple English)

In one line

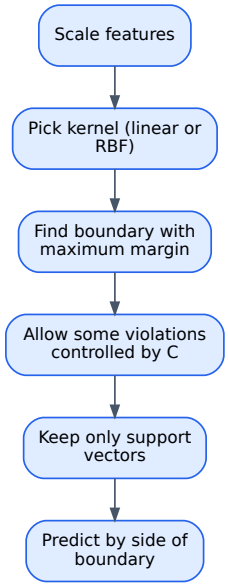
An **SVM** finds the decision boundary that separates classes with the **widest possible margin**. Only the points closest to the boundary (the *support vectors*) decide where it goes. With a kernel it can draw curved boundaries.

2. Hinglish Explanation

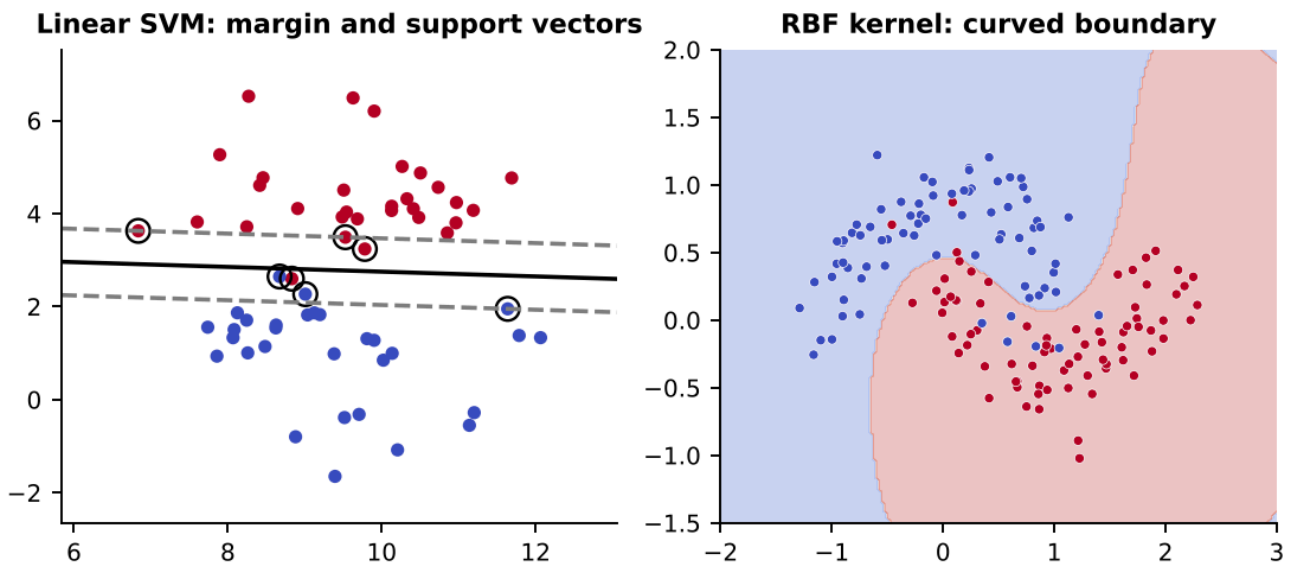
Hinglish mein samjho

Do groups ke beech sadak banani hai. Aap chahoge ki sadak itni **chaudi** ho ki dono taraf ke sabse paas ke ghar bhi door rahein. Yeh chaudai hi **margin** hai, aur sabse paas ke ghar **support vectors** hain. Agar data seedhi line se alag nahi hota, to **kernel trick** data ko higher dimension mein le jata hai jahan seedhi line kaam kar jaye. `C` batata hai ki galtiyan kitni bardasht karni hain: bada C = kam galti (overfit ka khatra), chhota C = chaudi margin (zyada smooth).

3. How It Works, Step by Step



4. Diagram



Left: circled points are support vectors; dashed lines mark the margin. Right: the RBF kernel separates the moons.

5. Formulas with a Numerical Example

$$\text{Boundary: } \mathbf{w}^\top \mathbf{x} + b = 0, \qquad \text{margin width} = \frac{2}{\|\mathbf{w}\|}, \qquad \min_{\mathbf{w},b} \tfrac{1}{2}\|\mathbf{w}\|^2 + C\sum_i \xi_i$$

$$K_{\text{RBF}}(\mathbf{a},\mathbf{b}) = \exp(-\gamma\|\mathbf{a}-\mathbf{b}\|^2)$$

Worked example

Suppose the learned boundary is $2x_1 + x_2 - 8 = 0$ so $\mathbf{w} = (2,1)$, $b = -8$. Then $\|\mathbf{w}\| = \sqrt{5} = 2.236$ and margin width $= 2/2.236 = 0.894$. For a new point $(5,2)$: $2(5) + 2 - 8 = 4 > 0$ so class +1; its distance to the boundary is $4/2.236 = 1.79$. For RBF with $\gamma = 0.5$, two points at distance 2 have similarity $e^{-0.5\times 4} = 0.135$.

PYTHON

```python
import numpy as np
w = np.array([2, 1]); b = -8; pt = np.array([5, 2])
print("margin width:", round(2 / np.linalg.norm(w), 3))
s = w @ pt + b; print("score:", s, "class:", int(np.sign(s)), "distance:", round(s / np.linalg.norm(w), 2))
print("RBF similarity (gamma=0.5, dist=2):", round(float(np.exp(-0.5 * 2 ** 2)), 3))
```

OUTPUT

```
margin width: 0.894
score: 4 class: 1 distance: 1.79
RBF similarity (gamma=0.5, dist=2): 0.135
```

6. Real-World Applications

Text and spam classification, image/handwriting recognition on small datasets, bioinformatics (gene/protein classification), face detection pipelines, and outlier detection (`OneClassSVM`).

7. When to Use and When Not to Use

Use it when	Avoid it when
Small/medium data, many features	Very large data (training is roughly $O(n^2)$ to $O(n^3)$)
Clear margin of separation	Heavily overlapping, noisy classes
Non-linear data with RBF kernel	You need probabilities or interpretability

8. Advantages, Disadvantages and Limitations

Advantages	Disadvantages
Strong in high dimensions	Slow on big data; kernel SVM memory heavy
Kernel trick handles non-linearity	Needs scaling and tuning of `C`, `gamma`
Robust: depends on support vectors only	No native probabilities (`probability=True` adds slow CV)

9. Python Example (scikit-learn)

PYTHON

```python
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

X, y = load_breast_cancer(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.25, stratify=y, random_state=3)
pipe = Pipeline([("scale", StandardScaler()), ("svm", SVC())])
grid = {"svm__kernel": ["linear", "rbf"], "svm__C": [0.1, 1, 10], "svm__gamma": ["scale", 0.01]}
gs = GridSearchCV(pipe, grid, cv=5, n_jobs=1).fit(X_tr, y_tr)
print("best params:", gs.best_params_)
print("CV accuracy:", round(gs.best_score_, 3), " test accuracy:", round(gs.score(X_te, y_te), 3))
print("support vectors per class:", gs.best_estimator_["svm"].n_support_)
```

OUTPUT

```
best params: {'svm__C': 1, 'svm__gamma': 'scale', 'svm__kernel': 'rbf'}
CV accuracy: 0.979  test accuracy: 0.965
support vectors per class: [53 50]
```

10. Common Mistakes and Related Concepts

Common mistakes

- Not scaling features (RBF and margins depend on distances).
- Large `gamma` + large `C` = memorising training data.
- Using kernel SVM on hundreds of thousands of rows (try `LinearSVC` or `SGDClassifier`).

Related: `LinearSVC`, `SVR`, kernel trick, hinge loss, `OneClassSVM`, logistic regression.

11. Quick Revision Notes

Chapter Summary

- Maximise margin $2/\|w\|$; support vectors define the boundary.
- `C`: penalty for errors (high = tight fit). `gamma`: reach of RBF (high = wiggly).
- Scale features, tune `C` / `gamma` with CV; use linear SVM for large sparse data.

PART C

Evaluation Metrics

8. Regression Metrics: R^2, MAE, MSE, RMSE

9. Classification Metrics: Accuracy, Precision, Recall, F1

10. Confusion Matrix and Classification Report

CHAPTER 8

Regression Metrics: R², MAE, MSE, RMSE

1. Definition (Simple English)

In one line

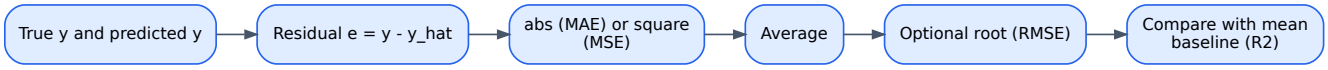
Regression metrics turn the gap between true values y and predictions $\hat{y}$ into a single number: **MAE** (average absolute miss), **MSE** (average squared miss), **RMSE** (square root of MSE, back in original units) and **R²** (share of variation explained compared with just predicting the mean).

2. Hinglish Explanation

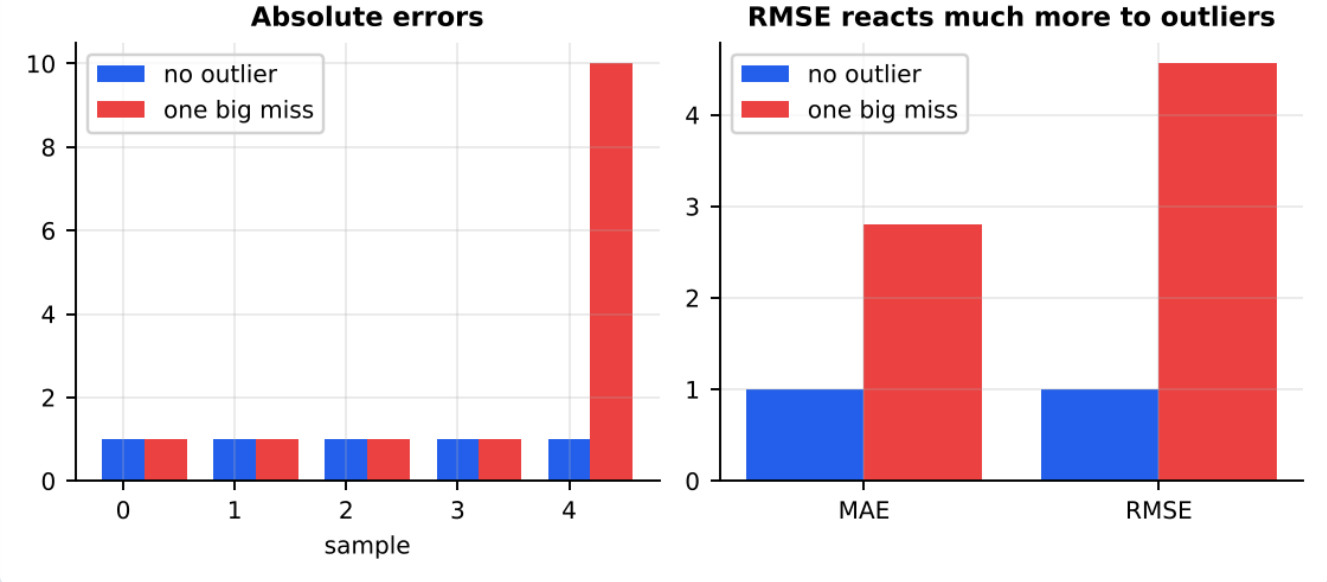
Hinglish mein samjho

Model ne price predict kiya, ab "kitna accha?" kaise bataye. **MAE**: har galti ka positive size lo aur average nikalo ("average 2 lakh ki galti"). **MSE**: galti ka square karke average, to badi galtiyan zyada punish hoti hain. **RMSE**: MSE ka square root, taaki unit wapas lakh/rupee mein aa jaye. **R²**: 1 ka matlab perfect, 0 ka matlab "sirf average bol dete to bhi itna hi hota", negative matlab average se bhi kharab.

3. How It Works, Step by Step



4. Diagram



One large error raises MAE from 1 to 2.8 but RMSE from 1 to 4.5.

5. Formulas with a Numerical Example

$$\text{MAE} = \frac{1}{n}\sum|y_i - \hat{y}_i|, \quad \text{MSE} = \frac{1}{n}\sum(y_i - \hat{y}_i)^2, \quad \text{RMSE} = \sqrt{\text{MSE}}, \quad R^2 = 1 - \frac{\sum(y_i - \hat{y}_i)^2}{\sum(y_i - \bar{y})^2}$$

Worked example

$y = [3, -0.5, 2, 7]$, $\hat{y} = [2.5, 0, 2, 8]$. Errors: $[0.5, -0.5, 0, -1]$. MAE $= (0.5 + 0.5 + 0 + 1)/4 = 0.5$. MSE $= (0.25 + 0.25 + 0 + 1)/4 = 0.375$. RMSE $= 0.612$. $\bar{y} = 2.875$, SST $= 0.0156 + 11.39 + 0.766 + 17.02 = 29.19$, SSE $= 1.5$, so $R^2 = 1 - 1.5/29.19 = 0.949$.

PYTHON

```python
from sklearn.metrics import mean_absolute_error, mean_squared_error, root_mean_squared_error, r2_score
y_true = [3, -0.5, 2, 7]; y_pred = [2.5, 0.0, 2, 8]
print("MAE :", mean_absolute_error(y_true, y_pred))
print("MSE :", mean_squared_error(y_true, y_pred))
print("RMSE:", round(root_mean_squared_error(y_true, y_pred), 3))
print("R2  :", round(r2_score(y_true, y_pred), 3))
```

OUTPUT

```
MAE : 0.5
MSE : 0.375
RMSE: 0.612
R2  : 0.949
```

Which to use? MAE when all errors cost the same and outliers are noise; RMSE when big errors are disproportionately costly; R^2 to compare against the mean baseline. Adjusted $R^2 = 1 - (1 - R^2)\frac{n-1}{n-p-1}$ penalises useless extra features.

6. Real-World Applications

House price error in lakhs (MAE/RMSE), demand forecasting (MAE), delivery-time estimation (RMSE), model comparison across datasets (R^2).

7. When to Use and When Not to Use

Metric	Best for	Watch out
MAE	Robust, easy to explain	Not smooth at 0; ignores error size pattern
MSE/RMSE	Penalising large errors; matches training loss of OLS	Dominated by outliers
R^2	Relative comparison, "% explained"	Can be negative on test data; grows when adding features; misleading for non-linear/near-constant targets

8. Advantages, Disadvantages and Limitations

MAE and RMSE are in target units, so a stakeholder can read them directly. R^2 is unit-free but says nothing about bias: always plot residuals. None of them tells you whether an error of 2 lakh is acceptable; compare with the baseline (`DummyRegressor`) and the business tolerance.

9. Python Example (scikit-learn)

PYTHON

```python
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split, cross_val_score
```

```python
from sklearn.dummy import DummyRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, root_mean_squared_error, r2_score

X, y = load_diabetes(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.25, random_state=0)
for name, m in [("mean baseline", DummyRegressor()), ("random forest", RandomForestRegressor(n_estimators=200,
random_state=0))]:
    p = m.fit(X_tr, y_tr).predict(X_te)
    print(f"{name:14s} MAE={mean_absolute_error(y_te, p):6.1f} RMSE={root_mean_squared_error(y_te, p):6.1f} R2=
{r2_score(y_te, p):6.3f}")
# scorers are 'neg_*' because sklearn always maximises a score
print("CV neg-RMSE:", cross_val_score(RandomForestRegressor(n_estimators=100, random_state=0), X_tr, y_tr, cv=5,
scoring="neg_root_mean_squared_error").round(1))
```

OUTPUT

```
mean baseline  MAE=  58.3 RMSE=  70.5 R2=-0.000
random forest  MAE=  48.1 RMSE=  61.0 R2= 0.250
CV neg-RMSE: [-55.5 -56.6 -54.5 -59.2 -57.9]
```

10. Common Mistakes and Related Concepts

Common mistakes

- Quoting training R^2 only; or comparing R^2 across different targets.
- Forgetting `neg_` scorers return negative numbers.
- Using MAPE when targets contain zeros.
- Ignoring residual plots (a good R^2 can hide patterned errors).

Related: MAPE, median absolute error, adjusted R^2, residual plots, Huber loss.

11. Quick Revision Notes

Chapter Summary

- MAE = mean $|e|$; MSE = mean e^2; RMSE = $\sqrt{\text{MSE}}$ (same unit as y); $R^2 = 1 - \text{SSE}/\text{SST}$.
- RMSE $\geq$ MAE; the gap grows with outliers.
- $R^2 = 0$ means "as good as predicting the mean"; test R^2 can be negative.
- Always compare to a baseline and judge on held-out data.

CHAPTER 9

Classification Metrics: Accuracy, Precision, Recall, F1

1. Definition (Simple English)

In one line

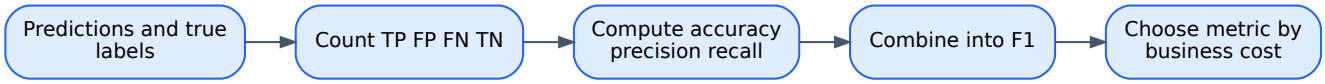
Accuracy = share of correct predictions. **Precision** = of the samples predicted positive, how many really are. **Recall** = of the real positives, how many we caught. **F1** = the harmonic mean of precision and recall.

2. Hinglish Explanation

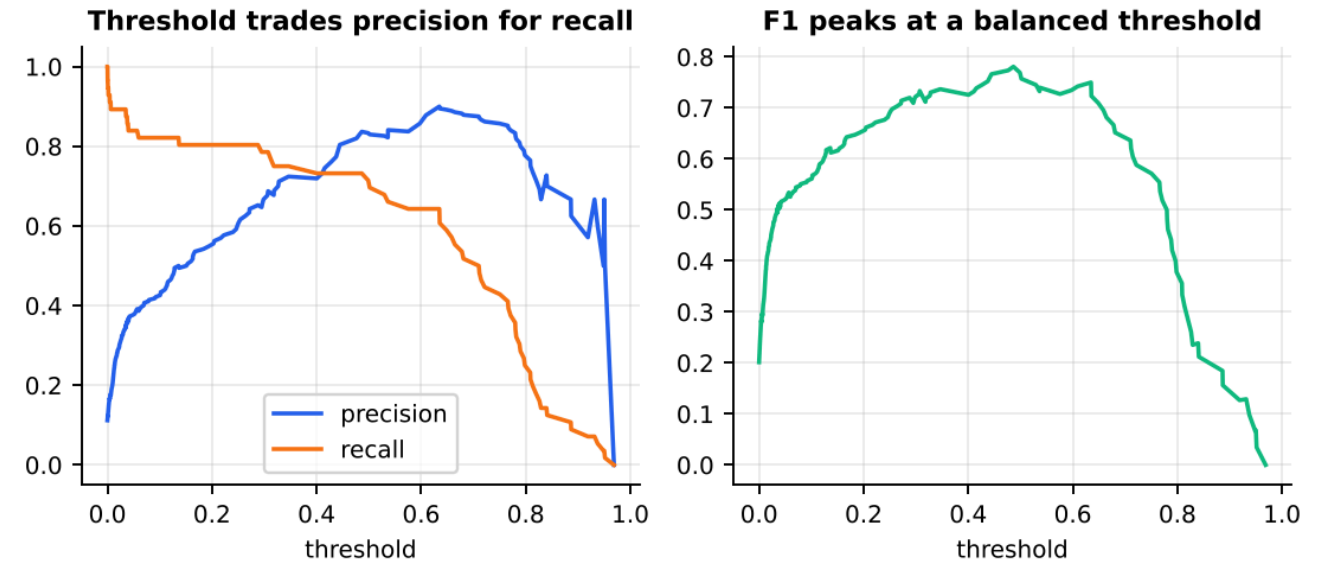
Hinglish mein samjho

Maan lo 100 mein se sirf 2 log cancer wale hain. Model sabko "healthy" bol de to **accuracy 98%**, par model bekaar hai! Isliye **Precision** dekhte hain ("jisko positive bola, usme se kitne sahi"), aur **Recall** ("jitne asli positive the, unme se kitne pakde"). Spam filter mein precision zaroori (galti se achhi mail spam na ho), cancer screening mein recall zaroori (koi marij chhoot na jaye). **F1** dono ka balance hai.

3. How It Works, Step by Step



4. Diagram



Raising the threshold makes the model pickier: precision up, recall down.

5. Formulas with a Numerical Example

$$\text{Accuracy} = \frac{TP+TN}{TP+TN+FP+FN}, \quad \text{Precision} = \frac{TP}{TP+FP}, \quad \text{Recall} = \frac{TP}{TP+FN}, \quad F_1 = \frac{2PR}{P+R}$$

Worked example

Out of 100 patients: TP=6, FN=4, FP=10, TN=80. Accuracy $= 86/100 = 0.86$. Precision $= 6/16 = 0.375$. Recall $= 6/10 = 0.60$. $F_1 = 2(0.375)(0.6)/(0.975) = 0.462$. A "everyone healthy" model would have accuracy 0.90 but recall 0.

PYTHON

```
TP, FN, FP, TN = 6, 4, 10, 80
acc = (TP + TN) / 100; prec = TP / (TP + FP); rec = TP / (TP + FN); f1 = 2 * prec * rec / (prec + rec)
print(f"accuracy={acc:.2f} precision={prec:.3f} recall={rec:.2f} f1={f1:.3f}")
print("always-negative model: accuracy =", (TN + FP) / 100, ", recall = 0.0")
```

OUTPUT

```
accuracy=0.86 precision=0.375 recall=0.60 f1=0.462
always-negative model: accuracy = 0.9 , recall = 0.0
```

Multi-class averaging: **macro** (mean of per-class scores), **weighted** (by class size), **micro** (pool all counts). F_β weights recall β times more than precision.

6. Real-World Applications

Fraud detection (recall + precision), spam filters (precision), disease screening (recall), search/recommendation (precision@k), churn targeting (precision under a marketing budget).

7. When to Use and When Not to Use

Metric	Use when	Avoid when
Accuracy	Balanced classes, equal error costs	Imbalanced data
Precision	False alarms are costly	Missing positives is worse
Recall	Missing a positive is costly	False alarms are expensive
F1	Need one balanced number on imbalanced data	Costs of FP and FN are very different

8. Advantages, Disadvantages and Limitations

Accuracy is simple but misleading under imbalance. Precision and recall each ignore one kind of error, so quote both. F1 ignores true negatives and treats FP/FN as equal cost. Threshold-free summaries (ROC-AUC, PR-AUC) complement them.

9. Python Example (scikit-learn)

PYTHON

```
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score

X, y = make_classification(n_samples=3000, weights=[0.95], flip_y=0.02, random_state=1)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, stratify=y, random_state=1)
m = LogisticRegression(max_iter=500).fit(X_tr, y_tr)
proba = m.predict_proba(X_te)[:, 1]
```

```
print("positives in test:", y_te.sum(), "of", len(y_te))
for thr in (0.5, 0.3, 0.15):
    p = (proba >= thr).astype(int)
    print(f"thr={thr:.2f} acc={accuracy_score(y_te, p):.3f} prec={precision_score(y_te, p):.3f} "
          f"rec={recall_score(y_te, p):.3f} f1={f1_score(y_te, p):.3f}")
print("ROC-AUC:", round(roc_auc_score(y_te, proba), 3))
```

OUTPUT

```
positives in test: 54 of 900
thr=0.50 acc=0.969 prec=0.783 rec=0.667 f1=0.720
thr=0.30 acc=0.957 prec=0.612 rec=0.759 f1=0.678
thr=0.15 acc=0.923 prec=0.426 rec=0.796 f1=0.555
ROC-AUC: 0.899
```

10. Common Mistakes and Related Concepts

Common mistakes

- Celebrating 95% accuracy on 95/5 data.
- Mixing up precision and recall; averaging F1 without saying macro/weighted.
- Picking the threshold on the test set (choose it on validation data).

Related: ROC-AUC, PR-AUC, balanced accuracy, MCC, log loss, `class_weight`, threshold tuning.

11. Quick Revision Notes

Chapter Summary

- Precision = TP/(TP+FP), Recall = TP/(TP+FN), F1 = harmonic mean.
- Imbalanced data: never rely on accuracy alone.
- Lower threshold: recall up, precision down (and vice versa).
- Multi-class: macro vs weighted vs micro averaging.

CHAPTER 10

Confusion Matrix and Classification Report

1. Definition (Simple English)

In one line

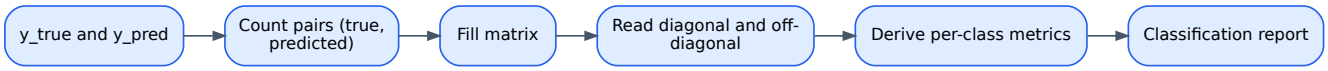
A **confusion matrix** is a table that counts, for each true class, how many samples were predicted as each class. The **classification report** turns it into per-class precision, recall, F1 and support in one printout.

2. Hinglish Explanation

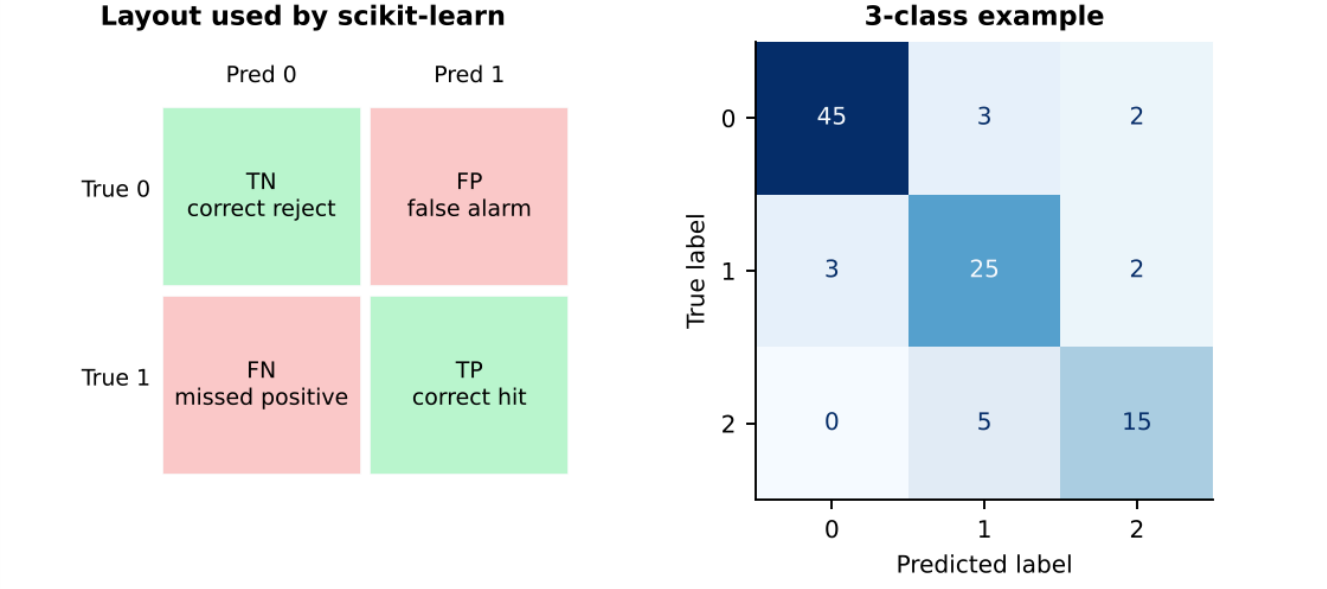
Hinglish mein samjho

Confusion matrix ek report card hai jo batata hai model **kahan confuse hua**. Rows = asli class, columns = predicted class (scikit-learn mein). Diagonal par sahi jawab, baaki jagah galtiyan. Binary case mein 4 khaane: **TP** (positive ko positive bola), **TN**, **FP** (galat alarm), **FN** (positive miss ho gaya). Classification report har class ke precision/recall/F1 ek table mein de deta hai.

3. How It Works, Step by Step



4. Diagram



Left: meaning of the four cells. Right: a 3-class matrix; off-diagonal cells show which classes get mixed up.

5. Formulas with a Numerical Example

$$\text{Specificity} = \frac{TN}{TN + FP}, \qquad \text{FPR} = \frac{FP}{FP + TN}, \qquad \text{Recall (TPR)} = \frac{TP}{TP + FN}$$

Worked example

Using the binary table $\begin{pmatrix} TN & FP \\ FN & TP \end{pmatrix} = \begin{pmatrix} 80 & 10 \\ 4 & 6 \end{pmatrix}$: precision for class 1 $= 6/16 = 0.375$, recall $= 6/10 = 0.6$. Specificity $= 80/90 = 0.889$. For class 0, precision $= 80/84 = 0.952$ and recall $= 0.889$. *Support* is the number of true samples per class (90 and 10). *Macro avg* recall $= (0.889 + 0.6)/2 = 0.744$; *weighted avg* uses supports $= (90 \times 0.889 + 10 \times 0.6)/100 = 0.86$.

PYTHON

```python
import numpy as np
from sklearn.metrics import confusion_matrix, classification_report
y_true = np.array([0] * 90 + [1] * 10)
y_pred = np.array([0] * 80 + [1] * 10 + [0] * 4 + [1] * 6)
print(confusion_matrix(y_true, y_pred))
tn, fp, fn, tp = confusion_matrix(y_true, y_pred).ravel(); print("TN FP FN TP:", tn, fp, fn, tp)
print(classification_report(y_true, y_pred, target_names=["healthy", "sick"], digits=3))
```

OUTPUT

```
[[80 10]
 [ 4  6]]
TN FP FN TP: 80 10 4 6
              precision    recall  f1-score   support

     healthy      0.952     0.889     0.920        90
        sick      0.375     0.600     0.462        10

    accuracy                          0.860       100
   macro avg      0.664     0.744     0.691       100
weighted avg      0.895     0.860     0.874       100
```

6. Real-World Applications

Medical diagnosis (how many sick patients are missed), fraud review queues (FP volume = analyst workload), multi-class document routing (which categories get confused), error analysis before improving a model.

7. When to Use and When Not to Use

Use it in **every** classification project for error analysis; use `normalize="true"` to see per-class recall when classes differ in size. It does not replace a threshold-free metric like ROC-AUC when you must compare models across thresholds, and becomes hard to read with hundreds of classes.

8. Advantages, Disadvantages and Limitations

Advantages	Disadvantages
Shows the *type* of mistakes	Depends on a fixed threshold
Source of all other metrics	Hard to read for many classes
Works for multi-class	Counts alone ignore the cost of each error type

9. Python Example (scikit-learn)

PYTHON

```python
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, classification_report

data = load_wine()
X_tr, X_te, y_tr, y_te = train_test_split(data.data, data.target, test_size=0.3, stratify=data.target,
random_state=5)
pred = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000)).fit(X_tr, y_tr).predict(X_te)
print(confusion_matrix(y_te, pred))
print(confusion_matrix(y_te, pred, normalize="true").round(2))
print(classification_report(y_te, pred, target_names=data.target_names, digits=3))
rep = classification_report(y_te, pred, output_dict=True)
print("macro F1 from dict:", round(rep["macro avg"]["f1-score"], 3))
```

OUTPUT

```
[[18  0  0]
 [ 0 20  1]
 [ 0  0 15]]
[[1.   0.   0.  ]
 [0.   0.95 0.05]
 [0.   0.   1.  ]]
              precision    recall  f1-score   support

     class_0      1.000     1.000     1.000        18
     class_1      1.000     0.952     0.976        21
     class_2      0.938     1.000     0.968        15

    accuracy                          0.981        54
   macro avg      0.979     0.984     0.981        54
weighted avg      0.983     0.981     0.982        54

macro F1 from dict: 0.981
```

10. Common Mistakes and Related Concepts

Common mistakes

- Transposing the matrix (scikit-learn rows = true, columns = predicted; other tools differ).
- Reading `ravel()` as `tp, fp, fn, tn` (correct order is `tn, fp, fn, tp`).
- Quoting weighted average on imbalanced data when minority performance matters.

Related: ROC/PR curves, `ConfusionMatrixDisplay`, `multilabel_confusion_matrix`, MCC, Cohen's kappa.

11. Quick Revision Notes

Chapter Summary

- Rows = true, columns = predicted. Binary `ravel()` gives TN, FP, FN, TP.
- Report = precision, recall, F1, support per class + macro/weighted averages.
- Specificity $= TN/(TN+FP)$; FPR $= 1-$ specificity.
- Use the matrix to see *which* classes are confused.

PART D

Model Tuning

11. Cross-Validation

12. GridSearchCV

13. RandomizedSearchCV

CHAPTER 11

Cross-Validation

1. Definition (Simple English)

In one line

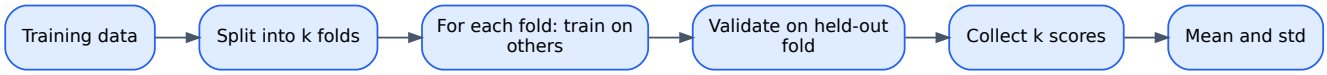
Cross-validation (CV) estimates how well a model will do on new data by splitting the training data into k folds, training on $k-1$ folds and validating on the remaining one, repeating until every fold has been the validation set once, then averaging the scores.

2. Hinglish Explanation

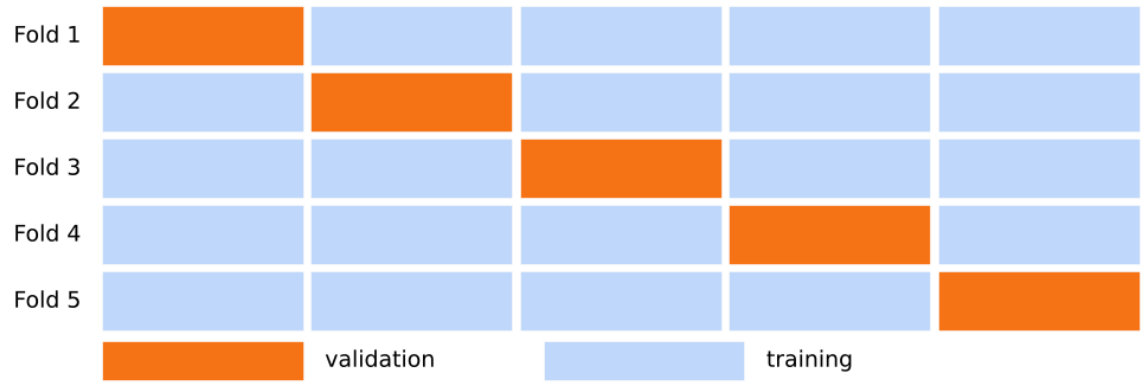
Hinglish mein samjho

Ek hi train/test split par score luck par depend karta hai: ho sakta hai test mein aasaan examples aa gaye. CV mein hum data ke 5 hisse karte hain, har baar ek hissa exam (validation) aur baaki 4 padhai. Paanch scores ka **average** zyada bharosemand hai, aur **standard deviation** batata hai ki score kitna hilta hai. Test set phir bhi alag rakhte hain, sirf final check ke liye.

3. How It Works, Step by Step



4. Diagram



5-fold CV: each fold is used once for validation (orange) and four times for training (blue).

5. Formulas with a Numerical Example

$$\text{CV score} = \frac{1}{k}\sum_{i=1}^{k} \text{score}_i, \qquad \text{std} = \sqrt{\frac{1}{k}\sum_{i} (\text{score}_i - \text{CV score})^2}$$

Worked example

5-fold accuracies: $0.90, 0.92, 0.88, 0.94, 0.91$. Mean $= 0.91$. Deviations: $-0.01, 0.01, -0.03, 0.03, 0$; squares sum $= 0.0020$; variance $= 0.0004$; std $= 0.02$. Report "0.91 ± 0.02". With 100 samples and $k = 5$ each fold trains on 80 and validates on 20.

PYTHON

```python
import numpy as np
s = np.array([0.90, 0.92, 0.88, 0.94, 0.91]); print(f"mean={s.mean():.2f} std={s.std():.2f}")
from sklearn.model_selection import KFold
for i, (tr, va) in enumerate(KFold(5).split(np.arange(100))):
    if i < 2: print(f"fold {i}: train={len(tr)} validate={len(va)}")
```

OUTPUT

```
mean=0.91 std=0.02
fold 0: train=80 validate=20
fold 1: train=80 validate=20
```

Variants: **StratifiedKFold** (keeps class ratios; default for classifiers), **Repeated K-Fold**, **Leave-One-Out** (k = n, costly), **GroupKFold** (keeps a group, e.g. a patient, in one fold), **TimeSeriesSplit** (always train on the past, validate on the future).

6. Real-World Applications

Comparing algorithms honestly, choosing hyper-parameters, estimating performance on small datasets, and checking stability of a model before deployment.

7. When to Use and When Not to Use

Use it when	Avoid / adapt when
Data is limited and a single split is noisy	Time series: use `TimeSeriesSplit`, not shuffled K-fold
Comparing models or tuning	Samples are grouped (patients, users): use `GroupKFold`
Model trains quickly	Training takes days: use a single validation split

8. Advantages, Disadvantages and Limitations

Advantages	Disadvantages
Uses all data for training and validation	k times more compute
Gives mean and spread	Preprocessing outside the pipeline leaks information
Reduces luck of one split	Still an estimate; keep a final test set

9. Python Example (scikit-learn)

PYTHON

```python
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, cross_val_score, cross_validate, StratifiedKFold
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier

X, y = load_breast_cancer(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, stratify=y, random_state=0)
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=0)
models = {"logreg": make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000)),
```

```
          "tree": DecisionTreeClassifier(random_state=0)}
for name, m in models.items():           # scaler is refit inside every fold: no leakage
    s = cross_val_score(m, X_tr, y_tr, cv=cv, scoring="accuracy")
    print(f"{name:7s} folds={s.round(3)} mean={s.mean():.3f} std={s.std():.3f}")
res = cross_validate(models["logreg"], X_tr, y_tr, cv=cv, scoring=["accuracy", "recall", "f1"],
return_train_score=True)
print({k: round(v.mean(), 3) for k, v in res.items() if k.startswith(("test", "train"))})
print("final test score (used once):", round(models["logreg"].fit(X_tr, y_tr).score(X_te, y_te), 3))
```

OUTPUT

```
logreg  folds=[0.989 0.967 0.978 0.967 0.989] mean=0.978 std=0.010
tree    folds=[0.978 0.956 0.923 0.945 0.912] mean=0.943 std=0.023
{'test_accuracy': np.float64(0.978), 'train_accuracy': np.float64(0.99), 'test_recall': np.float64(0.986),
'train_recall': np.float64(0.995), 'test_f1': np.float64(0.983), 'train_f1': np.float64(0.992)}
final test score (used once): 0.982
```

10. Common Mistakes and Related Concepts

Common mistakes

- Scaling/imputing the full data *before* CV (leakage). Put those steps in a Pipeline.
- Using plain K-fold for time series or grouped data.
- Using CV scores of the tuned model as the final performance (use a held-out test set or nested CV).
- Forgetting `shuffle=True` when data is ordered by class.

Related: Hold-out, nested CV, `cross_val_predict`, learning curves, `GridSearchCV`.

11. Quick Revision Notes

Chapter Summary

- k-fold: train on $k-1$ folds, validate on 1, average. Typical $k = 5$ or 10.
- Stratified for classification; Group/TimeSeries when data are dependent.
- Wrap preprocessing in a Pipeline so it is refit per fold.
- Report mean ± std; keep a final untouched test set.

CHAPTER 12

GridSearchCV

1. Definition (Simple English)

In one line

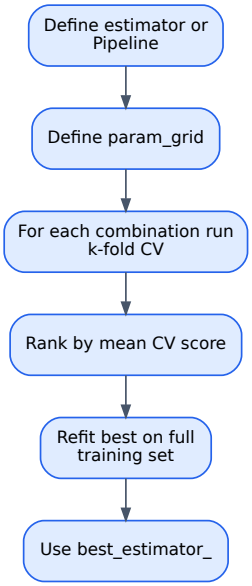
GridSearchCV tries every combination of the hyper-parameter values you list, scores each combination with cross-validation, and keeps the best one (then refits it on all training data).

2. Hinglish Explanation

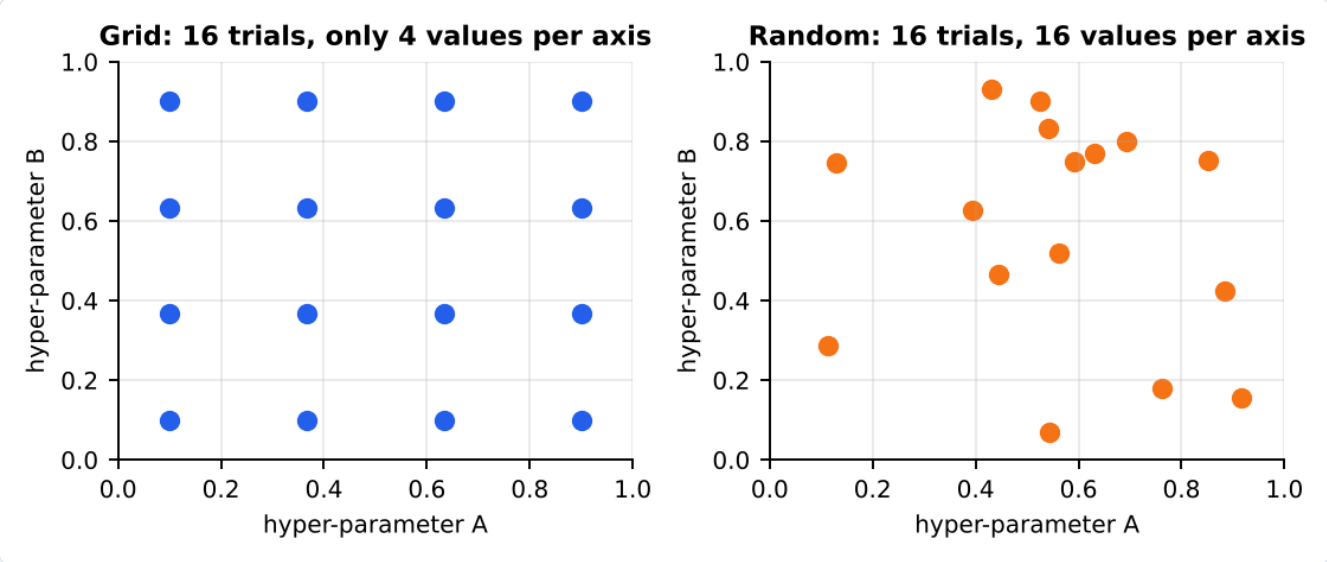
Hinglish mein samjho

Hyper-parameters woh settings hain jo model khud nahi seekhta, hum dete hain (jaise `max_depth`, `C`, `k`). Grid search bilkul "sab combination try karo" hai: agar `max_depth` ke 4 options aur `min_samples_leaf` ke 3 options hain to 12 combinations, aur 5-fold CV ke saath 60 model fits. Sabse accha CV score wala combination jeet jata hai. Simple aur thorough, par combinations badhne par bahut slow.

3. How It Works, Step by Step



4. Diagram



Same budget, different coverage. If only parameter A matters, random search tests 16 values of A; the grid tests 4.

5. Formulas with a Numerical Example

$$\text{number of fits} = \left(\prod_{p} |V_p|\right) \times k$$

Worked example

`{"max_depth":[2,4,6,None], "min_samples_leaf":[1,5,10]}` with `cv=5` : $4 \times 3 = 12$ combinations, $12 \times 5 = 60$ fits (+1 final refit). Adding a third parameter with 5 values makes it $12 \times 5 \times 5 = 300$ fits: the cost multiplies, so grids explode quickly.

PYTHON

```python
from sklearn.model_selection import ParameterGrid
g = {"max_depth": [2, 4, 6, None], "min_samples_leaf": [1, 5, 10]}
n = len(ParameterGrid(g)); print("combinations:", n, " fits with cv=5:", n * 5)
print(list(ParameterGrid(g))[:3])
```

OUTPUT

```
combinations: 12  fits with cv=5: 60
[{'max_depth': 2, 'min_samples_leaf': 1}, {'max_depth': 2, 'min_samples_leaf': 5}, {'max_depth': 2,
'min_samples_leaf': 10}]
```

6. Real-World Applications

Tuning SVM `C` / `gamma`, tree depth, regularisation strength, number of neighbours, and entire pipelines (imputer strategy, scaler type, model) in one search.

7. When to Use and When Not to Use

Use it when	Avoid it when
Few parameters with a small set of sensible values	Many parameters or continuous ranges (use RandomizedSearchCV)
Training is fast	Each fit takes minutes or hours
You want a reproducible, exhaustive search	You have no idea of good ranges (start coarse, then refine)

8. Advantages, Disadvantages and Limitations

Advantages	Disadvantages
Exhaustive within the grid; simple	Cost grows exponentially with parameters
Works on Pipelines (`step__param`)	Can only find values you listed
Provides `cv_results_` for analysis	Best CV score is optimistic; need a final test set

9. Python Example (scikit-learn)

PYTHON

```python
import pandas as pd
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

X, y = load_breast_cancer(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, stratify=y, random_state=0)
pipe = Pipeline([("scaler", StandardScaler()), ("svc", SVC())])
grid = {"svc__C": [0.1, 1, 10, 100], "svc__gamma": [0.001, 0.01, 0.1], "svc__kernel": ["rbf"]}
gs = GridSearchCV(pipe, grid, cv=5, scoring="f1", n_jobs=1, refit=True).fit(X_tr, y_tr)
print("best params:", gs.best_params_, " best CV F1:", round(gs.best_score_, 4))
res = pd.DataFrame(gs.cv_results_)[["param_svc__C", "param_svc__gamma", "mean_test_score", "std_test_score",
"rank_test_score"]]
print(res.sort_values("rank_test_score").head(4).round(4).to_string(index=False))
print("test F1 (touched once):", round(gs.score(X_te, y_te), 4))
```

OUTPUT

```
best params: {'svc__C': 10, 'svc__gamma': 0.01, 'svc__kernel': 'rbf'}  best CV F1: 0.979
 param_svc__C  param_svc__gamma  mean_test_score  std_test_score  rank_test_score
         10.0             0.010           0.9790          0.0044                1
         10.0             0.001           0.9775          0.0088                2
          1.0             0.010           0.9742          0.0121                3
        100.0             0.001           0.9740          0.0055                4
test F1 (touched once): 0.9796
```

10. Common Mistakes and Related Concepts

Common mistakes

- Scaling outside the Pipeline before `GridSearchCV` (leaks fold information).
- Using the same data for tuning and final reporting.
- Huge grids of unimportant parameters; best value sitting on the grid edge (extend the grid).
- Forgetting `scoring=` : the default metric may not match your goal.

Related: RandomizedSearchCV, `HalvingGridSearchCV` , Bayesian optimisation, nested CV, validation curves.

11. Quick Revision Notes

Chapter Summary

- Exhaustive search over `param_grid` ; fits = combinations × folds.
- Use `Pipeline` + `step__param` names; set `scoring` deliberately.
- After fitting: `best_params_` , `best_score_` , `best_estimator_` , `cv_results_` .
- If the best value is at the grid edge, widen the grid.

CHAPTER 13

RandomizedSearchCV

1. Definition (Simple English)

In one line

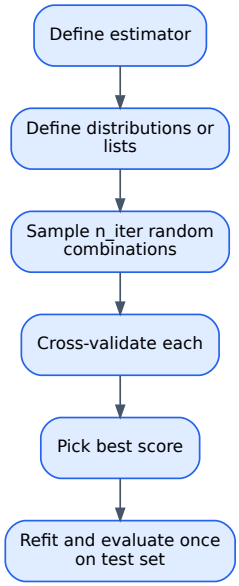
RandomizedSearchCV samples a fixed number (`n_iter`) of random hyper-parameter combinations from lists or probability distributions, evaluates each with cross-validation, and keeps the best.

2. Hinglish Explanation

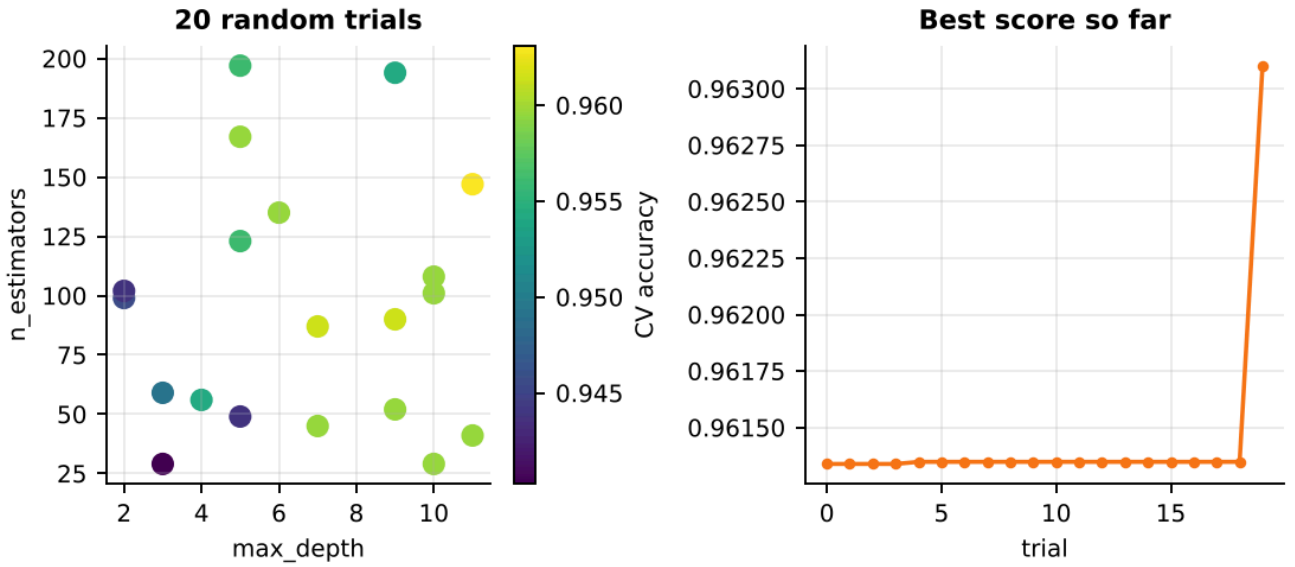
Hinglish mein samjho

Grid search har cheez try karta hai, par agar 5 parameters hain to combinations lakhon mein ho jate hain. Randomized search kehta hai: "mujhe sirf 30 random combinations try karne hain". Research (Bergstra & Bengio) dikhati hai ki aksar sirf 1-2 parameters asli mein matter karte hain, aur random search un important parameters ki **zyada alag-alag values** try kar leta hai. Budget aap `n_iter` se control karte ho.

3. How It Works, Step by Step



4. Diagram



Random trials (colour = CV accuracy) and the best-so-far curve: most gain comes early.

5. Formulas with a Numerical Example

$$\text{fits} = n_{\text{iter}} \times k, \qquad P(\text{hit top } q \text{ fraction in } n \text{ trials}) = 1 - (1-q)^n$$

Worked example

How many random trials to land in the best 5% of the space with 95% confidence? $1-(0.95)^n \geq 0.95 \Rightarrow n \geq \ln 0.05 / \ln 0.95 \approx 59$. So about 60 trials suffice **regardless of how many parameters** there are, whereas a grid grows multiplicatively.

PYTHON

```python
import math
q, conf = 0.05, 0.95
print("trials needed:", math.ceil(math.log(1 - conf) / math.log(1 - q)))
print("P(top 5% in 30 trials):", round(1 - (1 - q) ** 30, 3))
```

OUTPUT

```
trials needed: 59
P(top 5% in 30 trials): 0.785
```

6. Real-World Applications

Tuning Random Forest / Gradient Boosting (many parameters), SVMs with continuous `C` and `gamma` (use log-uniform), pipelines with several preprocessing choices, and quick first-pass tuning before a finer grid.

7. When to Use and When Not to Use

Use it when	Avoid it when
Many hyper-parameters or continuous ranges	Only 1-2 parameters with few values (a grid is fine)
Limited compute budget	You need a guaranteed exhaustive check
Exploring unknown ranges	You need smarter guided search at large scale (Bayesian tools)

8. Advantages, Disadvantages and Limitations

Advantages	Disadvantages
Fixed, controllable cost	May miss the exact optimum
Scales to many parameters	Results vary with `random_state` / `n_iter`
Supports continuous distributions	Poor distributions (wrong ranges/scale) give poor results

9. Python Example (scikit-learn)

PYTHON

```python
from scipy.stats import loguniform, randint
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, RandomizedSearchCV
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

X, y = load_breast_cancer(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, stratify=y, random_state=0)
pipe = Pipeline([("scaler", StandardScaler()), ("svc", SVC())])
dist = {"svc__C": loguniform(1e-2, 1e3), "svc__gamma": loguniform(1e-4, 1e0), "svc__kernel": ["rbf", "linear"]}
rs = RandomizedSearchCV(pipe, dist, n_iter=25, cv=5, scoring="f1", random_state=42, n_jobs=1).fit(X_tr, y_tr)
print("best params:", {k: (round(v, 4) if isinstance(v, float) else v) for k, v in rs.best_params_.items()})
print("best CV F1:", round(rs.best_score_, 4), " fits:", 25 * 5)
print("test F1:", round(rs.score(X_te, y_te), 4))
```

OUTPUT

```
best params: {'svc__C': np.float64(0.0603), 'svc__gamma': np.float64(0.0002), 'svc__kernel': 'linear'}
best CV F1: 0.9791  fits: 125
test F1: 0.9796
```

10. Common Mistakes and Related Concepts

Common mistakes

- Sampling `C` or `gamma` uniformly instead of log-uniformly (most samples land in the largest decade).
- Too small `n_iter`; never setting `random_state`.
- Reporting the best CV score as the expected test score.

Related: `GridSearchCV`, `HalvingRandomSearchCV`, Optuna / Bayesian optimisation, early stopping.

11. Quick Revision Notes

Chapter Summary

- Random sample of `n_iter` combinations; cost $= n_{iter} \times k$ fits.
- Use `scipy.stats` distributions; `loguniform` for scale-like parameters.
- Usually beats a same-budget grid when only some parameters matter.
- Follow with a narrower grid around the best region if needed.

PART E

Unsupervised Learning

14. K-Means Clustering

15. DBSCAN

16. Principal Component Analysis (PCA)

CHAPTER 14

K-Means Clustering

1. Definition (Simple English)

In one line

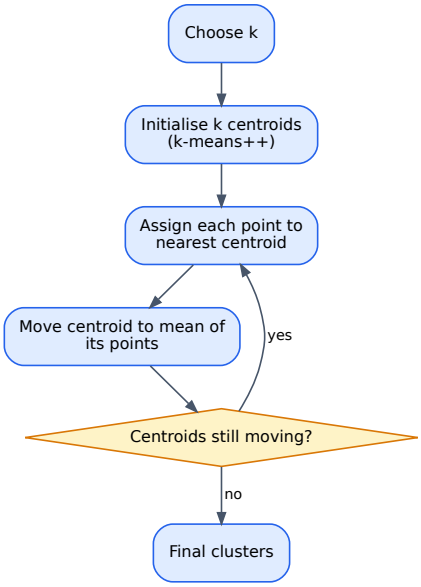
K-Means is an unsupervised algorithm that splits data into k groups (clusters) by repeatedly assigning each point to the nearest cluster centre (centroid) and moving each centre to the mean of its points.

2. Hinglish Explanation

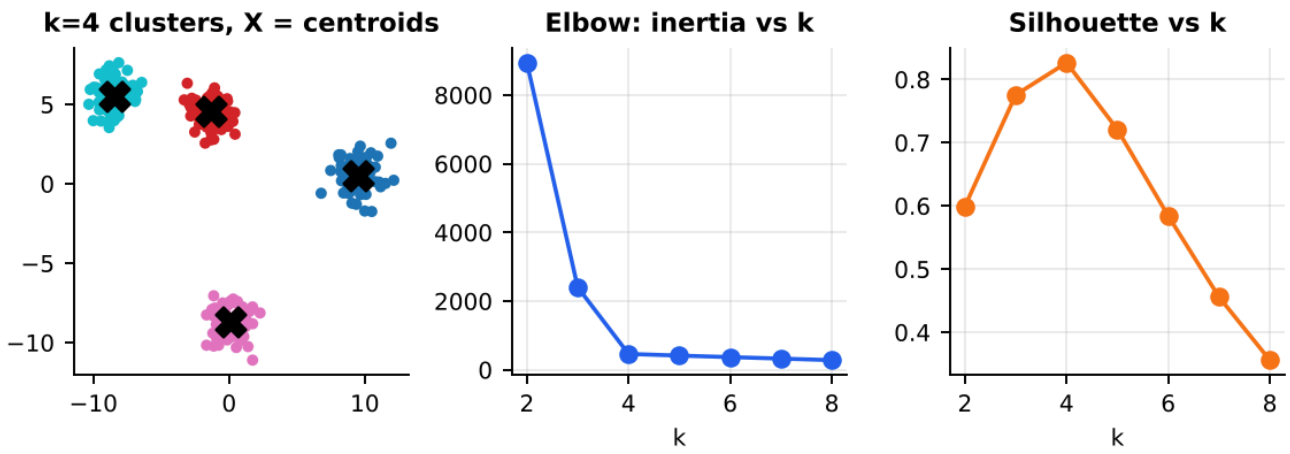
Hinglish mein samjho

Maan lo ek mall ke 1000 customers hain, par koi label nahi. K-Means pehle k random "kendra" (centroid) rakhta hai. Phir har customer ko uske sabse paas wale kendra se jod deta hai, aur har kendra ko apne group ke **average** par khiska deta hai. Yeh dohrate rehte hain jab tak kendra hilna band na kar dein. k aapko khud chunna padta hai: **elbow method** ya **silhouette score** isme madad karte hain. Scaling zaroori hai kyunki distance use hota hai.

3. How It Works, Step by Step



4. Diagram



Left: clusters and centroids. Middle: the "elbow" where inertia stops dropping fast. Right: silhouette peaks at the best k.

5. Formulas with a Numerical Example

$$\text{Inertia (WCSS)} = \sum_{j=1}^{k} \sum_{x \in C_j} \|x - \mu_j\|^2, \qquad s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

where $a(i)$ = mean distance to points in its own cluster and $b(i)$ = mean distance to the nearest other cluster.

Worked example

1-D points $[1, 2, 3, 10, 11, 12]$, $k = 2$, start centroids 2 and 10. Assign: $\{1, 2, 3\} \to$ centroid 2, $\{10, 11, 12\} \to$ centroid 10 (already stable). Inertia $= (1 + 0 + 1) + (1 + 0 + 1) = 4$. For point $x = 3$: $a = (2 + 1)/2 = 1.5$, $b =$ mean distance to the other cluster $= (7 + 8 + 9)/3 = 8$, so $s = (8 - 1.5)/8 = 0.81$ (well clustered).

PYTHON

```
import numpy as np
from sklearn.cluster import KMeans
x = np.array([1, 2, 3, 10, 11, 12]).reshape(-1, 1)
km = KMeans(2, n_init=10, random_state=0).fit(x)
print("centroids:", km.cluster_centers_.ravel(), " labels:", km.labels_, " inertia:", km.inertia_)
a = (abs(3 - 1) + abs(3 - 2)) / 2; b = np.mean([abs(3 - 10), abs(3 - 11), abs(3 - 12)]); print("silhouette of
x=3:", round((b - a) / max(a, b), 3))
```

OUTPUT

```
centroids: [11.  2.]  labels: [1 1 1 0 0 0]  inertia: 4.0
silhouette of x=3: 0.812
```

6. Real-World Applications

Customer segmentation, image colour compression, grouping documents or news, sensor/telemetry grouping, and creating features (cluster id) for other models.

7. When to Use and When Not to Use

Use it when	Avoid it when
Clusters are roughly round and similar in size	Clusters are elongated, nested or of very different density (use DBSCAN)
Large data, need speed	You do not know whether clusters exist at all
Features are numeric and scaled	Many categorical features (consider K-Modes)

8. Advantages, Disadvantages and Limitations

Advantages	Disadvantages
Simple, fast, scales well	Need to choose k
Easy to interpret centroids	Sensitive to initialisation, scale and outliers
Works with `MiniBatchKMeans` for big data	Assumes convex, similar-sized clusters; hard assignments

9. Python Example (scikit-learn)

PYTHON

```python
import pandas as pd
from sklearn.datasets import load_wine
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score, adjusted_rand_score

X, y = load_wine(return_X_y=True)
Xs = StandardScaler().fit_transform(X)          # unsupervised: no train/test split needed for exploration
for k in range(2, 7):
    km = KMeans(n_clusters=k, n_init=10, random_state=0).fit(Xs)
    print(f"k={k} inertia={km.inertia_:8.1f} silhouette={silhouette_score(Xs, km.labels_):.3f}")
km = KMeans(3, n_init=10, random_state=0).fit(Xs)
print("agreement with true wine classes (ARI):", round(adjusted_rand_score(y, km.labels_), 3))
print(pd.Series(km.labels_).value_counts().sort_index().to_dict())
```

OUTPUT

```
k=2 inertia=  1659.0 silhouette=0.268
k=3 inertia=  1277.9 silhouette=0.285
k=4 inertia=  1180.7 silhouette=0.246
k=5 inertia=  1110.4 silhouette=0.203
k=6 inertia=  1044.5 silhouette=0.196
agreement with true wine classes (ARI): 0.897
{0: 65, 1: 51, 2: 62}
```

10. Common Mistakes and Related Concepts

Common mistakes

- Clustering unscaled data (large-range features dominate).
- Choosing k only by elbow when no elbow exists; ignore silhouette and business meaning.
- Treating clusters as ground truth; always profile them.
- Using a single initialisation (`n_init=10` is safer).

Related: MiniBatchKMeans, K-Medoids, Gaussian Mixture Models, hierarchical clustering, DBSCAN, silhouette / Davies-Bouldin.

11. Quick Revision Notes

Chapter Summary

- Alternate: assign to nearest centroid, recompute means; minimises inertia.
- Pick k with elbow + silhouette + business sense; use k-means++ and `n_init`.
- Scale features; sensitive to outliers; assumes round clusters.
- Converges to a local minimum, so rerun with several seeds.

CHAPTER 15

DBSCAN

1. Definition (Simple English)

In one line

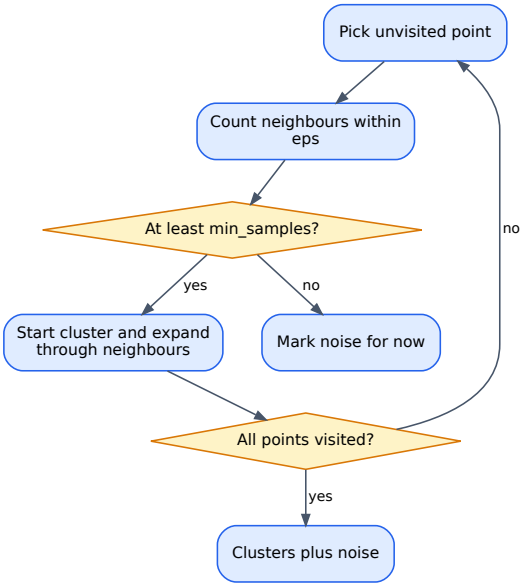
DBSCAN (Density-Based Spatial Clustering of Applications with Noise) groups points that are packed closely together and labels isolated points as **noise**. It finds clusters of any shape and does not need the number of clusters in advance.

2. Hinglish Explanation

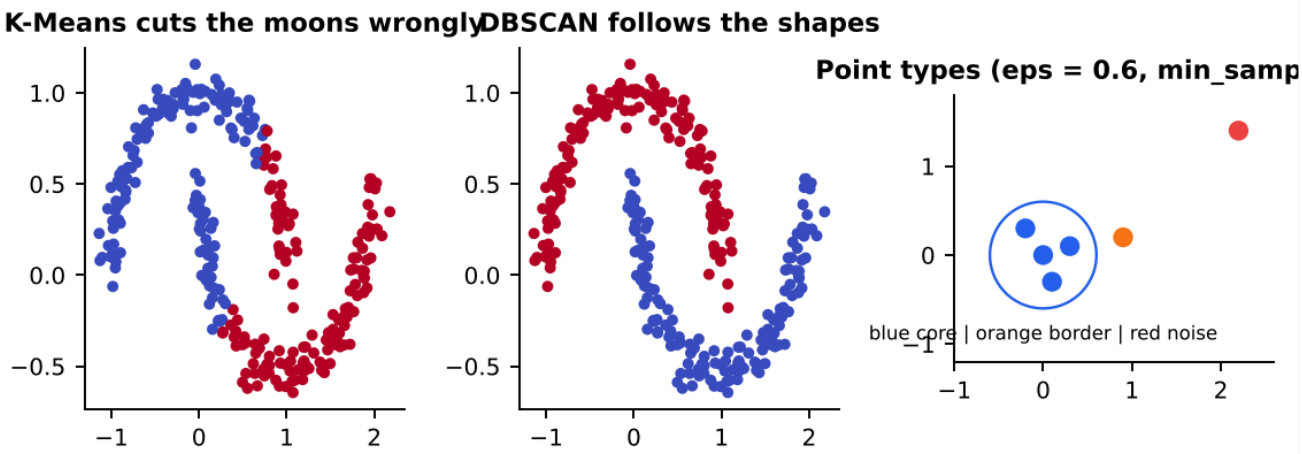
Hinglish mein samjho

Raat mein shehar ko aasman se dekho: jahan roshni ke **ghane** dhabbe hain woh shehar hain, beech ka andhera khaali jagah. DBSCAN yahi karta hai. Do settings: `eps` (padosi kitni door tak maane jayein) aur `min_samples` (ek ghani jagah banane ke liye kam se kam kitne points). Jis point ke `eps` ke andar kaafi padosi hain woh **core point**, jo core ke paas hai par khud ghana nahi woh **border**, aur baaki sab **noise** (label -1). K-Means ke ulat, ismein k nahi dena padta aur tedhe-medhe shape bhi mil jate hain.

3. How It Works, Step by Step



4. Diagram



K-Means vs DBSCAN on non-round clusters, and the three point types.

5. Formulas with a Numerical Example

$$N_\varepsilon(p) = \{q : d(p,q) \le \varepsilon\}, \qquad p \text{ is a core point if } |N_\varepsilon(p)| \ge \text{min_samples}$$

Worked example

1-D points $[1.0, 1.2, 1.4, 1.6, 5.0, 5.2, 9.0]$, `eps=0.5` , `min_samples=3` (a point counts itself). Point 1.2 has neighbours $\{1.0, 1.2, 1.4\}$ so 3, **core**. Likewise 1.4 and 1.6 chain together into cluster 0. 1.0 has only $\{1.0, 1.2\}$ (2) but lies within eps of core 1.2, so it is a **border** point of cluster 0. 5.0 and 5.2 have only 2 points each: **noise**. 9.0 is noise. Result labels: $[0, 0, 0, 0, -1, -1, -1]$.

PYTHON

```python
import numpy as np
from sklearn.cluster import DBSCAN
x = np.array([1.0, 1.2, 1.4, 1.6, 5.0, 5.2, 9.0]).reshape(-1, 1)
db = DBSCAN(eps=0.5, min_samples=3).fit(x)
print("labels:", db.labels_, " core indices:", db.core_sample_indices_)
```

OUTPUT

```
labels: [ 0  0  0  0 -1 -1 -1]  core indices: [0 1 2 3]
```

Choosing eps: plot the sorted distance to each point's k-th nearest neighbour (k = `min_samples`) and pick the "knee". Rule of thumb: `min_samples` $\ge$ dimensions + 1 (often $2\times$ dimensions).

6. Real-World Applications

Anomaly/outlier detection, geospatial hotspots (accident or crime clusters from GPS), grouping similar sensor events, cleaning noisy data, and image segmentation.

7. When to Use and When Not to Use

Use it when	Avoid it when
Clusters have arbitrary shapes	Clusters have very different densities (try HDBSCAN)
Noise/outliers are present and should be flagged	Data is high-dimensional (distances become similar)
k is unknown	You need every point assigned to a cluster

8. Advantages, Disadvantages and Limitations

Advantages	Disadvantages
No k; finds arbitrary shapes	Very sensitive to `eps`
Built-in noise detection	Single global density; struggles with varying density
Deterministic given data order for core points	No `predict` for new points; needs scaling

9. Python Example (scikit-learn)

PYTHON

```python
import numpy as np
from sklearn.datasets import make_moons
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import NearestNeighbors
from sklearn.cluster import DBSCAN
from sklearn.metrics import silhouette_score

X, _ = make_moons(n_samples=400, noise=0.08, random_state=1)
X = np.vstack([X, [[2.5, 2.5], [-2, 2.0], [3, -1.5]]])        # add 3 obvious outliers
Xs = StandardScaler().fit_transform(X)
kd = np.sort(NearestNeighbors(n_neighbors=5).fit(Xs).kneighbors(Xs)[0][:, -1])
print("5-NN distance percentiles 50/90/99:", np.percentile(kd, [50, 90, 99]).round(3))
db = DBSCAN(eps=0.3, min_samples=5).fit(Xs)
labels = db.labels_
print("clusters:", len(set(labels) - {-1}), " noise points:", int((labels == -1).sum()))
mask = labels != -1
print("silhouette (non-noise):", round(silhouette_score(Xs[mask], labels[mask]), 3))
```

OUTPUT

```
5-NN distance percentiles 50/90/99: [0.114 0.178 0.315]
clusters: 2  noise points: 3
silhouette (non-noise): 0.379
```

10. Common Mistakes and Related Concepts

Common mistakes

- Not scaling features before measuring distance.
- Default `eps=0.5` on data with a different scale.
- Expecting `predict()` for new data (use `fit_predict` only; refit or train a classifier on the labels).
- Treating all noise as errors: it may be the interesting anomalies.

Related: HDBSCAN, OPTICS, K-Means, Local Outlier Factor, Isolation Forest.

11. Quick Revision Notes

Chapter Summary

- Core / border / noise from `eps` and `min_samples`; label `-1` is noise.

- Finds arbitrary shapes, no k needed, flags outliers.
- Choose `eps` with a k-distance plot; scale features first.
- Weak with varying densities and in high dimensions.

CHAPTER 16

Principal Component Analysis (PCA)

1. Definition (Simple English)

In one line

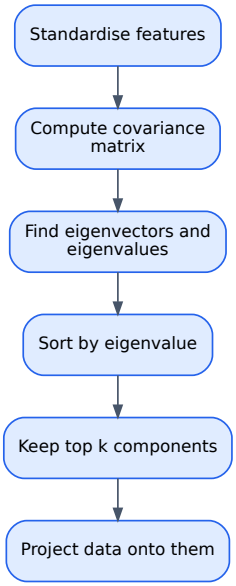
PCA is an unsupervised technique that rotates your data onto new, uncorrelated axes (principal components) ordered by how much variation they capture, so you can keep only the first few and reduce the number of features with minimal information loss.

2. Hinglish Explanation

Hinglish mein samjho

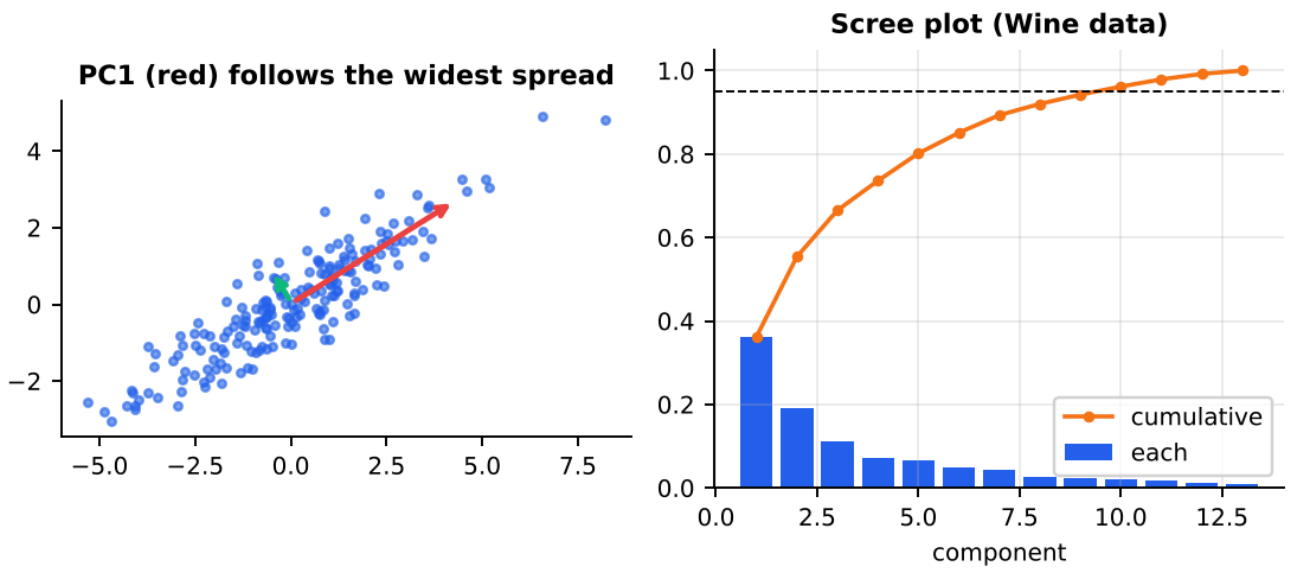
Socho 30 features hain, jinme se bahut saare ek doosre se related hain (jaise length aur area). PCA aisi **nayi axes** banata hai jo data ke sabse zyada phailav (variance) wali direction mein hoti hain. Pehli axis PC1 sabse zyada information rakhti hai, phir PC2, aur aage. Hum sirf pehli 2-3 rakh lete hain: data chhota ho jata hai, plot ho sakta hai, aur model tez chalta hai. Dhyan rahe: PCA se pehle **scaling** karni hoti hai, aur PCA labels ko dekhta hi nahi.

3. How It Works, Step by Step



(scikit-learn uses SVD, which gives the same result and is more stable.)

4. Diagram



Left: PCA axes on 2-D data. Right: scree plot, where a few components hold most of the variance.

5. Formulas with a Numerical Example

$$\Sigma = \frac{1}{n-1} X_c^\top X_c, \quad \Sigma \mathbf{v}_j = \lambda_j \mathbf{v}_j, \quad \text{explained ratio}_j = \frac{\lambda_j}{\sum_i \lambda_i}, \quad Z = X_c V_k$$

Worked example

Two standardised features with correlation 0.8 have $\Sigma = \begin{pmatrix} 1 & 0.8 \\ 0.8 & 1 \end{pmatrix}$. Eigenvalues are $1 + 0.8 = 1.8$ and $1 - 0.8 = 0.2$ (eigenvectors $(1,1)/\sqrt{2}$ and $(1,-1)/\sqrt{2}$). PC1 explains $1.8/2.0 = 90\%$ of variance. A point $(1,1)$ projects to $PC1 = (1+1)/\sqrt{2} = 1.414$ and $PC2 = 0$, so one number captures it fully.

PYTHON

```
import numpy as np
S = np.array([[1, 0.8], [0.8, 1]]); vals, vecs = np.linalg.eigh(S)
print("eigenvalues:", vals[::-1], " explained:", (vals[::-1] / vals.sum()).round(2))
print("PC1 direction:", np.abs(vecs[:, 1]).round(3), " projection of (1,1):", round(float(np.array([1, 1]) @
vecs[:, 1]), 3))
```

OUTPUT

```
eigenvalues: [1.8 0.2]  explained: [0.9 0.1]
PC1 direction: [0.707 0.707]  projection of (1,1): 1.414
```

6. Real-World Applications

Visualising high-dimensional data in 2-D, removing multicollinearity before regression, compressing images, noise reduction, speeding up K-Means/SVM/KNN, and genetics/finance factor analysis.

7. When to Use and When Not to Use

Use it when	Avoid it when
Many correlated numeric features	You need interpretable original features
Need 2-D/3-D visualisation	Relationships are strongly non-linear (try kernel PCA, t-SNE, UMAP)
Speed or noise reduction matters	Features are mostly categorical or unscaled and you forget to scale

8. Advantages, Disadvantages and Limitations

Advantages	Disadvantages
Removes correlation, reduces dimensions	Components are hard to interpret
Fast and well understood	Linear only; variance is not the same as predictive power
Reduces overfitting and noise	Sensitive to scaling and outliers

9. Python Example (scikit-learn)

PYTHON

```python
import numpy as np
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression

X, y = load_wine(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.25, stratify=y, random_state=0)
pca_pipe = make_pipeline(StandardScaler(), PCA(n_components=0.95))      # keep 95% of variance
pca_pipe.fit(X_tr)
pca = pca_pipe[-1]
print("components kept:", pca.n_components_, " explained:", pca.explained_variance_ratio_.round(3))
for name, m in [("all 13 features", make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))),
                ("PCA 95% + LogReg", make_pipeline(StandardScaler(), PCA(0.95),
LogisticRegression(max_iter=1000))),
                ("PCA 2 + LogReg", make_pipeline(StandardScaler(), PCA(2), LogisticRegression(max_iter=1000)))]:
    print(f"{name:18s} CV accuracy = {cross_val_score(m, X_tr, y_tr, cv=5).mean():.3f}")
```

OUTPUT

```
components kept: 10  explained: [0.365 0.184 0.122 0.073 0.065 0.05  0.04  0.027 0.023 0.015]
all 13 features    CV accuracy = 0.970
PCA 95% + LogReg   CV accuracy = 0.970
PCA 2 + LogReg     CV accuracy = 0.970
```

10. Common Mistakes and Related Concepts

Common mistakes

- Forgetting to standardise (the feature with the biggest units wins PC1).
- Fitting PCA on all data before splitting (leakage). Put it in the Pipeline.
- Assuming the top components are best for prediction (PCA ignores `y`).
- Reading components as original features.

Related: SVD, LDA (supervised reduction), Kernel PCA, t-SNE, UMAP, feature selection, `explained_variance_ratio_`.

11. Quick Revision Notes

Chapter Summary

- Principal components = eigenvectors of the covariance matrix, sorted by eigenvalue.
- Explained ratio $= \lambda_j / \sum \lambda$; choose k with a scree plot or `n_components=0.95`.
- Standardise first; fit inside a Pipeline on train data only.
- Linear, unsupervised, loses interpretability.

PART F

Ensemble Learning

17. Ensemble Learning: The Big Idea

18. Stacking

19. Bagging

20. Random Forest

21. AdaBoost

22. Gradient Boosting

CHAPTER 17

Ensemble Learning: The Big Idea

1. Definition (Simple English)

In one line

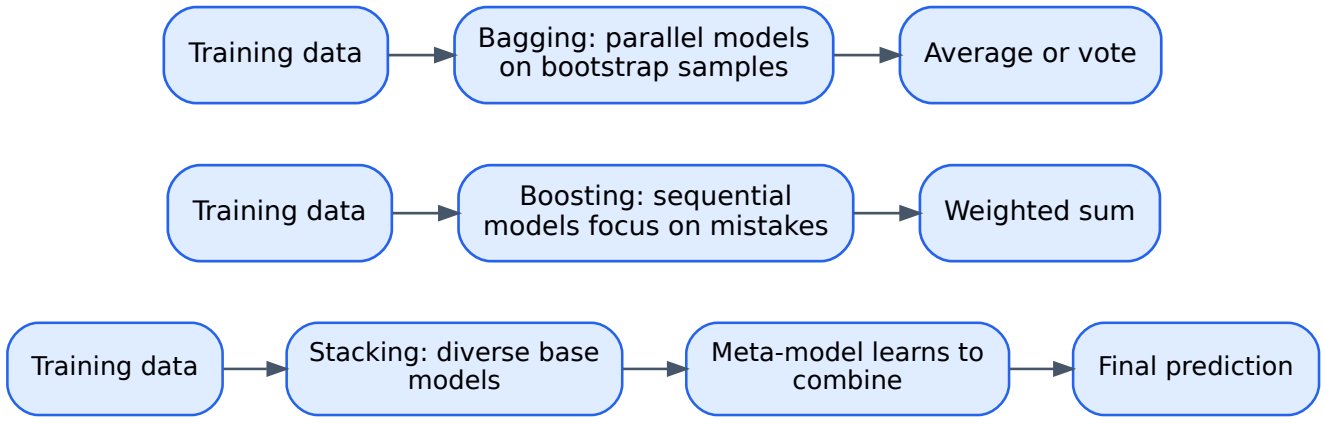
Ensemble learning combines several models so that the group predicts better than any single member. Three main strategies: **bagging** (train many models on resampled data and average, to cut variance), **boosting** (train models one after another, each fixing the previous errors, to cut bias) and **stacking** (let a meta-model learn how to combine different models).

2. Hinglish Explanation

Hinglish mein samjho

Quiz show mein "audience poll" yaad hai? Ek aadmi galat ho sakta hai, par 100 logon ka majority aksar sahi hota hai. Ensemble yahi hai. **Bagging**: bahut saare models alag-alag random data par independent train karo, phir vote/average. **Boosting**: models line mein train hote hain, har naya model pichhle ki galtiyon par dhyan deta hai. **Stacking**: alag-alag type ke models ke answers ko ek aur model ko do jo seekhe ki kis par kitna bharosa karna hai.

3. How the Three Strategies Differ



	Bagging / Random Forest	Boosting (AdaBoost, GB)	Stacking
Training	Parallel, independent	Sequential, dependent	Two levels
Main effect	Reduces **variance**	Reduces **bias** (and some variance)	Combines strengths of different models
Base learner	Deep (low-bias) trees	Shallow trees / stumps	Anything
Overfitting risk	Low	Higher; needs `learning_rate`, early stopping	Moderate; needs out-of-fold predictions

4. Why Averaging Helps

$$\mathrm{Var}\Big(\frac{1}{M}\sum_{m=1}^{M} f_m\Big) = \rho\sigma^2 + \frac{1-\rho}{M}\sigma^2$$

Worked example

Each model has variance $\sigma^2 = 1$. With $M = 100$ models and correlation $\rho = 0.3$ between them: variance $= 0.3 + 0.7/100 = 0.307$ instead of 1. If the models were perfectly correlated ($\rho = 1$) averaging would not help at all. **Diversity matters**, which is why random forests also randomise features.

PYTHON

```python
import numpy as np
rho, M, s2 = 0.3, 100, 1.0
print("ensemble variance:", rho * s2 + (1 - rho) / M * s2)
rng = np.random.RandomState(0)
# simulate: majority vote of 25 independent classifiers that are each 60% accurate
votes = (rng.rand(20000, 25) < 0.60).sum(axis=1) > 12
print("single accuracy 0.60 -> majority vote of 25 independent models:", votes.mean().round(3))
```

OUTPUT

```
ensemble variance: 0.307
single accuracy 0.60 -> majority vote of 25 independent models: 0.849
```

5. When to Use and When Not to Use

Use ensembles when accuracy matters more than interpretability and a single model is unstable or underfits. Avoid them when you need a tiny, instantly explainable model, extremely low latency, or when the base model is already near-perfect.

6. Quick Revision Notes

Chapter Summary

- Bagging = parallel + averaging (variance down). Boosting = sequential + error-correcting (bias down). Stacking = meta-learner.
- Ensembles work only if members are accurate **and** diverse.
- Trade-offs: more compute, less interpretability.

CHAPTER 18

Stacking

1. Definition (Simple English)

In one line

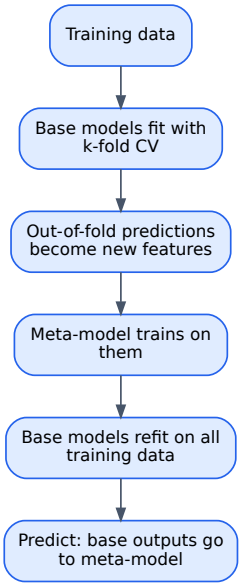
Stacking (stacked generalisation) trains several different base models, then trains a final **meta-model** on their predictions, so it learns which base model to trust and when.

2. Hinglish Explanation

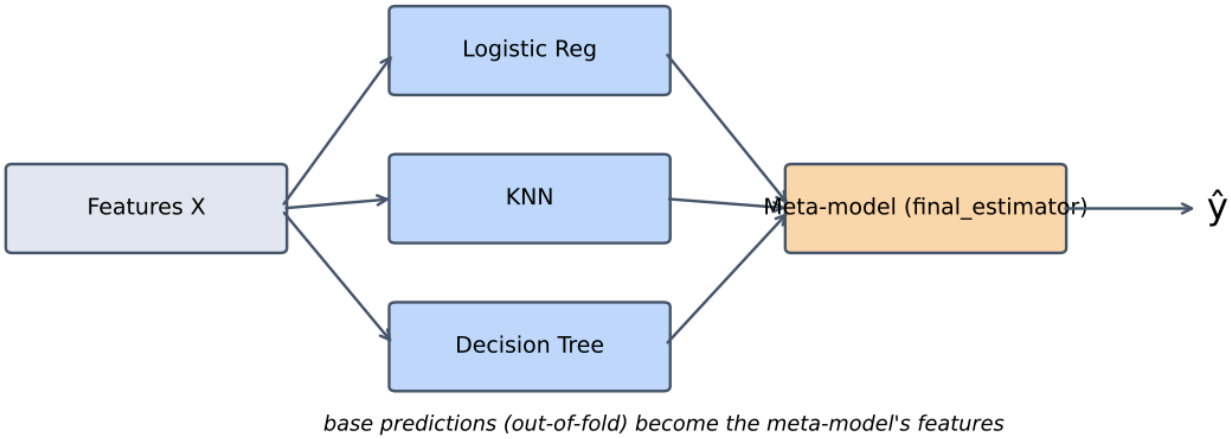
Hinglish mein samjho

Teen doctors se opinion lo (ek ka specialty heart, doosre ka lungs, teesre general). Phir ek senior doctor un teeno ki raay dekhkar final faisla leta hai, aur woh seekh chuka hai ki kaun kis cheez mein bharosemand hai. Yahan teen doctors = **base models** (Logistic Regression, KNN, Decision Tree), aur senior doctor = **meta-model** (final_estimator). Zaroori baat: meta-model ko base models ki **out-of-fold** predictions par train karte hain, warna leakage ho jata hai. scikit-learn yeh khud kar deta hai (`cv=5`).

3. How It Works, Step by Step



4. Diagram



Stacking: diverse base learners feed a meta-model.

5. Formulas with a Numerical Example

$$\hat{y} = g\big(f_1(\mathbf{x}), f_2(\mathbf{x}), \ldots, f_M(\mathbf{x})\big)$$

For a linear meta-model $g = \sum_m w_m f_m + b$, the weights show how much each base model is trusted.

Worked example

Base models output probabilities for one sample: $f_1 = 0.8,\ f_2 = 0.4,\ f_3 = 0.7$. The meta-model learned $w = (1.5, 0.5, 1.0)$, $b = -1.7$: $z = 1.5(0.8) + 0.5(0.4) + 1.0(0.7) - 1.7 = 0.4$, $p = \sigma(0.4) = 0.599$, so predict class 1. A plain average would give 0.633; stacking trusts model 1 more.

PYTHON

```
import numpy as np
f = np.array([0.8, 0.4, 0.7]); w = np.array([1.5, 0.5, 1.0]); b = -1.7
z = w @ f + b; print("z =", round(z, 2), " p =", round(1 / (1 + np.exp(-z)), 3), " simple average =",
round(f.mean(), 3))
```

OUTPUT

```
z = 0.4  p = 0.599  simple average = 0.633
```

6. Real-World Applications

Kaggle-style competitions, credit-risk scoring combining linear and tree models, demand forecasting with several model families, and any task where models make different kinds of errors.

7. When to Use and When Not to Use

Use it when	Avoid it when
Base models are diverse and individually good	Data is tiny (meta-model overfits)
You need the last bit of accuracy	Latency/complexity budget is tight
Models make uncorrelated errors	One model clearly dominates the others

8. Advantages, Disadvantages and Limitations

Advantages	Disadvantages
Often beats every single base model	More complex to train, tune and deploy
Combines different algorithm families	Harder to explain
Meta-model weights reveal model usefulness	Leakage if out-of-fold predictions are not used (sklearn handles this)

9. Python Example (scikit-learn)

PYTHON

```python
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
from sklearn.ensemble import StackingClassifier

X, y = load_breast_cancer(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.25, stratify=y, random_state=0)
base = [("lr", make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))),
        ("knn", make_pipeline(StandardScaler(), KNeighborsClassifier(7))),
        ("tree", DecisionTreeClassifier(max_depth=4, random_state=0)),
        ("svc", make_pipeline(StandardScaler(), SVC(probability=True, random_state=0)))]
stack = StackingClassifier(estimators=base, final_estimator=LogisticRegression(), cv=5)
for name, m in base + [("STACK", stack)]:
    print(f"{name:6s} CV accuracy = {cross_val_score(m, X_tr, y_tr, cv=5).mean():.3f}")
stack.fit(X_tr, y_tr); print("stack test accuracy:", round(stack.score(X_te, y_te), 3))
print("meta-model weights (lr, knn, tree, svc):", stack.final_estimator_.coef_.round(2))
```

OUTPUT

```
lr     CV accuracy = 0.974
knn    CV accuracy = 0.970
tree   CV accuracy = 0.923
svc    CV accuracy = 0.986
STACK  CV accuracy = 0.981
stack test accuracy: 0.958
meta-model weights (lr, knn, tree, svc): [[2.59 2.22 0.75 3.11]]
```

10. Common Mistakes and Related Concepts

Common mistakes

- Hand-building stacking by training the meta-model on in-sample base predictions (leakage).
- Stacking near-identical models (no diversity).
- Using a complex meta-model (a simple one such as logistic/ridge regression usually works best).

Related: `VotingClassifier` (simple average/vote), blending, `StackingRegressor`, multi-level stacking.

11. Quick Revision Notes

Chapter Summary

- Base models → out-of-fold predictions → meta-model → final output.
- Needs diversity; keep the meta-model simple.
- `StackingClassifier(estimators, final_estimator, cv=5)` handles leakage for you.
- Voting = fixed combination; stacking = learned combination.

CHAPTER 19

Bagging

1. Definition (Simple English)

In one line

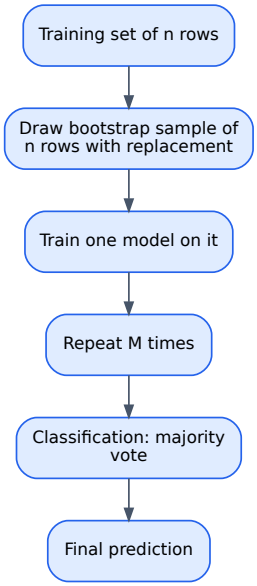
Bagging (Bootstrap AGGregatING) trains many copies of the same model, each on a different random sample of the training data drawn *with replacement* (a bootstrap sample), and averages their predictions (or takes a majority vote).

2. Hinglish Explanation

Hinglish mein samjho

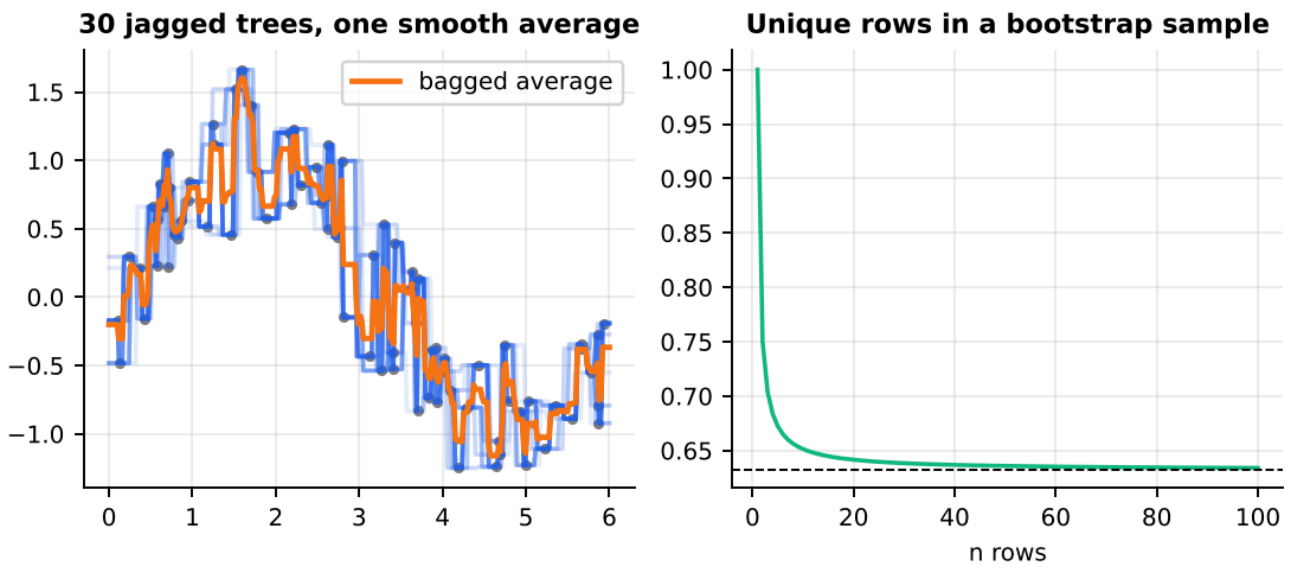
Ek bada decision tree data ko ratta maar leta hai: thoda data badla to poora tree badal jata hai (high variance). Bagging mein hum data ki **random copies** banate hain (ek hi row do-teen baar aa sakti hai, kuch rows chhoot jati hain), har copy par alag tree train karte hain, aur sab ka **average/vote** lete hain. Ek-ek tree galti karta hai, par galtiyan alag-alag jagah hoti hain, isliye average mein kat jati hain. Jo rows kisi tree ne dekhi hi nahi (**out-of-bag**), unse free mein validation score mil jata hai.

3. How It Works, Step by Step



Each bootstrap sample contains about 63% unique rows; the remaining ~37% are **out-of-bag (OOB)** and can validate that model.

4. Diagram



Left: individual overfit trees (blue) average into a stable curve (orange). Right: a bootstrap sample holds about 63% unique rows.

5. Formulas with a Numerical Example

$$\hat{f}_{\text{bag}}(x) = \frac{1}{M}\sum_{m=1}^{M}\hat{f}_m(x), \qquad P(\text{row not picked}) = \left(1-\frac{1}{n}\right)^n \xrightarrow{n\to\infty} e^{-1} \approx 0.368$$

Worked example

Five trees predict a house price (lakh ₹) of $48, 52, 50, 61, 49$. Bagged average $= (48+52+50+61+49)/5 = 52$; the median is 50. For classification with votes $[1, 1, 0, 1, 0]$ the majority vote is class 1 (3 of 5). For $n = 10$ rows, the chance a given row is missing from one bootstrap sample is $(0.9)^{10} = 0.349$.

PYTHON

```
import numpy as np
print("average:", np.mean([48, 52, 50, 61, 49]), " vote:", np.bincount([1, 1, 0, 1, 0]).argmax())
for n in (10, 100, 10000): print(n, "P(row left out) =", round((1 - 1 / n) ** n, 3))
```

OUTPUT

```
average: 52.0  vote: 1
10 P(row left out) = 0.349
100 P(row left out) = 0.366
10000 P(row left out) = 0.368
```

6. Real-World Applications

Stabilising high-variance models (deep trees, KNN), credit scoring, medical prediction, and as the foundation of Random Forest and Extra Trees.

7. When to Use and When Not to Use

Use it when	Avoid it when
The base model has high variance (overfits)	The base model is high-bias (linear model on curved data): bagging will not fix underfitting
You want a drop-in accuracy boost	You need a single interpretable model
You can train in parallel (`n_jobs`)	Prediction latency is critical

8. Advantages, Disadvantages and Limitations

Advantages	Disadvantages
Large variance reduction, hard to overfit by adding trees	Does not reduce bias
Free OOB error estimate	M times slower and larger than one model
Parallel; works with any base estimator	Less interpretable than one tree

9. Python Example (scikit-learn)

PYTHON

```python
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import BaggingClassifier

X, y = load_breast_cancer(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.25, stratify=y, random_state=0)
tree = DecisionTreeClassifier(random_state=0).fit(X_tr, y_tr)
bag = BaggingClassifier(estimator=DecisionTreeClassifier(), n_estimators=100, oob_score=True, n_jobs=1,
random_state=0).fit(X_tr, y_tr)
print("single deep tree test accuracy:", round(tree.score(X_te, y_te), 3))
print("bagged trees   test accuracy:", round(bag.score(X_te, y_te), 3))
print("out-of-bag accuracy (free validation):", round(bag.oob_score_, 3))
print("rows used by first estimator:", len(set(bag.estimators_samples_[0])), "unique of", len(X_tr))
```

OUTPUT

```
single deep tree test accuracy: 0.902
bagged trees   test accuracy: 0.937
out-of-bag accuracy (free validation): 0.96
rows used by first estimator: 265 unique of 426
```

10. Common Mistakes and Related Concepts

Common mistakes

- Bagging a stable/linear model and expecting gains.
- Forgetting that OOB score needs enough trees (each row must be out-of-bag at least once).
- Using the parameter name `base_estimator` (it is now `estimator`).

Related: Random Forest (bagging + feature randomness), Extra Trees, pasting (sampling without replacement), `max_samples` , `max_features` .

11. Quick Revision Notes

Chapter Summary

- Bootstrap samples + parallel models + average/vote → lower variance.

- ~63% unique rows per sample; ~37% out-of-bag gives free validation (`oob_score=True`).
- Best with high-variance learners such as deep trees.

CHAPTER 20

Random Forest

1. Definition (Simple English)

In one line

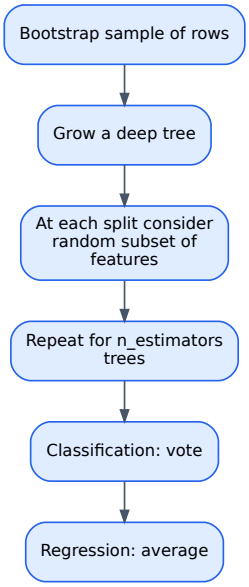
A **Random Forest** is a bagged collection of decision trees where each split considers only a random subset of features. The extra randomness makes the trees different from each other, so averaging them works even better.

2. Hinglish Explanation

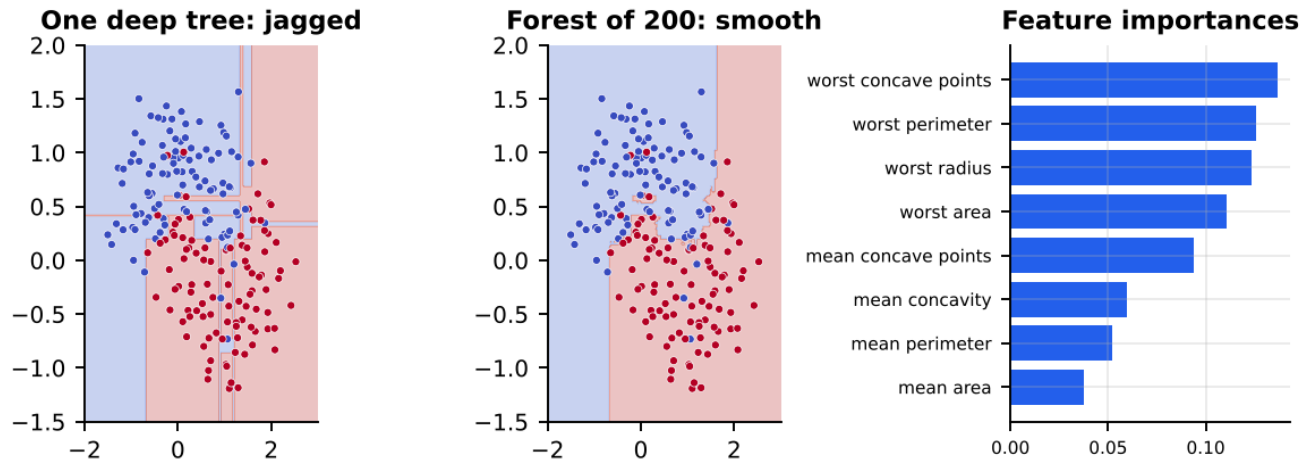
Hinglish mein samjho

Bagging mein agar ek feature bahut strong hai to saare trees pehle usi se split karenge aur sab ek jaise ho jayenge. Random Forest mein har split par sirf **kuch random features** dekhne ko milte hain, isliye trees ek doosre se alag sochte hain. Phir sab trees ka vote/average. Isliye forest ek tree se kahin zyada stable hai, aur bina zyada tuning ke bhi accha chalta hai. Bonus: yeh batata hai ki kaun se features zyada important hain.

3. How It Works, Step by Step



4. Diagram



A single tree overfits; the forest smooths the boundary. Right: impurity-based feature importances.

5. Formulas with a Numerical Example

$$\hat{y}(x) = \text{mode}\{T_m(x)\}_{m=1}^{M} \ \text{(class)}, \qquad \hat{y}(x) = \frac{1}{M}\sum_{m} T_m(x) \ \text{(regression)}, \qquad m_{\text{try}} \approx \sqrt{p} \ \text{(classification)}$$

Worked example

With $p = 30$ features, each split considers $\sqrt{30} \approx 5$ random features (`max_features="sqrt"`). For one sample the 100 trees give 70 votes for class 1 and 30 for class 0, so `predict_proba` $= [0.30, 0.70]$ and the forest predicts class 1. Gini importance of a feature is the total impurity decrease from all splits using it, averaged over trees and normalised to sum 1.

PYTHON

```python
import numpy as np
print("features per split for p=30:", int(np.sqrt(30)), "| p=100:", int(np.sqrt(100)))
votes = np.array([1] * 70 + [0] * 30); print("proba:", [round((votes == c).mean(), 2) for c in (0, 1)], "-> class",
np.bincount(votes).argmax())
```

OUTPUT

```
features per split for p=30: 5 | p=100: 10
proba: [np.float64(0.3), np.float64(0.7)] -> class 1
```

6. Real-World Applications

Credit scoring, churn and fraud detection, medical diagnosis, remote-sensing land classification, quick strong baselines for almost any tabular dataset.

7. When to Use and When Not to Use

Use it when	Avoid it when
Tabular data, mixed features, non-linear patterns	Very high-dimensional sparse text (linear models are better)
You want a strong model with little tuning	You must extrapolate beyond the training range
You need importance scores and OOB validation	Model size/latency must be tiny

8. Advantages, Disadvantages and Limitations

Advantages	Disadvantages
Accurate, robust to outliers and noise	Large and slower than one tree
No scaling needed; handles interactions	Less interpretable
Little tuning; OOB score; parallel	Impurity importances favour high-cardinality features (use permutation importance)

9. Python Example (scikit-learn)

PYTHON

```python
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.ensemble import RandomForestClassifier
from sklearn.inspection import permutation_importance

d = load_breast_cancer()
X_tr, X_te, y_tr, y_te = train_test_split(d.data, d.target, test_size=0.25, stratify=d.target, random_state=0)
rf = RandomForestClassifier(n_estimators=300, max_features="sqrt", min_samples_leaf=2, oob_score=True,
random_state=0, n_jobs=1).fit(X_tr, y_tr)
print("OOB accuracy :", round(rf.oob_score_, 3))
print("test accuracy:", round(rf.score(X_te, y_te), 3))
print("5-fold CV    :", cross_val_score(RandomForestClassifier(200, random_state=0), X_tr, y_tr,
cv=5).mean().round(3))
pi = permutation_importance(rf, X_te, y_te, n_repeats=10, random_state=0)
top = pi.importances_mean.argsort()[::-1][:3]
print("top permutation importances:", [(d.feature_names[i], round(pi.importances_mean[i], 3)) for i in top])
```

OUTPUT

```
OOB accuracy : 0.962
test accuracy: 0.951
5-fold CV    : 0.955
top permutation importances: [(np.str_('worst radius'), np.float64(0.015)), (np.str_('worst concave points'),
np.float64(0.014)), (np.str_('mean concave points'), np.float64(0.013))]
```

10. Common Mistakes and Related Concepts

Common mistakes

- Trusting Gini importance blindly (biased); confirm with permutation importance.
- Setting `n_estimators` too low; leaving trees unlimited on huge data (memory).
- Expecting good extrapolation in regression (predictions stay within the training range).
- Encoding categories as arbitrary integers and assuming order is meaningless to trees.

Related: Bagging, Extra Trees, Gradient Boosting, `max_depth`, `min_samples_leaf`, `class_weight="balanced"`, `IsolationForest`.

11. Quick Revision Notes

Chapter Summary

- Random Forest = bagging + random feature subset at each split.
- Key knobs: `n_estimators`, `max_features`, `max_depth`, `min_samples_leaf`.
- OOB score is a built-in validation; prefer permutation importance.
- Strong default for tabular data; not good at extrapolation.

CHAPTER 21

AdaBoost

1. Definition (Simple English)

In one line

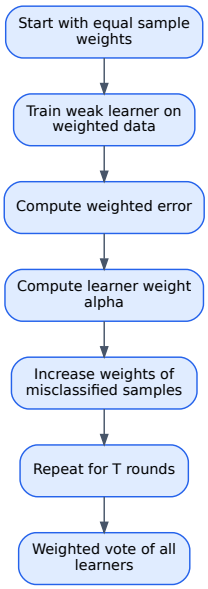
AdaBoost (Adaptive Boosting) builds a strong classifier from many weak ones (usually one-split "stumps"), training them in sequence while increasing the weight of samples that earlier models got wrong, and weighting each model's vote by its accuracy.

2. Hinglish Explanation

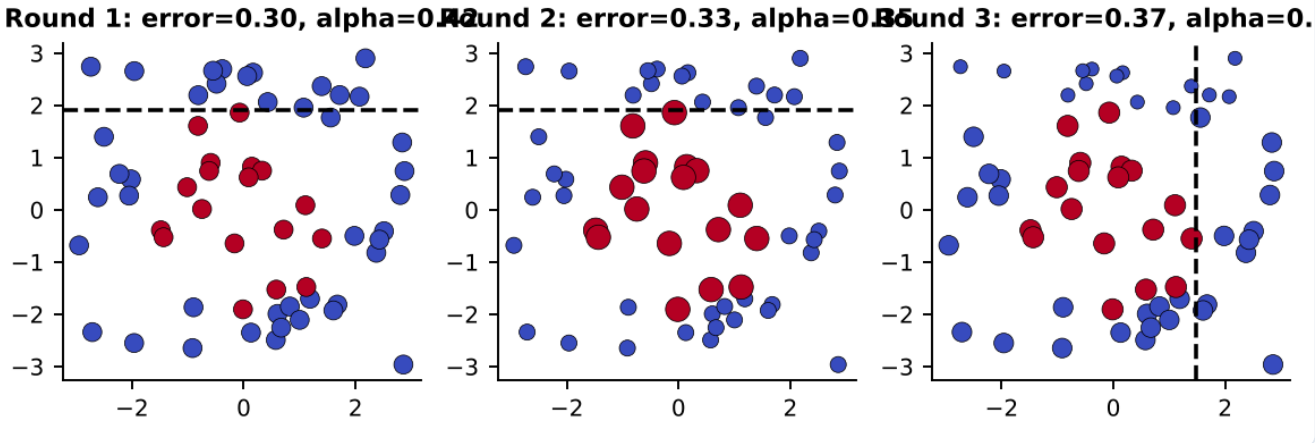
Hinglish mein samjho

Imagine ek teacher jo har test ke baad un sawaalon par zyada dhyan deta hai jo bachche galat karte hain. AdaBoost aisa hi hai. Pehla chhota tree (stump) data par train hota hai; jo samples galat nikle unka **weight badh jata hai**, to agla stump unhe sahi karne par focus karta hai. Aise 50-100 stumps banate hain. Final answer mein jo stump zyada accurate hai uska vote zyada bhaari (α) hota hai. Noise aur outliers par dhyan rakho, kyunki woh bhi baar-baar "mushkil sawal" ban jate hain.

3. How It Works, Step by Step



4. Diagram



Marker size = sample weight. Misclassified points grow, so the next stump focuses on them.

5. Formulas with a Numerical Example

$$\varepsilon_t = \sum_i w_i \, 1[h_t(x_i) \neq y_i], \quad \alpha_t = \tfrac{1}{2} \ln \frac{1-\varepsilon_t}{\varepsilon_t}, \quad w_i \leftarrow \frac{w_i \, e^{-\alpha_t y_i h_t(x_i)}}{Z}, \quad H(x) = \operatorname{sign}\Big(\sum_t \alpha_t h_t(x)\Big)$$

Worked example

10 samples, initial weights 0.1 each. Stump 1 misclassifies 2 samples: $\varepsilon = 0.2$, $\alpha = \frac{1}{2}\ln(0.8/0.2) = 0.693$. Wrong samples: $0.1 \times e^{0.693} = 0.2$ each; correct samples: $0.1 \times e^{-0.693} = 0.05$ each. Sum $= 2(0.2) + 8(0.05) = 0.8$, so normalise ($Z = 0.8$): wrong $= 0.25$ each, correct $= 0.0625$ each. The two hard samples now carry half of the total weight.

PYTHON

```python
import numpy as np
eps = 0.2; alpha = 0.5 * np.log((1 - eps) / eps)
wrong, right = 0.1 * np.exp(alpha), 0.1 * np.exp(-alpha); Z = 2 * wrong + 8 * right
print(f"alpha={alpha:.3f}  new weights: wrong={wrong / Z:.4f} right={right / Z:.4f}  (wrong total={2 * wrong / Z:.2f})")
```

OUTPUT

```
alpha=0.693  new weights: wrong=0.2500 right=0.0625  (wrong total=0.50)
```

6. Real-World Applications

Face detection (the classic Viola-Jones detector), text categorisation, customer-churn baselines, and as a simple boosting benchmark on clean tabular data.

7. When to Use and When Not to Use

Use it when	Avoid it when
Data is relatively clean	Labels are noisy or there are many outliers
You want a simple boosted baseline with stumps	You need maximum accuracy (gradient boosting usually wins)
Weak learners are slightly better than random	Training must run in parallel

8. Advantages, Disadvantages and Limitations

Advantages	Disadvantages
Few parameters, simple idea	Sensitive to noise and outliers (their weights explode)
Improves weak learners dramatically	Sequential, cannot be parallelised across rounds
Resists overfitting reasonably on clean data	Usually outperformed by modern gradient boosting

9. Python Example (scikit-learn)

PYTHON

```python
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import AdaBoostClassifier

X, y = load_breast_cancer(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.25, stratify=y, random_state=0)
stump = DecisionTreeClassifier(max_depth=1).fit(X_tr, y_tr)
print("single stump test accuracy:", round(stump.score(X_te, y_te), 3))
ada = AdaBoostClassifier(estimator=DecisionTreeClassifier(max_depth=1), n_estimators=100, learning_rate=0.5,
random_state=0).fit(X_tr, y_tr)
print("AdaBoost (100 stumps) test accuracy:", round(ada.score(X_te, y_te), 3))
gs = GridSearchCV(AdaBoostClassifier(estimator=DecisionTreeClassifier(max_depth=1), random_state=0),
                  {"n_estimators": [50, 150], "learning_rate": [0.1, 0.5, 1.0]}, cv=5).fit(X_tr, y_tr)
print("best:", gs.best_params_, round(gs.best_score_, 3))
print("first 5 learner weights:", ada.estimator_weights_[:5].round(2))
```

OUTPUT

```
single stump test accuracy: 0.888
AdaBoost (100 stumps) test accuracy: 0.937
best: {'learning_rate': 0.5, 'n_estimators': 150} 0.96
first 5 learner weights: [1.29 1.17 0.77 0.77 0.66]
```

10. Common Mistakes and Related Concepts

Common mistakes

- Boosting very deep trees (weak learners should be weak).
- Using AdaBoost on noisy labels without cleaning.
- Setting a large `learning_rate` with many estimators and wondering why it overfits.

Related: Gradient Boosting, XGBoost/LightGBM, bagging (parallel, variance) vs boosting (sequential, bias), exponential loss.

11. Quick Revision Notes

Chapter Summary

- Sequential stumps; misclassified samples get higher weights.
- $\alpha_t = \frac{1}{2}\ln\frac{1-\varepsilon_t}{\varepsilon_t}$ is each learner's vote weight.
- Tune `n_estimators` and `learning_rate` together.
- Sensitive to noise; GB usually better.

CHAPTER 22

Gradient Boosting

1. Definition (Simple English)

In one line

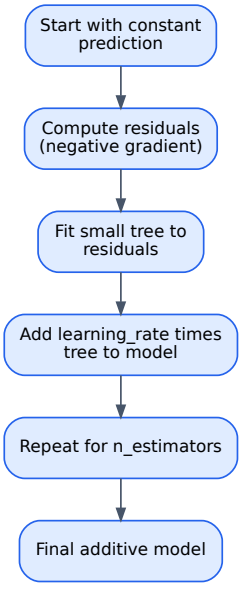
Gradient Boosting builds an additive model by adding small decision trees one at a time, where each new tree is trained to predict the *errors (residuals)* of the current model, scaled by a small learning rate.

2. Hinglish Explanation

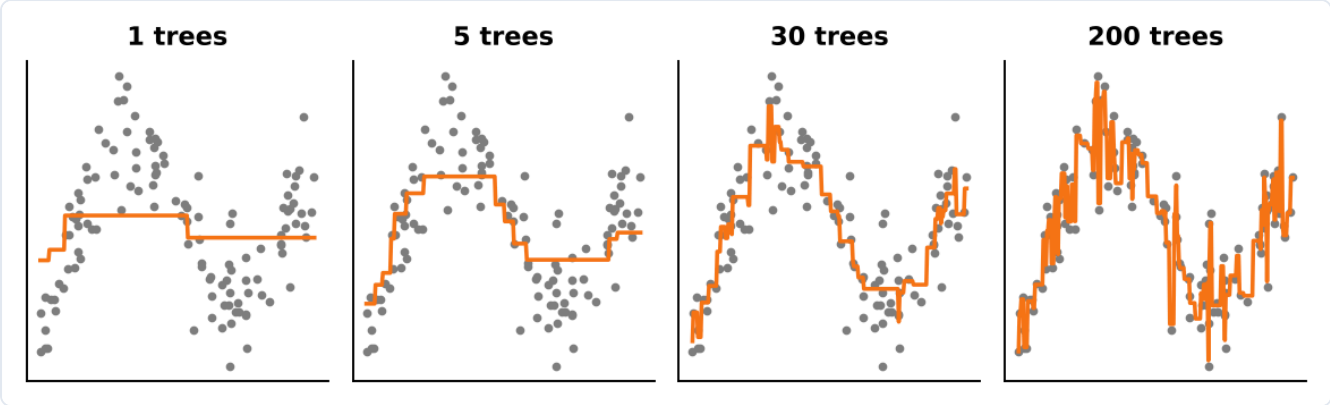
Hinglish mein samjho

Pehla model average price bol deta hai (jaise 50 lakh). Ab har ghar ki **galti** nikalo (asli price minus 50). Ek chhota tree in galtiyon ko predict karna seekhta hai. Hum is tree ki raay ka sirf thoda hissa (`learning_rate` , jaise 0.1) jodte hain, phir nayi galtiyan nikalte hain aur agla tree banate hain. Sau-do sau baar yeh karne par galtiyan bahut kam ho jati hain. Isse "gradient" isliye kehte hain kyunki squared-error ke liye residual hi loss ka negative gradient hota hai, aur doosre losses ke liye bhi yahi idea chalta hai.

3. How It Works, Step by Step



4. Diagram



The model starts crude and improves as trees are added; each tree corrects what is left.

5. Formulas with a Numerical Example

$$F_0 = \bar{y}, \qquad r_i = y_i - F_{m-1}(x_i), \qquad F_m(x) = F_{m-1}(x) + \eta\, h_m(x)$$

where h_m is a small tree fitted to residuals r_i and η is the learning rate.

Worked example

Prices (lakh ₹) $y = [40, 50, 60, 90]$, $F_0 = \bar{y} = 60$. Residuals $r = [-20, -10, 0, 30]$. Suppose tree 1 predicts $[-15, -15, 0, 30]$ (it groups the first two). With $\eta = 0.5$: $F_1 = 60 + 0.5 \times [-15, -15, 0, 30] = [52.5, 52.5, 60, 75]$. New residuals $= [-12.5, -2.5, 0, 15]$; the largest error fell from 30 to 15 after one step. Squared error dropped from 1400 to 387.5.

PYTHON

```python
import numpy as np
y = np.array([40, 50, 60, 90.]); F0 = y.mean(); r = y - F0; h = np.array([-15, -15, 0, 30.]); F1 = F0 + 0.5 * h
print("F0:", F0, " residuals:", r, " SSE:", (r ** 2).sum())
print("F1:", F1, " residuals:", y - F1, " SSE:", ((y - F1) ** 2).sum())
```

OUTPUT

```
F0: 60.0  residuals: [-20. -10.   0.  30.]  SSE: 1400.0
F1: [52.5 52.5 60.  75. ]  residuals: [-12.5  -2.5   0.   15. ]  SSE: 387.5
```

Key knobs: `n_estimators`, `learning_rate` (smaller = needs more trees, usually better), `max_depth` (3-5 is typical), `subsample` < 1 (stochastic GB), `n_iter_no_change` for early stopping. `HistGradientBoosting*` is the fast, modern scikit-learn version for larger data and handles missing values natively.

6. Real-World Applications

Click-through and ad ranking, credit risk, insurance pricing, demand forecasting, and the winning approach (via XGBoost, LightGBM, CatBoost) in most tabular-data competitions.

7. When to Use and When Not to Use

Use it when	Avoid it when
Tabular data and you want top accuracy	Data is tiny and noisy (overfits)
Mixed feature types and non-linear interactions	You need parallel training of a simple model (use Random Forest)
You can afford tuning	Images, text, audio (use deep learning)

8. Advantages, Disadvantages and Limitations

Advantages	Disadvantages
Often the most accurate tabular model	Many hyper-parameters; sensitive to tuning
Flexible loss functions; handles non-linearity	Sequential: slower to train than a forest
Feature importances; works on mixed scales	Overfits without `learning_rate`, `subsample`, early stopping; cannot extrapolate

9. Python Example (scikit-learn)

PYTHON

```python
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.ensemble import GradientBoostingRegressor, HistGradientBoostingRegressor, RandomForestRegressor
from sklearn.linear_model import LinearRegression
from sklearn.metrics import root_mean_squared_error, r2_score

X, y = load_diabetes(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.25, random_state=0)
models = {"Linear": LinearRegression(),
          "RandomForest": RandomForestRegressor(300, random_state=0),
          "GradientBoosting": GradientBoostingRegressor(n_estimators=300, learning_rate=0.03, max_depth=3,
subsample=0.8, random_state=0),
          "HistGB(early stop)": HistGradientBoostingRegressor(learning_rate=0.05, max_iter=500,
early_stopping=True, validation_fraction=0.2, n_iter_no_change=20, random_state=0)}
for n, m in models.items():
    p = m.fit(X_tr, y_tr).predict(X_te)
    print(f"{n:19s} RMSE={root_mean_squared_error(y_te, p):6.2f}  R2={r2_score(y_te, p):.3f}")
gb = models["GradientBoosting"]
stage_rmse = [root_mean_squared_error(y_te, p) for p in gb.staged_predict(X_te)]
print("best number of trees on test curve:", int(min(range(300), key=lambda i: stage_rmse[i])) + 1, "(for
illustration only; choose via validation)")
```

OUTPUT

```
Linear              RMSE= 56.39  R2=0.359
RandomForest        RMSE= 61.13  R2=0.247
GradientBoosting    RMSE= 61.27  R2=0.244
HistGB(early stop)  RMSE= 58.81  R2=0.303
best number of trees on test curve: 60 (for illustration only; choose via validation)
```

10. Common Mistakes and Related Concepts

Common mistakes

- High learning rate + many trees = overfitting; tune them together, use early stopping.
- Choosing `n_estimators` by peeking at the test curve (use a validation split).
- Deep trees as base learners.
- Expecting it to extrapolate trends.

Related: XGBoost, LightGBM, CatBoost, `HistGradientBoosting`, AdaBoost, shrinkage, stochastic gradient boosting.

11. Quick Revision Notes

Chapter Summary

- Additive trees fitted to residuals (negative gradients): $F_m = F_{m-1} + \eta h_m$.
- Small `learning_rate` + more trees + shallow trees + `subsample` = robust.
- Use early stopping; `HistGradientBoosting` for speed and native NaN support.
- Bagging cuts variance in parallel; boosting cuts bias sequentially.

PART G

Data Preprocessing

23. IQR and Outlier Detection

24. Pipeline

25. ColumnTransformer

26. OrdinalEncoder

27. OneHotEncoder

28. LabelEncoder

29. StandardScaler

30. Missing Value Handling

CHAPTER 23

IQR and Outlier Detection

1. Definition (Simple English)

In one line

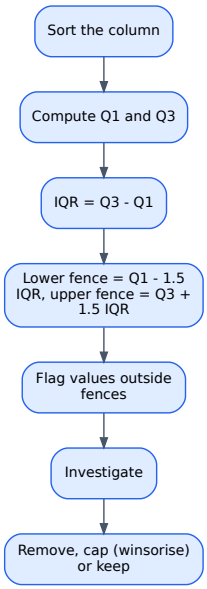
An **outlier** is a value far away from the rest of the data. The **IQR method** flags values below $Q_1 - 1.5\,\mathrm{IQR}$ or above $Q_3 + 1.5\,\mathrm{IQR}$, where $\mathrm{IQR} = Q_3 - Q_1$ is the spread of the middle 50% of the data.

2. Hinglish Explanation

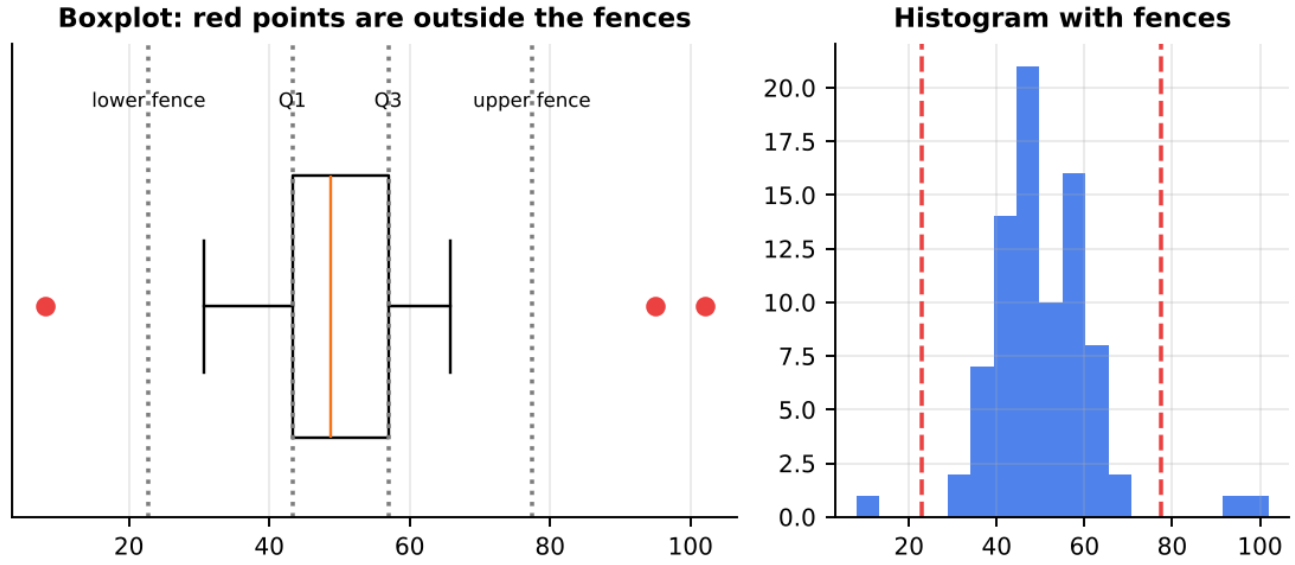
Hinglish mein samjho

Data ko sort karo aur 4 barabar hisson mein baanto. Q_1 (25% wali value) aur Q_3 (75% wali value) ke beech ka gap **IQR** hai, yaani "beech ke 50% log kitne phaile hain". Jo value is band se bahut bahar hai (1.5 x IQR se zyada) woh **outlier** hai. Jaise 10 logon ki salary 30-60 hazaar hai aur ek ki 10 lakh: yeh outlier hai. Outlier hamesha galat nahi hota (fraud ya rare event ho sakta hai), isliye hatane se pehle **samjho ki kyun hai**.

3. How It Works, Step by Step



4. Diagram



Whiskers extend to the last point inside the fences; points beyond are flagged.

5. Formulas with a Numerical Example

$$\text{IQR} = Q_3 - Q_1, \qquad \text{lower} = Q_1 - 1.5\,\text{IQR}, \qquad \text{upper} = Q_3 + 1.5\,\text{IQR}$$

Worked example

Monthly salaries (thousand ₹): $[30, 32, 35, 38, 40, 42, 45, 48, 50, 400]$. With linear interpolation $Q_1 = 35.75$, $Q_3 = 47.25$, so IQR $= 11.5$. Fences: $35.75 - 17.25 = 18.5$ and $47.25 + 17.25 = 64.5$. The value 400 is above 64.5, so it is an outlier; the rest are fine. For comparison the z-score of 400 is only about 3.0 (right at the usual cut-off, and the largest possible for 10 points) because the outlier inflates the standard deviation itself, a reason IQR is more *robust* than z-scores.

PYTHON

```python
import numpy as np
s = np.array([30, 32, 35, 38, 40, 42, 45, 48, 50, 400])
q1, q3 = np.percentile(s, [25, 75]); iqr = q3 - q1; lo, hi = q1 - 1.5 * iqr, q3 + 1.5 * iqr
print(f"Q1={q1} Q3={q3} IQR={iqr} fences=({lo}, {hi}) outliers={s[(s < lo) | (s > hi)]}")
print("z-score of 400:", round((400 - s.mean()) / s.std(), 2))
```

OUTPUT

```
Q1=35.75 Q3=47.25 IQR=11.5 fences=(18.5, 64.5) outliers=[400]
z-score of 400: 3.0
```

6. Real-World Applications

Cleaning salary or price data, spotting sensor faults, flagging suspicious transactions for review, deciding whether to cap extreme values before regression or distance-based models.

7. When to Use and When Not to Use

Use it when	Avoid it when
One numeric column, roughly unimodal, skewed or not	Data is multimodal or has natural heavy tails (price, income: consider log first)
You want a robust rule without assuming normality	Outliers are the signal (fraud, rare events)
Tree models: usually leave outliers alone	Multivariate outliers (use Isolation Forest / LOF)

8. Advantages, Disadvantages and Limitations

Advantages	Disadvantages
Simple, robust, no distribution assumption	Univariate: misses outliers that are only unusual in combination
Easy to visualise (boxplot)	The 1.5 multiplier is a convention
Resistant to the outliers themselves	May flag many points in skewed data

9. Python Example (scikit-learn)

PYTHON

```python
import numpy as np, pandas as pd
from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression

class IQRCapper(BaseEstimator, TransformerMixin):
    """Learns fences on TRAIN data only, then caps (winsorises) any data it sees."""
    def __init__(self, k=1.5): self.k = k
    def fit(self, X, y=None):
        X = np.asarray(X, float); q1, q3 = np.percentile(X, [25, 75], axis=0)
        self.lo_, self.hi_ = q1 - self.k * (q3 - q1), q3 + self.k * (q3 - q1); return self
    def transform(self, X): return np.clip(np.asarray(X, float), self.lo_, self.hi_)

rng = np.random.RandomState(0)
x = rng.normal(50, 10, 300); y = 3 * x + rng.normal(0, 10, 300)
x[:5] = [400, 380, 420, -200, 350]                    # inject outliers (with unrelated y)
df = pd.DataFrame({"x": x, "y": y})
tr, te = train_test_split(df, test_size=0.3, random_state=0)
q1, q3 = tr.x.quantile([.25, .75]); i = q3 - q1
print("outliers flagged in train:", int(((tr.x < q1 - 1.5 * i) | (tr.x > q3 + 1.5 * i)).sum()))
for name, steps in [("raw", [LinearRegression()]), ("IQR-capped", [IQRCapper(), LinearRegression()])]:
    m = make_pipeline(*steps).fit(tr[["x"]], tr.y)
    print(f"{name:10s} slope={m[-1].coef_[0]:.2f} (true 3.0)  test R2={m.score(te[['x']], te.y):.3f}")
```

OUTPUT

```
outliers flagged in train: 5
raw        slope=0.18 (true 3.0)  test R2=0.091
IQR-capped slope=2.59 (true 3.0)  test R2=0.869
```

10. Common Mistakes and Related Concepts

Common mistakes

- Computing fences on the full dataset before the split (leakage); learn them on train only.
- Deleting outliers blindly (lose real rare events) or deleting from the test set.
- Applying IQR to categorical or heavily skewed raw data without transformation.

Related: z-score, modified z-score (MAD), winsorising, `RobustScaler`, log transform, Isolation Forest, Local Outlier Factor.

11. Quick Revision Notes

Chapter Summary

- Fences: $Q_1 - 1.5\,\text{IQR}$ and $Q_3 + 1.5\,\text{IQR}$; IQR $= Q_3 - Q_1$.
- Robust to outliers (unlike mean/std z-score).
- Options: investigate → remove, cap, transform, or keep. Never fit fences on test data.

CHAPTER 24

Pipeline

1. Definition (Simple English)

In one line

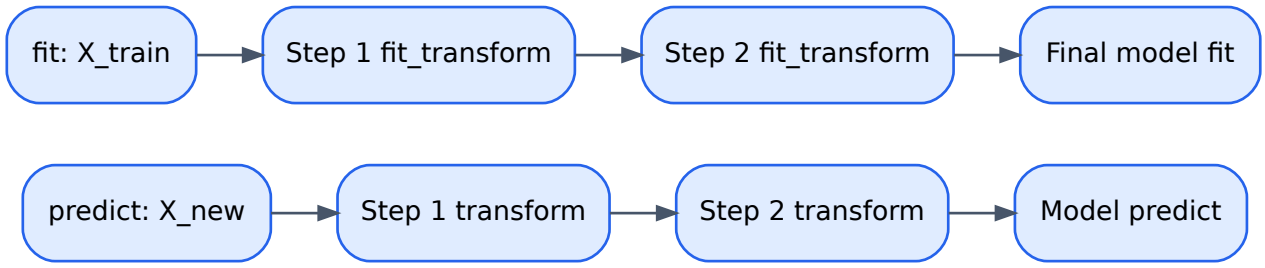
A **Pipeline** chains preprocessing steps and a final model into one object, so `fit` and `predict` run every step in the right order and each step learns *only* from the data it is fitted on.

2. Hinglish Explanation

Hinglish mein samjho

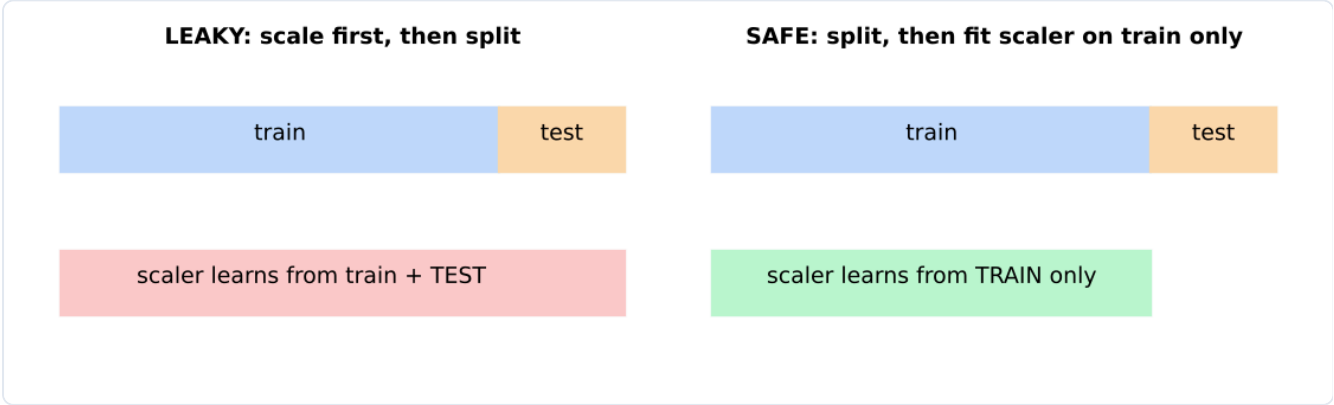
Factory assembly line socho: kaccha maal -> safai -> scaling -> model. Pipeline yahi line hai. Fayda: (1) code saaf, ek hi `fit()` / `predict()` ; (2) **data leakage band**, kyunki cross-validation ke har fold mein scaler/imputer sirf train hisse se seekhta hai; (3) deployment aasan, ek hi object save karo. Naye ho to rule yaad rakho: **koi bhi preprocessing Pipeline ke bahar mat karo.**

3. How It Works, Step by Step



Intermediate steps need `fit` and `transform`; the last step needs `fit` (and `predict`). Names are `step__parameter` so a search can tune any step.

4. Diagram: leaky vs safe



The leaky version lets test-set statistics (mean, std) influence training.

5. Formulas with a Numerical Example

There is no formula, but there is a *number* that shows leakage. Train $x = [10, 20, 30]$, test $x = [100]$.

Worked example

Safe: scaler learns $\mu = 20$, $\sigma = 8.16$ from train only. Test value $100 \to (100 - 20)/8.16 = 9.8$: the model correctly sees it is extremely unusual. **Leaky:** scaler learns from all four values: $\mu = 40$, $\sigma = 35.5$, so the same test point becomes $(100 - 40)/35.5 = 1.69$, which looks ordinary. The test set quietly changed the transformation, and offline scores become over-optimistic.

PYTHON

```python
import numpy as np
tr = np.array([10, 20, 30.]); te = 100.0
safe = (te - tr.mean()) / tr.std(); allv = np.append(tr, te); leaky = (te - allv.mean()) / allv.std()
print(f"safe z={safe:.2f}  leaky z={leaky:.2f}")
```

OUTPUT

```
safe z=9.80  leaky z=1.70
```

6. Real-World Applications

Every production ML system: imputation + encoding + scaling + model in one deployable object; tuning preprocessing choices with `GridSearchCV`; safe cross-validation.

7. When to Use and When Not to Use

Use a Pipeline **always** when preprocessing learns anything from data (scaling, imputation, encoding, PCA, feature selection). Stateless steps (such as `np.log1p`) are safe either way, but a Pipeline still keeps code tidy. Use `ColumnTransformer` (next chapter) inside it when columns need different treatment.

8. Advantages, Disadvantages and Limitations

Advantages	Disadvantages
Prevents leakage in CV and train/test	Slightly harder to inspect intermediate data (use `named_steps`, `get_feature_names_out`)
One object to tune, save and deploy	Steps must follow scikit-learn's fit/transform API
Cleaner, reproducible code	Cannot change the number of rows (use `imblearn` pipeline for resampling)

9. Python Example (scikit-learn)

PYTHON

```python
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, cross_val_score, GridSearchCV
from sklearn.pipeline import Pipeline, make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression
import joblib

X, y = load_breast_cancer(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, stratify=y, random_state=0)
pipe = Pipeline([("scale", StandardScaler()), ("pca", PCA(n_components=10)), ("clf",
LogisticRegression(max_iter=1000))])
print("steps:", list(pipe.named_steps))
print("CV accuracy (scaler refit in each fold):", cross_val_score(pipe, X_tr, y_tr, cv=5).mean().round(3))
gs = GridSearchCV(pipe, {"pca__n_components": [2, 5, 10, 20], "clf__C": [0.1, 1, 10]}, cv=5).fit(X_tr, y_tr)
print("best:", gs.best_params_, " CV:", round(gs.best_score_, 3), " test:", round(gs.score(X_te, y_te), 3))
joblib.dump(gs.best_estimator_, "model.joblib")                    # one file = preprocessing + model
print("reloaded model predicts:", joblib.load("model.joblib").predict(X_te[:5]))
```

OUTPUT

```
steps: ['scale', 'pca', 'clf']
CV accuracy (scaler refit in each fold): 0.969
best: {'clf__C': 0.1, 'pca__n_components': 20}  CV: 0.971  test: 0.965
reloaded model predicts: [0 0 0 1 0]
```

10. Common Mistakes and Related Concepts

Common mistakes

- Calling `fit_transform` on test data (use `transform`; a Pipeline does this for you).
- Scaling the whole dataset before `train_test_split`.
- Putting a transformer as the *last* step (it needs a final estimator for `predict`).
- Forgetting `step__param` naming in grid searches.

Related: `make_pipeline`, `ColumnTransformer`, `FeatureUnion`, `FunctionTransformer`, `joblib`, `set_output(transform="pandas")`.

11. Quick Revision Notes

Chapter Summary

- Pipeline = ordered steps + final estimator; `fit` learns on train only.
- Eliminates leakage inside cross-validation and simplifies deployment.
- Tune with `GridSearchCV` using `step__param`; save with `joblib`.
- Never preprocess outside the pipeline.

CHAPTER 25

ColumnTransformer

1. Definition (Simple English)

In one line

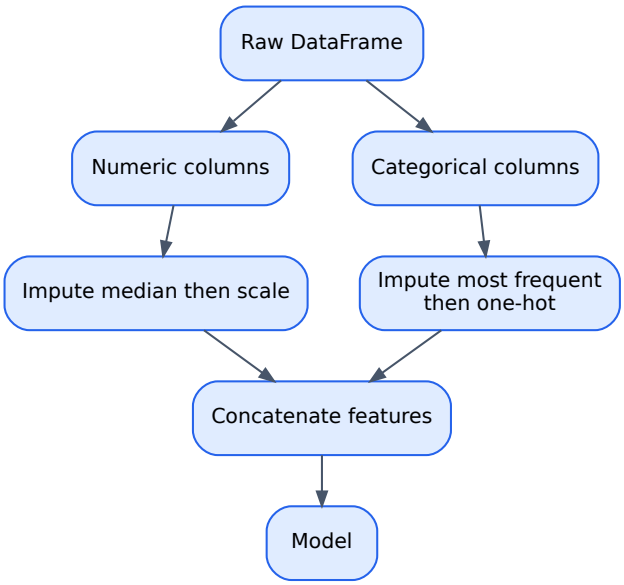
ColumnTransformer applies *different* preprocessing to *different columns* (for example scale the numbers, one-hot encode the categories) and joins the results into one feature matrix.

2. Hinglish Explanation

Hinglish mein samjho

Real dataset mein kuch columns numbers hote hain (age, salary) aur kuch text/category (city, gender). Dono par ek hi transformation nahi lag sakta. ColumnTransformer kehta hai: "numeric columns ko imputer + scaler do, categorical columns ko imputer + one-hot do", phir sabko jodkar model ko deta hai. Isko Pipeline ke andar rakhne se leakage bhi nahi hota.

3. How It Works, Step by Step



`remainder="drop"` (default) discards columns not listed; `remainder="passthrough"` keeps them unchanged.

4. Diagram: before and after

PYTHON

```
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
df = pd.DataFrame({"age": [25, 32, 47], "salary": [30, 50, 90], "city": ["Pune", "Delhi", "Pune"]})
ct = ColumnTransformer([("num", StandardScaler(), ["age", "salary"]), ("cat", OneHotEncoder(), ["city"])])
out = pd.DataFrame(ct.fit_transform(df), columns=ct.get_feature_names_out()).round(2)
print(df.to_string(index=False)); print(); print(out.to_string(index=False))
```

OUTPUT

```
 age  salary  city
  25      30  Pune
  32      50 Delhi
  47      90  Pune

 num__age  num__salary  cat__city_Delhi  cat__city_Pune
    -1.05        -1.07              0.0             1.0
    -0.29        -0.27              1.0             0.0
     1.34         1.34              0.0             1.0
```

5. Formulas with a Numerical Example

Each branch applies its own formula: scaled column $z = (x - \mu)/\sigma$; one-hot column $= 1[\text{city} = c]$.

Worked example

From the table above: ages $[25, 32, 47]$ have $\mu = 34.67$, $\sigma = 9.30$, so 47 becomes $(47 - 34.67)/9.30 = 1.33$. City "Delhi" and "Pune" become two 0/1 columns (`cat__city_Delhi` , `cat__city_Pune`). The output has $2 + 2 = 4$ columns, ordered by the transformer list.

PYTHON

```python
import numpy as np
a = np.array([25, 32, 47.]); print("z-score of 47:", round((47 - a.mean()) / a.std(), 2))
```

OUTPUT

```
z-score of 47: 1.34
```

6. Real-World Applications

Any tabular project mixing numeric, categorical, ordinal and date-derived columns: churn, loan approval, house prices, medical records.

7. When to Use and When Not to Use

Use it when	Avoid it when
Columns have different types or need different handling	All columns are numeric and need the same step (a plain Pipeline is enough)
You want one leakage-safe object	You need row-wise (not column-wise) logic
Selecting columns by dtype (`make_column_selector`)	Column order matters to downstream code and you rely on positions

8. Advantages, Disadvantages and Limitations

Advantages	Disadvantages
Clean, declarative, reusable	Output column names get prefixes (use `get_feature_names_out`)
Works inside Pipeline and GridSearch	Dropped columns are silent with `remainder="drop"`
`make_column_selector` auto-detects columns	Sparse/dense mixing can create large matrices

9. Python Example (scikit-learn)

PYTHON

```python
import numpy as np, pandas as pd
from sklearn.compose import ColumnTransformer, make_column_selector as selector
```

```
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder, OrdinalEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split, cross_val_score

rng = np.random.RandomState(0); n = 400
df = pd.DataFrame({"age": rng.randint(21, 65, n).astype(float), "income": rng.lognormal(10.5, .4, n),
                   "city": rng.choice(["Pune", "Delhi", "Mumbai"], n), "education": rng.choice(["school", "grad",
"pg"], n)})
df.loc[rng.rand(n) < .1, "age"] = np.nan; df.loc[rng.rand(n) < .05, "city"] = np.nan
y = ((df.income > 40000) ^ (df.education == "pg") ^ (rng.rand(n) < .15)).astype(int)
X_tr, X_te, y_tr, y_te = train_test_split(df, y, test_size=0.25, random_state=0, stratify=y)

num = Pipeline([("imp", SimpleImputer(strategy="median")), ("sc", StandardScaler())])
nom = Pipeline([("imp", SimpleImputer(strategy="most_frequent")), ("oh", OneHotEncoder(handle_unknown="ignore"))])
pre = ColumnTransformer([("num", num, ["age", "income"]),
                         ("nom", nom, ["city"]),
                         ("ord", OrdinalEncoder(categories=[["school", "grad", "pg"]]), ["education"])])
model = Pipeline([("pre", pre), ("clf", LogisticRegression(max_iter=500))]).fit(X_tr, y_tr)
print("feature names:", list(model["pre"].get_feature_names_out()))
print("CV accuracy:", cross_val_score(model, X_tr, y_tr, cv=5).mean().round(3), " test accuracy:",
round(model.score(X_te, y_te), 3))
auto = ColumnTransformer([("num", num, selector(dtype_include="number")), ("cat", nom,
selector(dtype_include=object))])
print("auto-selected output shape:", auto.fit_transform(X_tr).shape)
```

OUTPUT

```
feature names: ['num__age', 'num__income', 'nom__city_Delhi', 'nom__city_Mumbai', 'nom__city_Pune',
'ord__education']
CV accuracy: 0.63  test accuracy: 0.61
auto-selected output shape: (300, 8)
```

10. Common Mistakes and Related Concepts

Common mistakes

- Listing a column in two transformers, or in none while `remainder="drop"` silently discards it.
- Fitting the ColumnTransformer on all data outside a Pipeline.
- Forgetting `handle_unknown="ignore"` for categories unseen during training.

Related: `make_column_transformer`, `make_column_selector`, `FunctionTransformer`, `set_output(transform="pandas")`, `FeatureUnion`.

11. Quick Revision Notes

Chapter Summary

- `ColumnTransformer([(name, transformer, columns), ...], remainder=...)`.
- Typical: numeric → impute + scale; categorical → impute + OneHot; ordinal → OrdinalEncoder.
- Put it inside a Pipeline with the model; use `get_feature_names_out()` to read outputs.

CHAPTER 26

OrdinalEncoder

1. Definition (Simple English)

In one line

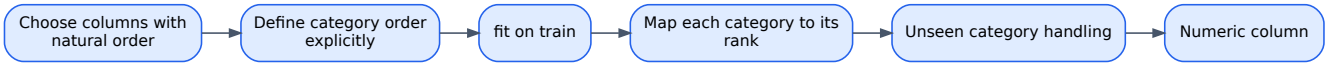
OrdinalEncoder converts categorical **feature** columns whose values have a natural order (small < medium < large) into integers 0, 1, 2, ... that preserve that order.

2. Hinglish Explanation

Hinglish mein samjho

Kuch categories mein **order** hota hai: education = school < graduate < post-graduate, ya size = S < M < L < XL. Model ko numbers chahiye, to hum inhe 0, 1, 2, 3 bana dete hain, par **order hum khud batate hain** (`categories=[[...]]`), warna scikit-learn alphabetical order maan leta hai, jo galat ho sakta hai ("L" < "M" < "S" !). Agar category mein koi order nahi (city, colour), to OrdinalEncoder mat use karo, wahan OneHotEncoder chalta hai.

3. How It Works, Step by Step



4. Diagram

Original `size`	Alphabetical (wrong)	Explicit order (right)
S	2	0
M	1	1
L	0	2
XL	3	3

Alphabetical order puts L < M < S < XL, which scrambles the real size order.

5. Formulas with a Numerical Example

$$\text{code}(c) = \text{index of } c \text{ in the ordered category list}$$

Worked example

Order `["school","graduate","postgrad"]` . Row values `[graduate, school, postgrad, graduate]` become `[1, 0, 2, 1]` . A linear model then treats the gap school→graduate as equal to graduate→postgrad, which is an *assumption* you accept when you choose ordinal coding. Tree models only use the order, not the gaps.

PYTHON

```
import numpy as np
from sklearn.preprocessing import OrdinalEncoder
enc = OrdinalEncoder(categories=[["school", "graduate", "postgrad"]])
print(enc.fit_transform(np.array([["graduate"], ["school"], ["postgrad"], ["graduate"]])).ravel())
alpha = OrdinalEncoder().fit(np.array([["S"], ["M"], ["L"], ["XL"]])); print("alphabetical:", alpha.categories_[0],
"-> codes", alpha.transform([["S"], ["M"], ["L"], ["XL"]]).ravel())
```

OUTPUT

```
[1. 0. 2. 1.]
alphabetical: ['L' 'M' 'S' 'XL'] -> codes [2. 1. 0. 3.]
```

6. Real-World Applications

Education level, satisfaction ratings (poor…excellent), clothing sizes, credit grades (A, B, C), severity levels, and as a compact encoding for tree models with high-cardinality categories.

7. When to Use and When Not to Use

Use it when	Avoid it when
Categories have a real order	Categories are unordered (city, colour): use OneHot
Using tree-based models (compact, fine)	Using linear/distance models with unordered data (fake order misleads them)
High-cardinality with trees as a quick option	The *gaps* between levels are clearly unequal and the model is linear

8. Advantages, Disadvantages and Limitations

Advantages	Disadvantages
One column per feature, compact	Imposes equal spacing between levels
Preserves order	Wrong order silently hurts models
Supports `handle_unknown="use_encoded_value"`	Not meant for the target (use LabelEncoder)

9. Python Example (scikit-learn)

PYTHON

```
import numpy as np, pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OrdinalEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split

rng = np.random.RandomState(0); n = 300
df = pd.DataFrame({"education": rng.choice(["school", "graduate", "postgrad"], n), "grade": rng.choice(["C", "B",
"A"], n)})
y = ((df.education == "postgrad") | (df.grade == "A")).astype(int)
X_tr, X_te, y_tr, y_te = train_test_split(df, y, random_state=0)
enc = ColumnTransformer([("ord", OrdinalEncoder(categories=[["school", "graduate", "postgrad"], ["C", "B", "A"]],
                                                handle_unknown="use_encoded_value", unknown_value=-1),
["education", "grade"])])
model = Pipeline([("enc", enc), ("rf", RandomForestClassifier(100, random_state=0))]).fit(X_tr, y_tr)
print("test accuracy:", round(model.score(X_te, y_te), 3))
print("encoded sample:\n", enc.fit(X_tr).transform(X_te.head(3)))
print("unseen category 'phd' ->", enc.transform(pd.DataFrame({"education": ["phd"], "grade": ["A"]}))[0])
```

OUTPUT

```
test accuracy: 1.0
encoded sample:
 [[1. 1.]
```

```
 [1. 2.]
 [1. 0.]]
unseen category 'phd' -> [-1.  2.]
```

10. Common Mistakes and Related Concepts

Common mistakes

- Relying on the default alphabetical order.
- Using it for unordered features with linear models.
- Using `OrdinalEncoder` on the target `y` (use `LabelEncoder`), or fitting on all data before the split.

Related: OneHotEncoder, LabelEncoder, TargetEncoder, `pd.Categorical(ordered=True)`, binning.

11. Quick Revision Notes

Chapter Summary

- For features with natural order; always pass `categories=[[...]]` explicitly.
- Linear models assume equal gaps; trees only use the order.
- Handle unseen values with `handle_unknown="use_encoded_value"`.
- Features → OrdinalEncoder; target → LabelEncoder.

CHAPTER 27

OneHotEncoder

1. Definition (Simple English)

In one line

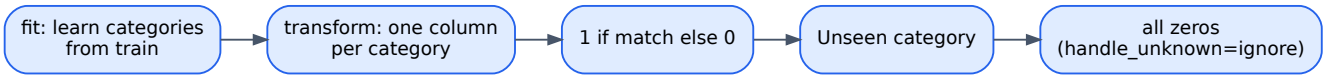
OneHotEncoder turns a categorical column with K distinct values into K separate 0/1 columns, where exactly one column is 1 for each row. It adds no fake order between categories.

2. Hinglish Explanation

Hinglish mein samjho

City = Pune, Delhi, Mumbai. Agar hum Pune=0, Delhi=1, Mumbai=2 bana dein to model sochega Mumbai > Delhi > Pune, jo bekaar hai. One-hot mein har city ka **apna column** hota hai: Pune wali row mein `city_Pune=1` aur baaki 0. Isse koi order nahi banta. Dhyan rakho: agar category bahut zyada hain (1000 pin codes) to columns phat jate hain; tab rare categories ko group karo (`min_frequency`) ya trees/target encoding socho. Linear models mein `drop="first"` se *dummy variable trap* (perfect multicollinearity) bachta hai.

3. How It Works, Step by Step



4. Diagram

city	city_Delhi	city_Mumbai	city_Pune
Pune	0	0	1
Delhi	1	0	0
Mumbai	0	1	0
Pune	0	0	1

Each row has exactly one 1. With `drop="first"` the `city_Delhi` column is removed: Delhi is then the all-zero baseline.

5. Formulas with a Numerical Example

$$x_c = 1[\text{value} = c], \qquad \sum_{c=1}^{K} x_c = 1 \;\; (\text{the "trap": columns sum to a constant})$$

Worked example

A linear model with all $K = 3$ city columns **and** an intercept is not uniquely solvable, because `Delhi + Mumbai + Pune = 1` equals the constant column. Dropping one (`drop="first"`) fixes it, and the dropped city becomes the baseline absorbed in the intercept. A categorical feature with 1000 values produces 1000 columns, so a dataset of 100,000 rows becomes a 100000×1000 matrix (stored sparse by default).

PYTHON

```python
import numpy as np
from sklearn.preprocessing import OneHotEncoder
X = np.array([["Pune"], ["Delhi"], ["Mumbai"], ["Pune"]])
enc = OneHotEncoder(sparse_output=False).fit(X); print(enc.get_feature_names_out());
print(enc.transform(X).astype(int))
print("unseen 'Goa' ->", OneHotEncoder(handle_unknown="ignore", sparse_output=False).fit(X).transform([["Goa"]]))
print("drop first   ->", OneHotEncoder(drop="first", sparse_output=False).fit(X).get_feature_names_out())
```

OUTPUT

```
['x0_Delhi' 'x0_Mumbai' 'x0_Pune']
[[0 0 1]
 [1 0 0]
 [0 1 0]
 [0 0 1]]
unseen 'Goa' -> [[0. 0. 0.]]
drop first   -> ['x0_Mumbai' 'x0_Pune']
```

6. Real-World Applications

Gender, city, product category, payment method, browser, department: any nominal feature for linear models, SVMs, neural networks and KNN.

7. When to Use and When Not to Use

Use it when	Avoid it when
Unordered categories, low to medium cardinality	Hundreds or thousands of categories (consider grouping or target/ordinal encoding)
Linear, SVM, KNN, neural models	Tree models with very high cardinality (one-hot makes splits inefficient)
New categories may appear (use `handle_unknown="ignore"`)	Categories have a natural order (use OrdinalEncoder)

8. Advantages, Disadvantages and Limitations

Advantages	Disadvantages
No false ordering	Many columns for high-cardinality features
Simple, widely supported	Sparse, memory heavy; curse of dimensionality
Safe for unseen categories with `ignore`	Dummy-variable trap in unregularised linear models

9. Python Example (scikit-learn)

PYTHON

```python
import numpy as np, pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.model_selection import train_test_split, cross_val_score

rng = np.random.RandomState(1); n = 600
df = pd.DataFrame({"city": rng.choice(["Pune", "Delhi", "Mumbai", "Goa"], n, p=[.4, .3, .25, .05]), "age":
rng.randint(20, 60, n)})
logit = 0.04 * (df.age - 40) + df.city.map({"Pune": 1.0, "Delhi": -1.0, "Mumbai": 0.0, "Goa": 0.5})
y = (rng.rand(n) < 1 / (1 + np.exp(-logit))).astype(int)
X_tr, X_te, y_tr, y_te = train_test_split(df, y, test_size=0.25, random_state=0, stratify=y)
pre = ColumnTransformer([("cat", OneHotEncoder(handle_unknown="ignore", min_frequency=0.1), ["city"]), ("num",
StandardScaler(), ["age"])])
model = Pipeline([("pre", pre), ("lr", LogisticRegression())]).fit(X_tr, y_tr)
print("columns:", list(model["pre"].get_feature_names_out()))
print("CV accuracy:", cross_val_score(model, X_tr, y_tr, cv=5).mean().round(3), " test:", round(model.score(X_te,
y_te), 3))
print("brand-new city 'Jaipur' still predicts:", model.predict(pd.DataFrame({"city": ["Jaipur"], "age": [30]})))
```

OUTPUT

```
columns: ['cat__city_Delhi', 'cat__city_Mumbai', 'cat__city_Pune', 'cat__city_infrequent_sklearn', 'num__age']
CV accuracy: 0.662  test: 0.74
brand-new city 'Jaipur' still predicts: [0]
```

10. Common Mistakes and Related Concepts

Common mistakes

- Using `pd.get_dummies` separately on train and test (different columns!). Use the encoder in a Pipeline.
- Leaving `handle_unknown="error"` and crashing in production on a new category.
- One-hot encoding an ID column with tens of thousands of values.
- Forgetting the dummy trap in plain linear regression (use `drop="first"` or regularisation).

Related: OrdinalEncoder, TargetEncoder, hashing (`FeatureHasher`), embeddings, `min_frequency`, `max_categories`.

11. Quick Revision Notes

Chapter Summary

- One 0/1 column per category; no fake order.
- Use `handle_unknown="ignore"`, `drop="first"` (linear models), `min_frequency` (rare groups).
- Fit inside a Pipeline; avoid for very high cardinality.
- Nominal → OneHot, ordinal → Ordinal.

CHAPTER 28

LabelEncoder

1. Definition (Simple English)

In one line

LabelEncoder converts the **target** column (`y`) of text labels such as "cat", "dog", "bird" into integers 0, 1, 2 (alphabetical by default), and can convert predictions back to the original names with `inverse_transform` .

2. Hinglish Explanation

Hinglish mein samjho

LabelEncoder sirf **target (y)** ke liye bana hai, features ke liye nahi. Agar aapka target "spam"/"ham" ya "setosa"/"versicolor" jaisa text hai, to isse numbers (0, 1, 2) bana do aur predictions ko `inverse_transform` se wapas naam bana lo. Common galti: LabelEncoder ko X ke columns par lagana. Wahan woh category ko fake order de deta hai (0 < 1 < 2). Features ke liye OneHotEncoder ya OrdinalEncoder use karo. Note: aajkal scikit-learn ke zyada tar classifiers string labels khud sambhal lete hain, to kai baar LabelEncoder zaroori hi nahi hota.

3. How It Works, Step by Step



4. Diagram

Label	Encoded
bird	0
cat	1
dog	2

`classes_ = ['bird', 'cat', 'dog']` ; the code is simply the position in this sorted list.
`inverse_transform([2,0])` gives `['dog','bird']` .

5. Formulas with a Numerical Example

$$\text{code}(\ell) = \text{index of } \ell \text{ in sorted(unique labels)}$$

Worked example

Labels `[cat, dog, cat, bird]` → sorted unique `[bird, cat, dog]` → codes `[1, 2, 1, 0]` . Because classifiers do not treat the target codes as *magnitudes*, the arbitrary order of target classes is harmless. As a **feature**, though, "dog = 2" would be read as twice "cat = 1", a false relationship.

PYTHON

```python
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder(); y = ["cat", "dog", "cat", "bird"]
codes = le.fit_transform(y); print("classes_:", le.classes_, " codes:", codes)
print("inverse:", le.inverse_transform([2, 0, 1]))
try: le.transform(["fish"])
except ValueError as e: print("unseen label error:", str(e)[:60])
```

OUTPUT

```
classes_: ['bird' 'cat' 'dog']  codes: [1 2 1 0]
inverse: ['dog' 'bird' 'cat']
unseen label error: y contains previously unseen labels: np.str_('fish')
```

6. Real-World Applications

Encoding class names for XGBoost/LightGBM-style APIs that require integer labels, mapping predicted class ids back to readable names in reports, preparing targets for neural networks, and building confusion-matrix labels.

7. When to Use and When Not to Use

Use it when	Avoid it when
Encoding the **target** `y`	Encoding input **features** (use OneHot/Ordinal)
A library demands integer class labels	A scikit-learn classifier already accepts strings
You need `inverse_transform` for reporting	Multi-column features (it handles only 1-D data)

8. Advantages, Disadvantages and Limitations

Advantages	Disadvantages
Simple, reversible	1-D only, no `ColumnTransformer` use
Stable alphabetical mapping	Fails on unseen labels at transform time
Works for any hashable labels	Misused on features creates fake order

9. Python Example (scikit-learn)

PYTHON

```python
from sklearn.datasets import load_iris
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
import numpy as np

iris = load_iris(); names = np.array(iris.target_names)[iris.target]       # text labels, as in a CSV
X_tr, X_te, n_tr, n_te = train_test_split(iris.data, names, test_size=0.3, stratify=names, random_state=0)
le = LabelEncoder().fit(n_tr)                                              # learn mapping from TRAIN labels
y_tr, y_te = le.transform(n_tr), le.transform(n_te)
clf = RandomForestClassifier(100, random_state=0).fit(X_tr, y_tr)
pred = clf.predict(X_te)
print("classes_:", le.classes_, " accuracy:", round(accuracy_score(y_te, pred), 3))
print("decoded predictions:", le.inverse_transform(pred[:5]))
# modern alternative: sklearn classifiers accept strings directly
print("string labels directly:", RandomForestClassifier(100, random_state=0).fit(X_tr, n_tr).predict(X_te[:3]))
```

OUTPUT

```
classes_: ['setosa' 'versicolor' 'virginica']  accuracy: 0.978
decoded predictions: ['virginica' 'virginica' 'setosa' 'setosa' 'versicolor']
string labels directly: ['virginica' 'virginica' 'setosa']
```

10. Common Mistakes and Related Concepts

Common mistakes

- Applying LabelEncoder to feature columns (false numeric order).
- Fitting separate encoders on train and test (mappings can differ); fit once on train.
- Forgetting to `inverse_transform` before showing results.
- Assuming the mapping order is meaningful (it is alphabetical).

Related: OrdinalEncoder (features), OneHotEncoder, `LabelBinarizer`, `label_binarize` for ROC curves, `pd.factorize`.

11. Quick Revision Notes

Chapter Summary

- LabelEncoder = for the **target**; 1-D; alphabetical integer codes; reversible.
- Features need OneHot/Ordinal instead.
- Fit on train labels once; keep `classes_` to decode predictions.

CHAPTER 29

StandardScaler

1. Definition (Simple English)

In one line

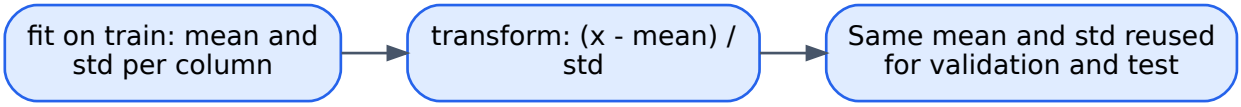
StandardScaler rescales each numeric feature to have **mean 0 and standard deviation 1** using $z = (x - \mu)/\sigma$, so features measured in very different units become comparable.

2. Hinglish Explanation

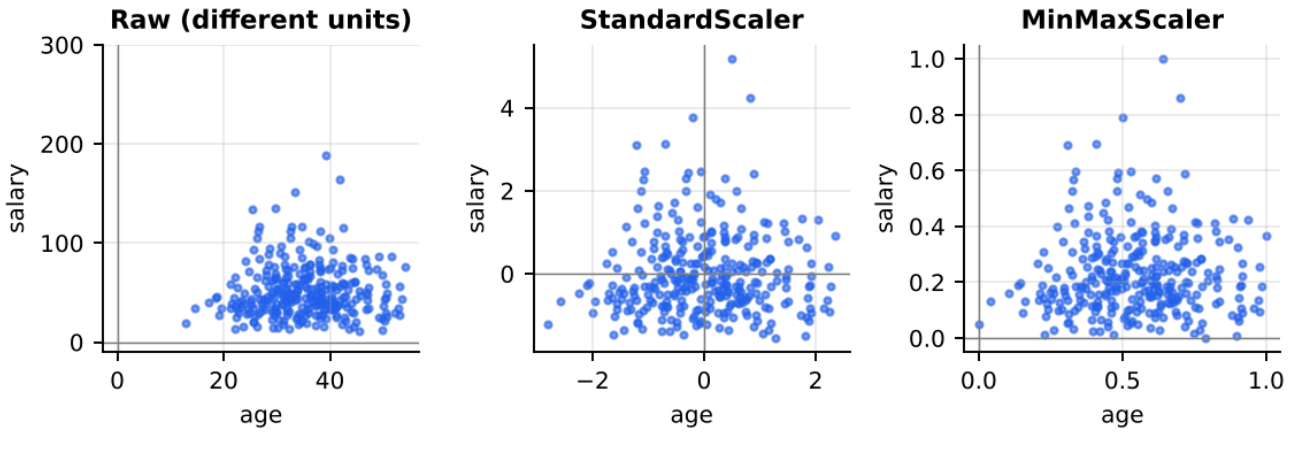
Hinglish mein samjho

Age 20-60 ke beech hai aur salary 20,000-2,00,000 ke beech. Agar KNN ya SVM mein distance naapa jaye to salary ka number itna bada hai ki age ka asar dikhta hi nahi. StandardScaler har feature se uska **average ghata deta hai** aur **standard deviation se bhaag deta hai**, to sab features ek hi scale par aa jate hain. Yaad rakho: μ aur σ sirf **train data** se seekho (`fit`), phir test par sirf `transform` lagao. Trees ko scaling ki zaroorat nahi.

3. How It Works, Step by Step



4. Diagram



Same data, same shape; only the axes change. After scaling both features have comparable ranges.

5. Formulas with a Numerical Example

$$z = \frac{x - \mu}{\sigma}, \qquad \mu = \frac{1}{n}\sum x_i, \qquad \sigma = \sqrt{\frac{1}{n}\sum (x_i - \mu)^2} \ \text{(population std; scikit-learn uses } n\text{)}$$

Worked example

Train ages $[20, 30, 40]$: $\mu = 30$, $\sigma = \sqrt{(100 + 0 + 100)/3} = 8.165$. Scaled train $= [-1.225, 0, 1.225]$. A new person aged 50 becomes $(50 - 30)/8.165 = 2.449$. Min-Max would give $(x - x_{min})/(x_{max} - x_{min}) = (50 - 20)/20 = 1.5$ (outside [0,1], which is allowed for unseen data). **Robust** scaling uses the median and IQR, so outliers hardly change it.

PYTHON

```python
import numpy as np
from sklearn.preprocessing import StandardScaler, MinMaxScaler
tr = np.array([[20.], [30.], [40.]]); sc = StandardScaler().fit(tr)
print("mean:", sc.mean_, " std:", sc.scale_.round(3)); print("train scaled:", sc.transform(tr).ravel().round(3))
print("new 50 ->", sc.transform([[50.]]).round(3).ravel(), " | MinMax ->",
MinMaxScaler().fit(tr).transform([[50.]]).ravel())
```

OUTPUT

```
mean: [30.]  std: [8.165]
train scaled: [-1.225  0.     1.225]
new 50 -> [2.449]  | MinMax -> [1.5]
```

6. Real-World Applications

Preprocessing for KNN, SVM, logistic/linear regression with regularisation, PCA, K-Means, neural networks, and gradient-descent-based models where scale affects convergence.

7. When to Use and When Not to Use

Use it when	Avoid it when
Distance-based or margin-based models (KNN, SVM, K-Means, PCA)	Tree-based models (Decision Tree, Random Forest, GB): they ignore scale
Regularised linear models and neural nets	Data is full of outliers (use `RobustScaler`)
Features have different units	Sparse data where centring destroys sparsity (use `MaxAbsScaler` or `with_mean=False`)

8. Advantages, Disadvantages and Limitations

Advantages	Disadvantages
Simple, fast, reversible (`inverse_transform`)	Sensitive to outliers (mean and std get pulled)
Faster, more stable gradient-based training	Does not make a skewed feature normal
Makes regularisation fair across features	Must be fitted on train only

9. Python Example (scikit-learn)

PYTHON

```python
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.tree import DecisionTreeClassifier

X, y = load_wine(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.25, stratify=y, random_state=0)
print("feature ranges (train):", X_tr.min(0).round(1)[:4], "...", X_tr.max(0).round(1)[:4])
def cv(m): return cross_val_score(m, X_tr, y_tr, cv=5).mean().round(3)
print("KNN  raw   :", cv(KNeighborsClassifier()))
```

```
print("KNN  scaled:", cv(make_pipeline(StandardScaler(), KNeighborsClassifier())))
print("Tree raw   :", cv(DecisionTreeClassifier(random_state=0)), "| Tree scaled:",
cv(make_pipeline(StandardScaler(), DecisionTreeClassifier(random_state=0))))
sc = StandardScaler().fit(X_tr); Z = sc.transform(X_tr)
print("train mean ~0:", abs(Z.mean(0)).max().round(6), " std ~1:", Z.std(0).round(2)[:4], " test mean not exactly
0:", sc.transform(X_te).mean(0).round(2)[:3])
```

OUTPUT

```
feature ranges (train): [11.4  0.7  1.4 10.6] ... [14.8  5.8  3.2 30. ]
KNN  raw   : 0.721
KNN  scaled: 0.94
Tree raw   : 0.896 | Tree scaled: 0.896
train mean ~0: 0.0  std ~1: [1. 1. 1. 1.]  test mean not exactly 0: [-0.11  0.03 -0.14]
```

10. Common Mistakes and Related Concepts

Common mistakes

- `fit_transform` on the full data or on the test set (leakage). Use a Pipeline.
- Scaling the target `y` accidentally, or one-hot columns unnecessarily.
- Scaling for tree models and expecting an accuracy gain.
- Comparing scaled coefficients to unscaled ones.

Related: MinMaxScaler, RobustScaler, MaxAbsScaler, Normalizer (row-wise), PowerTransformer / log transform, `QuantileTransformer`.

11. Quick Revision Notes

Chapter Summary

- $z = (x - \mu)/\sigma$; learn μ, σ from train only.
- Needed for KNN, SVM, K-Means, PCA, linear/NN models; not for trees.
- Outliers hurt it; use RobustScaler then. Put it in a Pipeline.

CHAPTER 30

Missing Value Handling

1. Definition (Simple English)

In one line

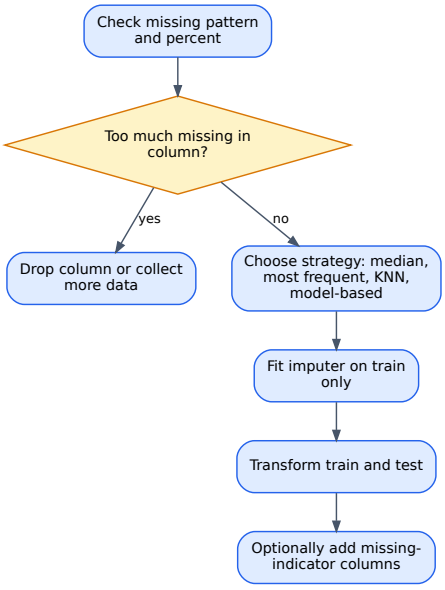
Missing values are empty cells (`NaN`). Handling them means either removing the affected rows/columns or **imputing** (filling) them with a sensible value such as the median, the most frequent category, or a value predicted from neighbouring rows.

2. Hinglish Explanation

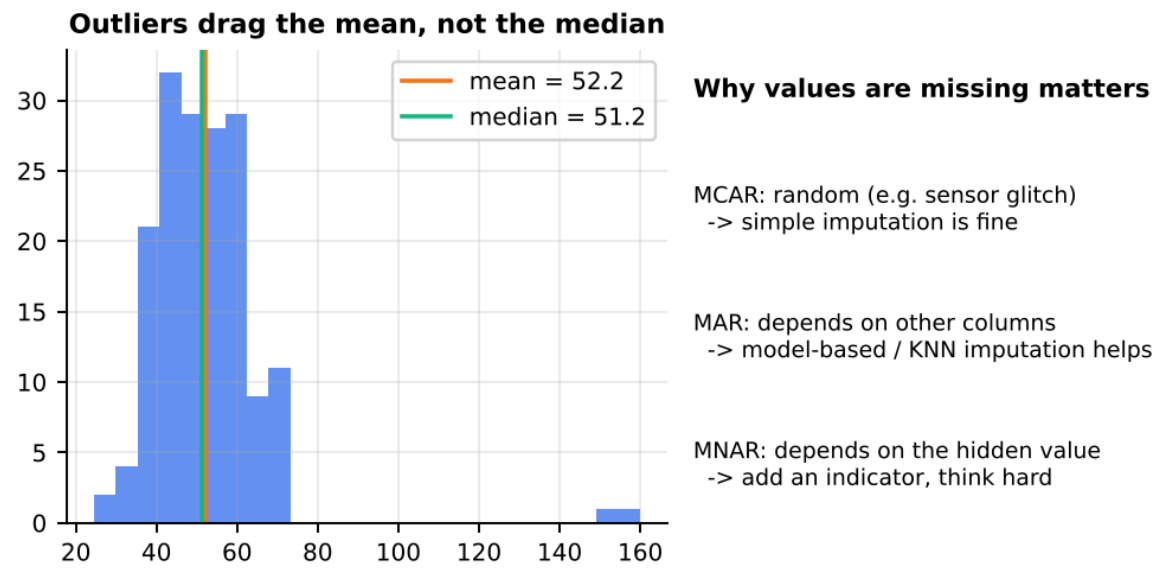
Hinglish mein samjho

Real data mein khaali jagah hoti hi hain (kisi ne age nahi bataya). Zyada tar scikit-learn models NaN par error de dete hain. Options: (1) row/column hata do (agar bahut kam ya bahut zyada missing), (2) **fill** karo: numbers ke liye median (outlier se safe), category ke liye most-frequent, ya KNNImputer jo milte-julte rows se value leta hai. Sabse zaroori: **imputer ko sirf train data se fit karo**, aur "value missing thi" ka khud ek indicator column bana lo (`add_indicator=True`), kyunki missing hona bhi information ho sakta hai.

3. How It Works, Step by Step



4. Diagram



Left: with outliers the mean is pulled away from the typical value. Right: the three missingness mechanisms.

5. Formulas with a Numerical Example

$$\text{mean impute: } x_{\text{miss}} \leftarrow \frac{1}{n_{obs}} \sum x_{obs}, \qquad \text{median impute: } x_{\text{miss}} \leftarrow \text{median}(x_{obs}), \qquad \text{KNN impute: mean of } k \text{ nearest rows'}$$

Worked example

Salaries (thousand ₹) with a gap: $[30,\ 35,\ \text{NaN},\ 40,\ 400]$. Mean of observed $= 126.25$; median $= 37.5$. Filling the gap with the mean gives a salary of 126, absurd for a typical row; the median gives 37.5, sensible. KNN with $k = 2$ using a second feature (age) might pick the two rows closest in age and average their salaries.

PYTHON

```python
import numpy as np
from sklearn.impute import SimpleImputer, KNNImputer
s = np.array([[30], [35], [np.nan], [40], [400.]])
print("mean fill  :", SimpleImputer(strategy="mean").fit_transform(s).ravel())
print("median fill:", SimpleImputer(strategy="median").fit_transform(s).ravel())
X = np.array([[25, 30], [26, 35], [27, np.nan], [60, 400], [61, 390.]])
print("KNN fill (k=2):", KNNImputer(n_neighbors=2).fit_transform(X)[2])
```

OUTPUT

```
mean fill  : [ 30.    35.   126.25  40.   400.  ]
median fill: [ 30.   35.   37.5  40.  400. ]
KNN fill (k=2): [27.  32.5]
```

6. Real-World Applications

Survey data with skipped questions, medical records with untested measurements, sensor logs with dropouts, loan applications with optional fields.

7. When to Use and When Not to Use

Method	Use when	Avoid when
Drop rows	Very few rows affected, missing at random	Many rows affected (lose data) or biased missingness
Mean	Symmetric numeric, no outliers	Skewed data or outliers
Median	Skewed numeric, outliers	Almost never harmful; reduces variance
Most frequent / "missing" category	Categorical features	The mode dominates and hides signal
KNN / Iterative	Features correlate; moderate data size	Huge data (slow), many missing values
Native NaN support	Tree boosting (`HistGradientBoosting`)	Models that need dense numbers

8. Advantages, Disadvantages and Limitations

Imputation keeps rows and lets standard models run, but every method shrinks variance and can bias relationships. Indicators preserve the "was missing" signal. No method recovers information that truly is not there: when more than ~40-50% of a column is missing, think about dropping it or collecting data.

9. Python Example (scikit-learn)

PYTHON

```python
import numpy as np, pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer, KNNImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split, cross_val_score

rng = np.random.RandomState(0); n = 500
df = pd.DataFrame({"age": rng.randint(20, 65, n).astype(float), "income": rng.lognormal(10.5, .5, n), "city":
rng.choice(["A", "B", "C"], n)})
y = ((df.income > 40000) & (df.age > 30)).astype(int).values
for c, p in [("age", .15), ("income", .20), ("city", .10)]: df.loc[rng.rand(n) < p, c] = np.nan
print("missing % per column:\n", (df.isna().mean() * 100).round(1).to_dict())
X_tr, X_te, y_tr, y_te = train_test_split(df, y, test_size=0.25, random_state=0, stratify=y)
num = ["age", "income"]
def make(imp_num):
    pre = ColumnTransformer([("num", Pipeline([("imp", imp_num), ("sc", StandardScaler())]), num),
                             ("cat", Pipeline([("imp", SimpleImputer(strategy="most_frequent")), ("oh",
OneHotEncoder(handle_unknown="ignore"))]), ["city"])])
    return Pipeline([("pre", pre), ("rf", RandomForestClassifier(150, random_state=0))])
for name, imp in [("median", SimpleImputer(strategy="median")), ("median + indicator",
SimpleImputer(strategy="median", add_indicator=True)), ("KNN (k=5)", KNNImputer(n_neighbors=5))]:
    print(f"{name:20s} CV accuracy = {cross_val_score(make(imp), X_tr, y_tr, cv=5).mean():.3f}")
final = make(SimpleImputer(strategy="median", add_indicator=True)).fit(X_tr, y_tr)
print("test accuracy:", round(final.score(X_te, y_te), 3))
```

OUTPUT

```
missing % per column:
 {'age': 14.4, 'income': 19.0, 'city': 9.2}
median               CV accuracy = 0.877
median + indicator   CV accuracy = 0.891
KNN (k=5)            CV accuracy = 0.885
test accuracy: 0.904
```

10. Common Mistakes and Related Concepts

Common mistakes

- Imputing with statistics of the *full* dataset before splitting (leakage); use a Pipeline.
- Mean imputation on skewed/outlier-heavy columns.
- Not checking *why* values are missing, or never using missing indicators.
- Imputing the target variable or dropping rows only from the training set but not at prediction time.
- Treating placeholders like `-999`, `"?"` or `0` as real values.

Related: `IterativeImputer` (MICE), `MissingIndicator`, `HistGradientBoosting` native NaN support, MCAR/MAR/MNAR, `df.isna().mean()`.

11. Quick Revision Notes

Chapter Summary

- Inspect first: how much, which columns, why. Convert placeholders (`?`, `-999`) to `NaN`.
- Numeric: median (robust); categorical: most frequent or "missing"; correlated features: KNN/Iterative.
- Fit imputers on train only inside a Pipeline; consider `add_indicator=True`.
- Drop a column when most of it is missing and uninformative.

PART H

Practical Projects

31. Project 1: House Price Prediction (Regression)

32. Project 2: Loan Approval Prediction (Classification)

33. Project 3: Customer Segmentation (Clustering)

34. Project 4: Comparing Multiple ML Models

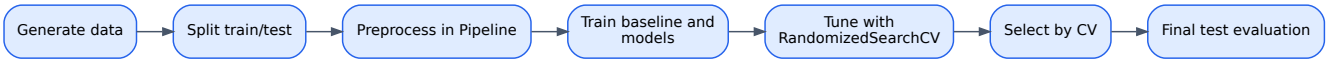
CHAPTER 31

Project 1: House Price Prediction (Regression)

Goal

Predict a house's price (in lakh ₹) from area, bedrooms, age, distance to the city centre and locality. We use a **synthetic dataset generated in code** (so it runs offline) with realistic quirks: skewed prices, missing values and outliers. Replace `make_data()` with `pd.read_csv(...)` to use real data.

1. Problem and Workflow



2. Data and Exploration

PYTHON

```
import numpy as np, pandas as pd

def make_data(n=1500, seed=0):
    rng = np.random.RandomState(seed)
    area = rng.normal(1100, 300, n).clip(350, 3000)
    bed = np.clip((area / 450 + rng.normal(0, .6, n)).round(), 1, 6)
    age = rng.randint(0, 40, n).astype(float)
    dist = rng.gamma(3, 3, n).clip(0.5, 40)
    loc = rng.choice(["Prime", "Mid", "Suburb"], n, p=[.2, .45, .35])
    base = {"Prime": 1.35, "Mid": 1.0, "Suburb": 0.75}
    price = (area * 0.055 + bed * 2.5 - age * 0.35 - dist * 0.9 + 25) * pd.Series(loc).map(base).values +
rng.normal(0, 5, n)
    df = pd.DataFrame(dict(area=area, bedrooms=bed, age=age, dist_center=dist, locality=loc, price=price.clip(10)))
    df.loc[rng.rand(n) < .06, "age"] = np.nan                    # missing values
    df.loc[rng.choice(n, 6, replace=False), "area"] *= 4         # data-entry outliers
    return df

df = make_data()
print(df.shape); print(df.head(3).round(1).to_string(index=False))
print("missing %:", (df.isna().mean() * 100).round(1).to_dict())
print("price skew:", round(df.price.skew(), 2), "| price quantiles:", df.price.quantile([.05, .5,
.95]).round(1).tolist())
print(df.groupby("locality").price.median().round(1).to_dict())
```

OUTPUT

```
(1500, 6)
  area  bedrooms  age  dist_center locality  price
1629.2       4.0 27.0          9.7      Mid  110.0
1220.0       3.0 24.0          2.9      Mid   86.7
1393.6       3.0  9.0          4.4      Mid  103.4
missing %: {'area': 0.0, 'bedrooms': 0.0, 'age': 6.1, 'dist_center': 0.0, 'locality': 0.0, 'price': 0.0}
price skew: 0.62 | price quantiles: [37.9, 71.8, 119.8]
{'Mid': 74.8, 'Prime': 100.8, 'Suburb': 57.3}
```

3. Preprocessing, Models and Evaluation (no leakage)

The split happens first. Imputation, scaling and encoding live inside the Pipeline, so every cross-validation fold learns them from its own training part only.

PYTHON

```
import numpy as np, pandas as pd
from sklearn.model_selection import train_test_split, cross_validate
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
```

```
from sklearn.preprocessing import StandardScaler, OneHotEncoder, RobustScaler
from sklearn.dummy import DummyRegressor
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor

X = df.drop(columns="price"); y = df.price
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, random_state=42)

num, cat = ["area", "bedrooms", "age", "dist_center"], ["locality"]
def pre(scaler):
    return ColumnTransformer([("num", Pipeline([("imp", SimpleImputer(strategy="median")), ("sc", scaler)]), num),
                              ("cat", OneHotEncoder(handle_unknown="ignore"), cat)])
models = {"Mean baseline": Pipeline([("p", pre(StandardScaler())), ("m", DummyRegressor())]),
          "Linear": Pipeline([("p", pre(RobustScaler())), ("m", LinearRegression())]),
          "Ridge": Pipeline([("p", pre(RobustScaler())), ("m", Ridge(alpha=1.0))]),
          "RandomForest": Pipeline([("p", pre(StandardScaler())), ("m", RandomForestRegressor(200,
random_state=0))]),
          "GradBoost": Pipeline([("p", pre(StandardScaler())), ("m", GradientBoostingRegressor(random_state=0))])}
rows = []
for name, m in models.items():
    r = cross_validate(m, X_tr, y_tr, cv=5, scoring=["neg_root_mean_squared_error", "neg_mean_absolute_error",
"r2"])
    rows.append((name, -r["test_neg_root_mean_squared_error"].mean(), -r["test_neg_mean_absolute_error"].mean(),
r["test_r2"].mean()))
res = pd.DataFrame(rows, columns=["model", "CV RMSE", "CV MAE", "CV R2"]).round(3)
print(res.to_string(index=False))
```

OUTPUT

```
        model  CV RMSE  CV MAE  CV R2
Mean baseline   25.298  19.891 -0.004
       Linear    9.608   6.278  0.854
        Ridge    9.606   6.281  0.854
 RandomForest    6.892   5.073  0.925
    GradBoost    5.991   4.655  0.943
```

4. Hyper-parameter Tuning and Final Test

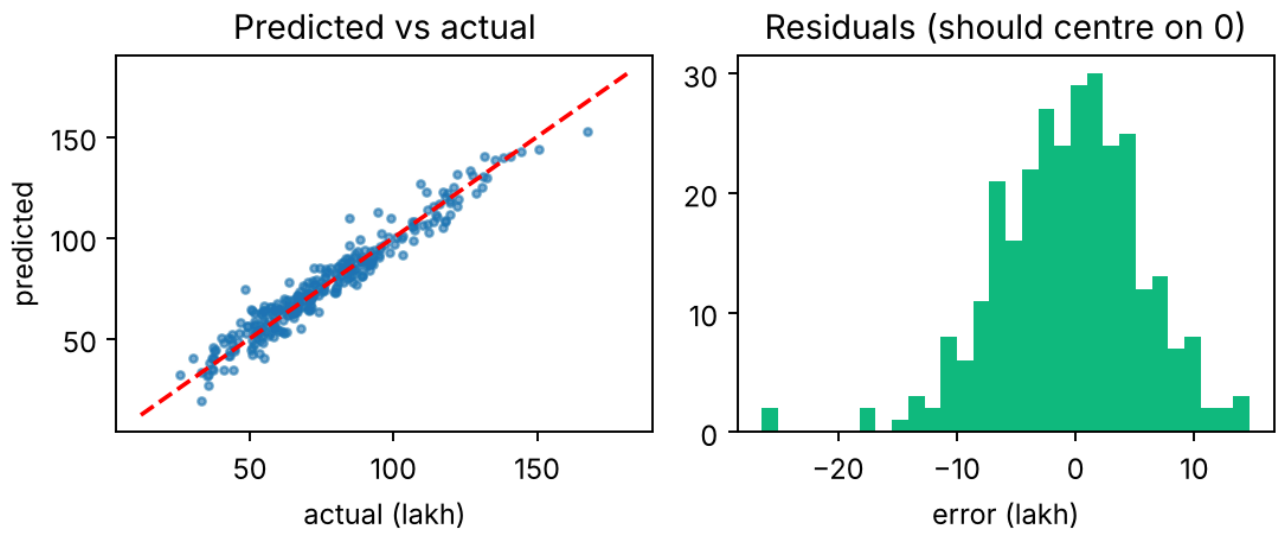
PYTHON

```
import numpy as np, pandas as pd
from scipy.stats import randint, uniform
from sklearn.model_selection import train_test_split, RandomizedSearchCV
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.metrics import mean_absolute_error, root_mean_squared_error, r2_score
import matplotlib; matplotlib.use("Agg"); import matplotlib.pyplot as plt

pre = ColumnTransformer([("num", Pipeline([("imp", SimpleImputer(strategy="median")), ("sc", StandardScaler())]),
["area", "bedrooms", "age", "dist_center"]),
                         ("cat", OneHotEncoder(handle_unknown="ignore"), ["locality"])])
pipe = Pipeline([("p", pre), ("m", GradientBoostingRegressor(random_state=0))])
space = {"m__n_estimators": randint(100, 500), "m__learning_rate": uniform(0.02, 0.2), "m__max_depth": randint(2,
5), "m__subsample": uniform(0.6, 0.4)}
rs = RandomizedSearchCV(pipe, space, n_iter=15, cv=5, scoring="neg_root_mean_squared_error", random_state=0,
n_jobs=1).fit(X_tr, y_tr)
print("best params:", {k.split("__")[1]: (round(float(v), 3) if isinstance(v, float) else int(v)) for k, v in
rs.best_params_.items()}, " CV RMSE:", round(-rs.best_score_, 3))
pred = rs.predict(X_te)                                    # test set used ONCE
print(f"TEST  MAE={mean_absolute_error(y_te, pred):.2f} lakh  RMSE={root_mean_squared_error(y_te, pred):.2f} lakh
R2={r2_score(y_te, pred):.3f}")
fig, ax = plt.subplots(1, 2, figsize=(6.6, 2.9))
ax[0].scatter(y_te, pred, s=7, alpha=.6); ax[0].plot([y.min(), y.max()], [y.min(), y.max()], "r--");
ax[0].set_xlabel("actual (lakh)"); ax[0].set_ylabel("predicted"); ax[0].set_title("Predicted vs actual")
ax[1].hist(y_te - pred, bins=30, color="#10b981"); ax[1].set_title("Residuals (should centre on 0)");
ax[1].set_xlabel("error (lakh)")
fig.tight_layout(); fig.savefig("proj_house.png", dpi=170)
```

OUTPUT

```
best params: {'learning_rate': 0.11, 'max_depth': 2, 'n_estimators': 357, 'subsample': 0.879}  CV RMSE: 5.757
TEST  MAE=4.74 lakh  RMSE=6.13 lakh  R2=0.940
```



Test-set performance of the tuned model: points hug the diagonal and residuals centre on zero.

5. Model Selection and Takeaways

What to conclude

- Compare every model with the **mean baseline**; a model is only useful if it clearly beats it.
- Choose by **cross-validated** RMSE/MAE (on training data), then report the **test** score once.
- Linear models already explain most variance (R^2 about 0.85), but tree ensembles do clearly better (RMSE about 6 vs 9.6 lakh) because locality *multiplies* the price, an interaction a plain linear model cannot represent. On real data always let CV decide.
- Check residuals for patterns, and inspect the largest errors: they often reveal data problems (the injected area outliers are one).
- Next steps: log-transform price, cap area outliers with the IQR rule, add features (price per sqft is a *target leak*, avoid!).

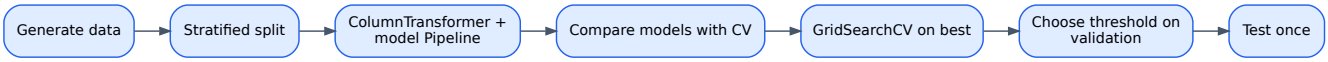
CHAPTER 32

Project 2: Loan Approval Prediction (Classification)

Goal

Predict whether a loan application should be **approved (1) or rejected (0)** using applicant data. The dataset is **synthetic** (generated below) and deliberately imbalanced with missing values and mixed feature types, so we practise the full leakage-safe workflow. Because rejecting a good customer and approving a bad one have different costs, we focus on **precision, recall, F1 and ROC-AUC**, not only accuracy.

1. Workflow



2. Data

PYTHON

```
import numpy as np, pandas as pd

def make_loans(n=3000, seed=1):
    rng = np.random.RandomState(seed)
    income = rng.lognormal(10.9, .45, n)                              # yearly income
    credit = rng.normal(680, 70, n).clip(300, 850)
    amount = income * rng.uniform(.1, 1.2, n)
    emp = rng.choice(["salaried", "self_employed", "unemployed"], n, p=[.65, .27, .08])
    edu = rng.choice(["school", "graduate", "postgrad"], n, p=[.25, .5, .25])
    prev = rng.choice(["none", "paid", "defaulted"], n, p=[.35, .55, .10])
    z = (-3.2 + 0.02 * (credit - 600) + 0.6 * np.log(income / amount) - 1.6 * (prev == "defaulted")
         - 2.0 * (emp == "unemployed") + 0.3 * (edu == "postgrad") + rng.normal(0, .6, n))
    df = pd.DataFrame(dict(income=income, credit_score=credit, loan_amount=amount, employment=emp, education=edu,
prev_loans=prev))
    df["approved"] = (rng.rand(n) < 1 / (1 + np.exp(-z))).astype(int)
    df.loc[rng.rand(n) < .07, "credit_score"] = np.nan
    df.loc[rng.rand(n) < .04, "employment"] = np.nan
    return df

df = make_loans()
print(df.shape, "| approval rate:", round(df.approved.mean(), 3))
print("missing %:", (df.isna().mean() * 100).round(1)[lambda s: s > 0].to_dict())
print(df.groupby("prev_loans").approved.mean().round(2).to_dict())
```

OUTPUT

```
(3000, 7) | approval rate: 0.284
missing %: {'credit_score': 7.3, 'employment': 4.0}
{'defaulted': 0.14, 'none': 0.31, 'paid': 0.3}
```

3. Preprocessing and Model Comparison

PYTHON

```
from sklearn.model_selection import train_test_split, cross_validate, StratifiedKFold
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder, OrdinalEncoder
from sklearn.dummy import DummyClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
```

```
X = df.drop(columns="approved"); y = df.approved
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, stratify=y, random_state=0)
num = ["income", "credit_score", "loan_amount"]
pre = ColumnTransformer([
    ("num", Pipeline([("imp", SimpleImputer(strategy="median", add_indicator=True)), ("sc", StandardScaler())]),
num),
    ("nom", Pipeline([("imp", SimpleImputer(strategy="most_frequent")), ("oh",
OneHotEncoder(handle_unknown="ignore"))]), ["employment", "prev_loans"]),
    ("ord", OrdinalEncoder(categories=[["school", "graduate", "postgrad"]]), ["education"])])
def make(m): return Pipeline([("pre", pre), ("m", m)])
models = {"Baseline (majority)": DummyClassifier(strategy="most_frequent"),
          "LogisticRegression": LogisticRegression(max_iter=1000, class_weight="balanced"),
          "DecisionTree(d=4)": DecisionTreeClassifier(max_depth=4, class_weight="balanced", random_state=0),
          "RandomForest": RandomForestClassifier(300, class_weight="balanced", min_samples_leaf=3, random_state=0),
          "GradientBoosting": GradientBoostingClassifier(random_state=0)}
cv = StratifiedKFold(5, shuffle=True, random_state=0); rows = []
for n_, m in models.items():
    r = cross_validate(make(m), X_tr, y_tr, cv=cv, scoring=["accuracy", "precision", "recall", "f1", "roc_auc"])
    rows.append([n_] + [r[f"test_{k}"].mean() for k in ["accuracy", "precision", "recall", "f1", "roc_auc"]])
print(pd.DataFrame(rows, columns=["model", "acc", "prec", "recall", "f1",
"roc_auc"]).round(3).to_string(index=False))
```

OUTPUT

```
              model   acc  prec  recall    f1  roc_auc
Baseline (majority) 0.716 0.000   0.000 0.000    0.500
 LogisticRegression 0.709 0.492   0.719 0.584    0.792
  DecisionTree(d=4) 0.663 0.451   0.721 0.550    0.744
       RandomForest 0.714 0.498   0.632 0.557    0.760
   GradientBoosting 0.762 0.634   0.387 0.480    0.777
```

4. Tuning, Threshold and Final Evaluation

PYTHON

```
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score, ConfusionMatrixDisplay,
precision_recall_curve
import matplotlib; matplotlib.use("Agg"); import matplotlib.pyplot as plt

grid = {"m__C": [0.01, 0.1, 1, 10], "m__penalty": ["l2"]}
gs = GridSearchCV(make(LogisticRegression(max_iter=1000, class_weight="balanced")), grid, cv=cv,
scoring="roc_auc").fit(X_tr, y_tr)
print("best LogReg:", gs.best_params_, " CV ROC-AUC:", round(gs.best_score_, 3))
gb = make(GradientBoostingClassifier(n_estimators=150, learning_rate=0.05, max_depth=3, subsample=0.8,
random_state=0))
gs2 = GridSearchCV(gb, {"m__max_depth": [2, 3], "m__n_estimators": [100, 200]}, cv=cv, scoring="roc_auc").fit(X_tr,
y_tr)
print("best GB:", gs2.best_params_, " CV ROC-AUC:", round(gs2.best_score_, 3))
best = gs if gs.best_score_ >= gs2.best_score_ else gs2
# choose the decision threshold on TRAINING data via cross-validated probabilities, never on the test set
from sklearn.model_selection import cross_val_predict
oof = cross_val_predict(best.best_estimator_, X_tr, y_tr, cv=cv, method="predict_proba")[:, 1]
p, r, th = precision_recall_curve(y_tr, oof); f1 = 2 * p * r / (p + r + 1e-9); thr = th[f1[:-1].argmax()]
print("chosen threshold (max CV F1):", round(float(thr), 3))
proba = best.predict_proba(X_te)[:, 1]; pred = (proba >= thr).astype(int)
print("TEST ROC-AUC:", round(roc_auc_score(y_te, proba), 3)); print(confusion_matrix(y_te, pred))
print(classification_report(y_te, pred, target_names=["rejected", "approved"], digits=3))
fig, ax = plt.subplots(1, 2, figsize=(6.6, 2.9))
ConfusionMatrixDisplay.from_predictions(y_te, pred, ax=ax[0], cmap="Blues", colorbar=False, display_labels=
["rejected", "approved"]); ax[0].grid(False); ax[0].set_title("Test confusion matrix")
ax[1].plot(th, p[:-1], label="precision"); ax[1].plot(th, r[:-1], label="recall"); ax[1].axvline(thr, color="k",
ls="--"); ax[1].legend(); ax[1].set_title("Threshold trade-off (train CV)"); ax[1].set_xlabel("threshold")
fig.tight_layout(); fig.savefig("proj_loan.png", dpi=170)
```

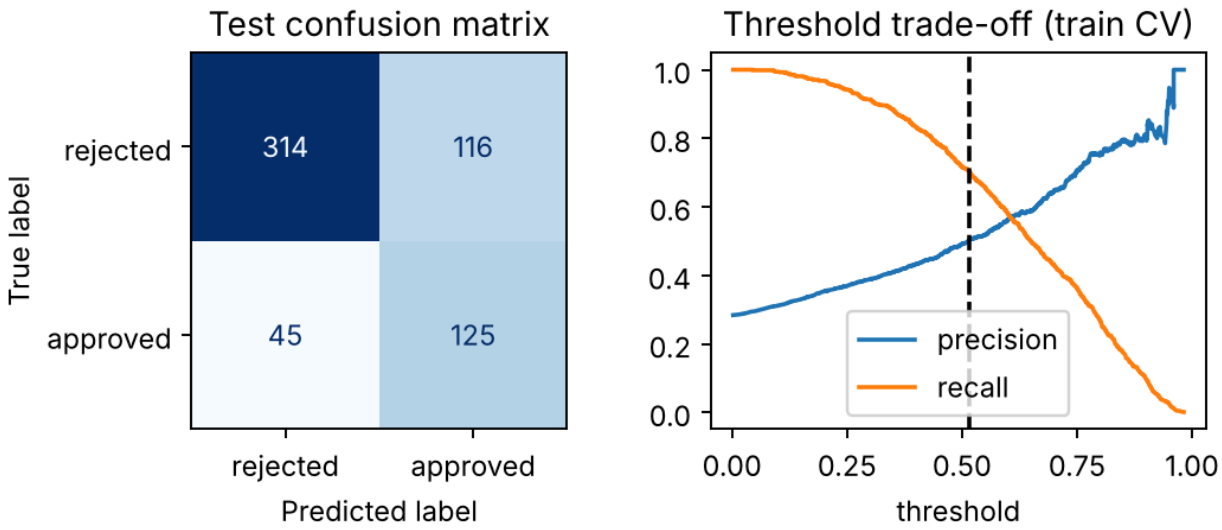
OUTPUT

```
best LogReg: {'m__C': 1, 'm__penalty': 'l2'}  CV ROC-AUC: 0.792
best GB: {'m__max_depth': 2, 'm__n_estimators': 100}  CV ROC-AUC: 0.785
chosen threshold (max CV F1): 0.515
TEST ROC-AUC: 0.818
[[314 116]
 [ 45 125]]
              precision    recall  f1-score   support

    rejected      0.875     0.730     0.796       430
    approved      0.519     0.735     0.608       170

    accuracy                          0.732       600
```

```
   macro avg      0.697     0.733     0.702       600
weighted avg      0.774     0.732     0.743       600
```



Final test confusion matrix and the precision/recall trade-off used to pick the threshold.

5. Model Selection and Takeaways

What to conclude

- **Accuracy alone misleads:** the majority baseline gets 72% accuracy with zero recall, and gradient boosting has the *highest* accuracy yet the *lowest* recall (it rarely approves). Rankings by accuracy, recall and F1 disagree, so choose the metric from the business cost, and read ROC-AUC next to it.
- Models were compared with **stratified 5-fold CV on the training split**; the test set was used once at the end.
- The decision **threshold was chosen from out-of-fold training predictions**, not from the test set, which would be leakage.
- Business view: raising the threshold means fewer bad loans approved (higher precision) but more good customers rejected (lower recall). Pick it from the cost of each error.
- Fairness note: features like employment or education can encode social bias. Audit performance by group before using a model like this on real applicants.

CHAPTER 33

Project 3: Customer Segmentation (Clustering)

Goal

Group customers into meaningful segments from their purchase behaviour (recency, frequency, spend, basket size, online share) so marketing can treat each group differently. There are no labels: we judge clusters by **silhouette score, stability and business interpretability**. The data is synthetic with four hidden customer types.

1. Workflow



2. Data and Preparation

PYTHON

```
import numpy as np, pandas as pd

def make_customers(seed=2):
    rng = np.random.RandomState(seed)
    specs = [  # name, n, recency(days), frequency, spend, basket, online share
        ("Champions", 250, 10, 30, 900, 5, .6), ("Loyal regulars", 400, 25, 14, 350, 3.5, .5),
        ("At-risk", 350, 150, 6, 220, 3, .4), ("New/low value", 500, 60, 2, 60, 1.8, .7)]
    parts = []
    for name, n, rec, fr, sp, bk, on in specs:
        parts.append(pd.DataFrame({"recency_days": rng.gamma(4, rec / 4, n), "frequency": rng.poisson(fr, n) + 1,
                                   "total_spend": rng.lognormal(np.log(sp), .35, n), "avg_basket": rng.gamma(8, bk
/ 8, n),
                                   "online_share": rng.beta(on * 8, (1 - on) * 8, n), "true_group": name}))
    return pd.concat(parts, ignore_index=True).sample(frac=1, random_state=0).reset_index(drop=True)

df = make_customers(); feats = ["recency_days", "frequency", "total_spend", "avg_basket", "online_share"]
print(df.shape); print(df[feats].describe().loc[["mean", "50%", "max"]].round(1).to_string())
print("skew:", df[feats].skew().round(2).to_dict())
```

OUTPUT

```
(1500, 6)
      recency_days  frequency  total_spend  avg_basket  online_share
mean          64.5       11.7        331.6         3.0           0.6
50%           40.5        8.0        234.8         2.7           0.6
max          456.1       47.0       2750.6        13.1           1.0
skew: {'recency_days': 1.95, 'frequency': 1.11, 'total_spend': 1.96, 'avg_basket': 1.4, 'online_share': -0.19}
```

3. Choosing k and Fitting K-Means

PYTHON

```
from sklearn.preprocessing import StandardScaler, FunctionTransformer
from sklearn.pipeline import make_pipeline
from sklearn.cluster import KMeans, DBSCAN
from sklearn.metrics import silhouette_score, adjusted_rand_score
from sklearn.decomposition import PCA
import matplotlib; matplotlib.use("Agg"); import matplotlib.pyplot as plt

prep = make_pipeline(FunctionTransformer(lambda a: np.log1p(a), feature_names_out="one-to-one"), StandardScaler())
# log tames skew, then scale
Z = prep.fit_transform(df[feats])
res = []
for k in range(2, 8):
    km = KMeans(k, n_init=10, random_state=0).fit(Z); res.append((k, km.inertia_, silhouette_score(Z, km.labels_)))
print(pd.DataFrame(res, columns=["k", "inertia", "silhouette"]).round(3).to_string(index=False))
```

```
km = KMeans(4, n_init=10, random_state=0).fit(Z); df["segment"] = km.labels_
print("silhouette k=4:", round(silhouette_score(Z, km.labels_), 3), "| agreement with hidden groups (ARI, unknown
in real life):", round(adjusted_rand_score(df.true_group, km.labels_), 3))
seeds = [adjusted_rand_score(km.labels_, KMeans(4, n_init=10, random_state=s).fit_predict(Z)) for s in range(1, 6)]
print("stability across seeds (ARI):", np.round(seeds, 3))
```

OUTPUT

```
 k  inertia  silhouette
 2 4156.619       0.389
 3 2850.811       0.385
 4 2377.266       0.342
 5 2176.405       0.326
 6 2007.806       0.294
 7 1865.878       0.231
silhouette k=4: 0.342 | agreement with hidden groups (ARI, unknown in real life): 0.846
stability across seeds (ARI): [0.989 1.    1.    1.    1.   ]
```

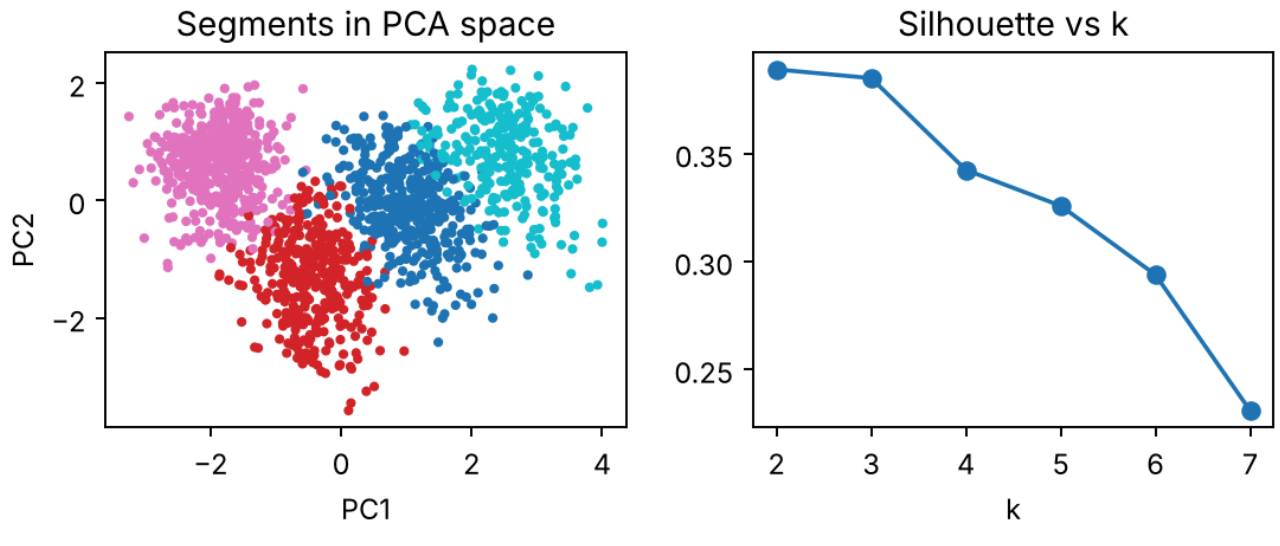
4. Profiling, DBSCAN Comparison and Visualisation

PYTHON

```
prof = df.groupby("segment")[feats].median().round(1); prof["customers"] = df.segment.value_counts().sort_index()
print(prof.to_string())
db = DBSCAN(eps=0.55, min_samples=12).fit(Z); n_cl = len(set(db.labels_) - {-1})
print(f"DBSCAN: {n_cl} clusters, {int((db.labels_ == -1).sum())} noise points -> K-Means is the better fit for
round, similar-density groups")
P = PCA(2).fit_transform(Z)
fig, ax = plt.subplots(1, 2, figsize=(6.6, 2.9))
ax[0].scatter(P[:, 0], P[:, 1], c=df.segment, cmap="tab10", s=6); ax[0].set_title("Segments in PCA space");
ax[0].set_xlabel("PC1"); ax[0].set_ylabel("PC2")
ks = [r[0] for r in res]; ax[1].plot(ks, [r[2] for r in res], marker="o"); ax[1].set_xlabel("k");
ax[1].set_title("Silhouette vs k")
fig.tight_layout(); fig.savefig("proj_seg.png", dpi=170)
```

OUTPUT

```
         recency_days  frequency  total_spend  avg_basket  online_share  customers
segment
0                23.5       15.0        348.1         3.2           0.5        381
1               137.4        7.0        210.5         2.7           0.4        352
2                56.3        3.0         59.7         1.7           0.7        491
3                 9.2       28.0        819.1         4.8           0.6        276
DBSCAN: 5 clusters, 979 noise points -> K-Means is the better fit for round, similar-density groups
```



Left: customer segments projected onto two principal components. Right: silhouette is highest at k = 2 and 3 and lower at 4.

5. Interpretation and Takeaways

From clusters to action

- Read the **median profile table**: a segment with low recency, high frequency and high spend is your *champions* (reward them); high recency with low frequency is *at-risk* (win-back offers); and so on. Name segments by behaviour, not by number.
- We **log-transformed then scaled** before clustering: without this, a few huge spenders dominate the distance.
- Silhouette is highest for $k = 2$ or 3 here because the four customer types overlap, yet four segments are stable across seeds and far more actionable. Choose k by silhouette, stability **and** business use, never by a single number.
- Clustering has no test set. Validate by stability, by a holdout of customers assigned to the nearest centroid, and by business outcomes (A/B test a campaign per segment).
- In production, store the fitted `prep` + `KMeans` so new customers can be assigned with `km.predict(prep.transform(new))`. Re-cluster periodically because behaviour drifts.

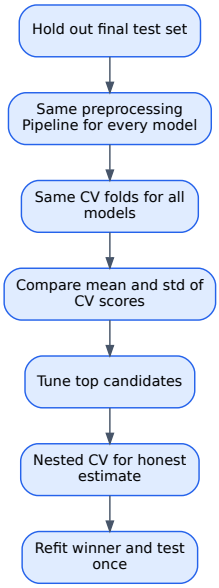
CHAPTER 34

Project 4: Comparing Multiple ML Models

Goal

Compare many algorithms fairly on one dataset (the scikit-learn **Breast Cancer** data), tune the promising ones, and select a final model with a defensible procedure. The method is reusable on any tabular classification problem.

1. A Fair Comparison Protocol



Rules: identical splits for every model, scaling inside the Pipeline (only models that need it get a scaler), more than one metric, and a baseline.

2. Cross-validated Leaderboard

PYTHON

```
import numpy as np, pandas as pd, time
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, cross_validate, RepeatedStratifiedKFold
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.dummy import DummyClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier, GradientBoostingClassifier,
BaggingClassifier, StackingClassifier

X, y = load_breast_cancer(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, stratify=y, random_state=0)
sc = StandardScaler
models = {"Baseline": DummyClassifier(strategy="most_frequent"),
  "LogReg": make_pipeline(sc(), LogisticRegression(max_iter=2000)),
  "KNN(k=7)": make_pipeline(sc(), KNeighborsClassifier(7)),
  "GaussianNB": GaussianNB(),
  "SVM (RBF)": make_pipeline(sc(), SVC(probability=True, random_state=0)),
  "DecisionTree": DecisionTreeClassifier(max_depth=4, random_state=0),
  "Bagging": BaggingClassifier(n_estimators=100, random_state=0),
  "RandomForest": RandomForestClassifier(300, random_state=0),
  "AdaBoost": AdaBoostClassifier(n_estimators=200, random_state=0),
  "GradBoost": GradientBoostingClassifier(random_state=0),
```

```
  "Stacking": StackingClassifier([("lr", make_pipeline(sc(), LogisticRegression(max_iter=2000))), ("rf",
RandomForestClassifier(200, random_state=0)),
                                ("svm", make_pipeline(sc(), SVC(probability=True, random_state=0)))],
final_estimator=LogisticRegression(), cv=5)}
cv = RepeatedStratifiedKFold(n_splits=5, n_repeats=2, random_state=0); rows = []
for n_, m in models.items():
    t0 = time.time(); r = cross_validate(m, X_tr, y_tr, cv=cv, scoring=["accuracy", "f1", "roc_auc"])
    rows.append((n_, r["test_accuracy"].mean(), r["test_accuracy"].std(), r["test_f1"].mean(),
r["test_roc_auc"].mean(), time.time() - t0))
board = pd.DataFrame(rows, columns=["model", "acc", "acc_std", "f1", "roc_auc", "secs"]).sort_values("roc_auc",
ascending=False).round(3)
print(board.to_string(index=False))
```

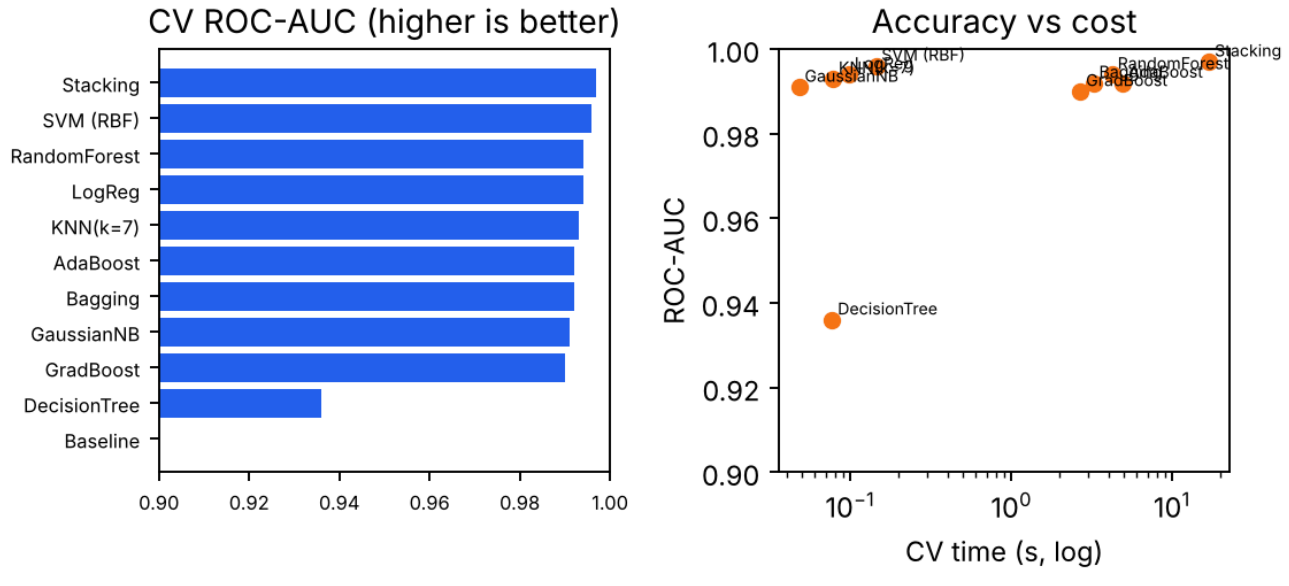
OUTPUT

```
       model   acc  acc_std    f1  roc_auc   secs
    Stacking 0.981    0.012 0.985    0.997 16.775
   SVM (RBF) 0.981    0.012 0.985    0.996  0.205
      LogReg 0.976    0.010 0.981    0.994  0.079
RandomForest 0.958    0.018 0.967    0.994  3.654
    KNN(k=7) 0.973    0.015 0.978    0.993  0.117
    AdaBoost 0.958    0.015 0.967    0.992  4.598
     Bagging 0.953    0.015 0.962    0.992  3.114
  GaussianNB 0.936    0.022 0.950    0.991  0.058
   GradBoost 0.963    0.017 0.970    0.990  2.738
DecisionTree 0.929    0.026 0.943    0.936  0.085
    Baseline 0.626    0.000 0.770    0.500  0.041
```

3. Visual Comparison

PYTHON

```
import matplotlib; matplotlib.use("Agg"); import matplotlib.pyplot as plt
b = board.sort_values("roc_auc"); fig, ax = plt.subplots(1, 2, figsize=(6.8, 3.2))
ax[0].barh(b.model, b.roc_auc, color="#2563eb"); ax[0].set_xlim(0.9, 1.0); ax[0].set_title("CV ROC-AUC (higher is
better)"); ax[0].tick_params(labelsize=7)
ax[1].scatter(b.secs, b.roc_auc, s=30, color="#f97316")
for _, r in b.iterrows(): ax[1].annotate(r.model, (r.secs, r.roc_auc), fontsize=6, xytext=(2, 2),
textcoords="offset points")
ax[1].set_xscale("log"); ax[1].set_xlabel("CV time (s, log)"); ax[1].set_ylabel("ROC-AUC"); ax[1].set_ylim(0.9,
1.0); ax[1].set_title("Accuracy vs cost")
fig.tight_layout(); fig.savefig("proj_compare.png", dpi=170)
```



Left: ranking by cross-validated ROC-AUC. Right: the cost of each model. The top group is separated by tiny margins.

4. Tune the Finalists and Estimate Honestly

When scores differ by less than the fold-to-fold standard deviation, they are statistically indistinguishable: prefer the simpler, faster or more interpretable model. We tune two finalists and use **nested cross-**

validation (tuning inside, evaluation outside) so the reported score is not optimistic.

PYTHON

```python
from sklearn.model_selection import GridSearchCV, cross_val_score, StratifiedKFold
from sklearn.metrics import classification_report, roc_auc_score

inner = StratifiedKFold(5, shuffle=True, random_state=1); outer = StratifiedKFold(5, shuffle=True, random_state=2)
cands = {"LogReg": (make_pipeline(sc(), LogisticRegression(max_iter=2000)), {"logisticregression__C": [0.03, 0.1,
0.3, 1, 3, 10]}),
         "SVM (RBF)": (make_pipeline(sc(), SVC(probability=True, random_state=0)), {"svc__C": [1, 3, 10],
"svc__gamma": ["scale", 0.01, 0.03]}),
         "GradBoost": (GradientBoostingClassifier(random_state=0), {"n_estimators": [100, 200], "learning_rate":
[0.05, 0.1], "max_depth": [2, 3]})}
fitted = {}
for n_, (est, g) in cands.items():
    gs = GridSearchCV(est, g, cv=inner, scoring="roc_auc")
    nested = cross_val_score(gs, X_tr, y_tr, cv=outer, scoring="roc_auc")
    fitted[n_] = gs.fit(X_tr, y_tr)
    print(f"{n_:10s} nested ROC-AUC = {nested.mean():.4f} +/- {nested.std():.4f} | best:
{fitted[n_].best_params_}")
winner = max(cands, key=lambda k: fitted[k].best_score_); w = fitted[winner]
proba = w.predict_proba(X_te)[:, 1]
print("\nSELECTED:", winner, "| final TEST ROC-AUC:", round(roc_auc_score(y_te, proba), 4))
print(classification_report(y_te, w.predict(X_te), target_names=["malignant", "benign"], digits=3))
```

OUTPUT

```
LogReg     nested ROC-AUC = 0.9957 +/- 0.0048 | best: {'logisticregression__C': 0.3}
SVM (RBF)  nested ROC-AUC = 0.9975 +/- 0.0038 | best: {'svc__C': 3, 'svc__gamma': 'scale'}
GradBoost  nested ROC-AUC = 0.9922 +/- 0.0063 | best: {'learning_rate': 0.05, 'max_depth': 3, 'n_estimators': 100}

SELECTED: SVM (RBF) | final TEST ROC-AUC: 0.9944
              precision    recall  f1-score   support

   malignant      0.976     0.952     0.964        42
      benign      0.973     0.986     0.979        72

    accuracy                          0.974       114
   macro avg      0.974     0.969     0.972       114
weighted avg      0.974     0.974     0.974       114
```

5. Model Selection Checklist

How to choose the final model

1. **Performance:** compare CV mean *and* spread on several metrics. Ignore differences smaller than the spread.
2. **Cost of errors:** in screening, prefer high recall for the dangerous class; adjust the threshold, not just the model.
3. **Simplicity and speed:** between equals pick the simpler model (Occam). Linear models are easy to explain and audit.
4. **Robustness:** check stability across seeds/folds and performance on subgroups.
5. **Honest estimate:** nested CV or a final untouched test set, used exactly once.
6. **Deployment:** ship the whole Pipeline with `joblib`, monitor drift, retrain on schedule.

PART I

Interview Preparation

35. Interview Q&A · ML Fundamentals

36. Interview Q&A · Linear and Logistic Regression

37. Interview Q&A · KNN, Decision Tree and Naive Bayes

38. Interview Q&A · Support Vector Machine

39. Interview Q&A · Evaluation Metrics

40. Interview Q&A · Ensemble Learning

41. Interview Q&A · Model Tuning and Cross-Validation

42. Interview Q&A · Unsupervised Learning: K-Means, DBSCAN, PCA

43. Interview Q&A · Data Preprocessing

44. Interview Q&A · Scenario-Based Questions

45. Interview Q&A · Mixed Rapid-Fire Round

CHAPTER 35

Interview Q&A · ML Fundamentals

Warm-up questions every interviewer asks. Answer in two or three crisp sentences first, then add an example. B = beginner, I = intermediate, S = scenario.

Q1 BEGINNER What is machine learning?

Interview answer. Machine learning is a way of building systems that learn patterns from data to make predictions or decisions, instead of following hand-written rules.

Hinglish samjho. Normal programming mein hum rules likhte hain; ML mein hum data dete hain aur computer rules khud seekhta hai. Isse woh kaam ho jate hain jahan rules likhna mushkil hai, jaise spam pehchanna.

Example. Instead of writing "if the mail contains 'lottery' then spam", we show 10,000 labelled mails and let the model learn.

Q2 BEGINNER What is the difference between supervised and unsupervised learning?

Interview answer. Supervised learning uses labelled data (inputs with known answers) to predict a target; unsupervised learning has no labels and finds structure such as clusters or compressed features.

Hinglish samjho. Supervised mein teacher hota hai (answer diya hua), unsupervised mein nahi. Price predict karna supervised hai; customers ko groups mein baantna unsupervised hai.

Example. House price prediction is supervised; customer segmentation with K-Means is unsupervised.

Q3 BEGINNER What is the difference between classification and regression?

Interview answer. Classification predicts a category (spam or not), regression predicts a continuous number (price).

Hinglish samjho. Jawab agar "kaunsi class" hai to classification, agar "kitna" hai to regression. Dono supervised learning ke part hain.

Q4 BEGINNER What are features and labels?

Interview answer. Features are the input variables used to make a prediction; the label (target) is the value we want to predict.

Hinglish samjho. Features matlab jo hum model ko dikhate hain (area, rooms), label matlab jo model ko predict karna hai (price).

Q5 BEGINNER What is training data, validation data and test data?

Interview answer. Training data is used to fit the model, validation data to tune choices such as hyper-parameters, and test data for one final unbiased evaluation.

Hinglish samjho. Train = padhai, validation = practice test, test = final exam. Final exam ka paper padhai ke time dekhna allowed nahi hai, warna score jhootha hoga.

Q6 BEGINNER Why do we split data into train and test sets?

Interview answer. To measure how the model performs on data it has never seen, which estimates real-world performance and exposes overfitting.

Hinglish samjho. Agar usi data par test karenge jis par padha hai, to model ratta maar ke bhi 100% la sakta hai. Alag test set se asli performance pata chalti hai.

Q7 BEGINNER What is overfitting?

Interview answer. Overfitting is when a model learns noise and details of the training data so well that it performs poorly on new data.

Hinglish samjho. Model ne concept samajhne ki jagah answers ratt liye. Train score bahut high, test score low. Ye high variance ki nishani hai.

Example. A depth-unlimited decision tree scoring 100% on train but 85% on test.

Q8 BEGINNER What is underfitting?

Interview answer. Underfitting is when a model is too simple to capture the pattern, so it performs poorly on both training and test data.

Hinglish samjho. Model ne kuch seekha hi nahi; train aur test dono score low. Ye high bias hai. Jaise curved data par seedhi line fit karna.

Q9 BEGINNER How do you detect and fix overfitting?

Interview answer. Detect it by a large gap between training and validation scores. Fix it with more data, simpler models, regularisation, pruning, early stopping, dropout or feature selection.

Hinglish samjho. Train aur validation score ka gap dekho. Gap bada to model ko simple karo, regularisation lagao, data badhao ya cross-validation se tuning karo.

Q10 BEGINNER How do you fix underfitting?

Interview answer. Use a more flexible model, add informative features or polynomial terms, reduce regularisation and train longer.

Hinglish samjho. Model ko zyada taakat do: naye features, zyada complex model, kam regularisation.

Q11 INTERMEDIATE Explain the bias-variance trade-off.

Interview answer. Bias is error from overly simple assumptions; variance is error from sensitivity to the particular training sample. Making a model more flexible lowers bias but raises variance, so we look for the sweet spot with the lowest total error.

Hinglish samjho. Simple model = high bias, low variance (hamesha thoda galat par stable). Complex model = low bias, high variance (training data ke noise ko bhi pakad leta hai). Dono ke beech ka balance dhoondhna hi tuning hai.

Example. Expected error $= \text{Bias}^2 + \text{Variance} + \text{noise}$.

Q12 INTERMEDIATE What is irreducible error?

Interview answer. It is the noise inherent in the data (measurement error, unobserved factors) that no model can remove.

Hinglish samjho. Kuch randomness data mein hi hoti hai, jaise ghar ki price mein seller ka mood. Isse koi model kam nahi kar sakta.

Q13 BEGINNER What is data leakage?

Interview answer. Data leakage is when information that would not be available at prediction time (test data or future information) influences training, giving unrealistically good scores.

Hinglish samjho. Exam ka paper pehle se dekh lena. Offline score mast, production mein flop. Preprocessing ko split ke baad aur Pipeline ke andar karo.

Example. Scaling the whole dataset before splitting lets test-set statistics leak into training.

Q14 INTERMEDIATE Give three common sources of data leakage.

Interview answer. (1) Fitting scalers or imputers on all data before splitting, (2) using a feature derived from the target or from the future, (3) duplicate rows or the same entity appearing in both train and test.

Hinglish samjho. Teen jagah dhyan do: preprocessing split se pehle, target se bani hui feature (jaise "refund_issued" churn predict karne ke liye), aur ek hi customer ke rows train aur test dono mein.

Q15 BEGINNER What is a hyper-parameter and how is it different from a parameter?

Interview answer. Parameters are learned from data (weights, split points); hyper-parameters are set by us before training (k in KNN, max_depth, C).

Hinglish samjho. Parameter model khud seekhta hai, hyper-parameter hum dete hain aur cross-validation se chunte hain.

Q16 BEGINNER What is a baseline model and why use one?

Interview answer. A baseline is a trivial model (predict the mean or the most frequent class) that gives a minimum score to beat, showing whether your model adds value.

Hinglish samjho. Pehle sabse simple model chalao. Agar tumhara fancy model usse behtar nahi, to wo bekaar hai. `DummyClassifier` isi ke liye hai.

Q17 INTERMEDIATE What is the curse of dimensionality?

Interview answer. As the number of features grows, data becomes sparse and distances become less meaningful, so models need exponentially more data and may overfit.

Hinglish samjho. Zyada features matlab space bahut bada, points door door. KNN jaise distance wale models toot jate hain. Feature selection ya PCA se dimensions kam karo.

Q18 BEGINNER What is feature engineering?

Interview answer. Creating, transforming or selecting input variables so that a model can learn better, such as ratios, date parts, log transforms or encodings.

Hinglish samjho. Raw data ko model ke liye behtar banana: "date" se "weekday" nikalna, "price/area" ka ratio banana. Aksar model badalne se zyada fayda yahi deta hai.

Q19 INTERMEDIATE What is the difference between parametric and non-parametric models?

Interview answer. Parametric models assume a fixed functional form with a fixed number of parameters (linear regression); non-parametric models let complexity grow with the data (KNN, decision trees).

Hinglish samjho. Parametric mein pehle se shape tay (line), non-parametric mein data decide karta hai. Non-parametric flexible hote hain par zyada data maangte hain.

Q20 INTERMEDIATE What is the difference between generative and discriminative models?

Interview answer. Discriminative models learn the boundary or P(y|x) directly (logistic regression, SVM); generative models learn how the data is generated, P(x|y) and P(y) (Naive Bayes).

Hinglish samjho. Discriminative sirf "class A aur B ko alag kaise karein" seekhta hai; generative har class ka poora distribution seekhta hai aur Bayes rule se answer nikalta hai.

Q21 BEGINNER What is cross-validation and why use it?

Interview answer. Cross-validation repeatedly trains and validates on different splits of the training data and averages the scores, giving a more reliable estimate than a single split.

Hinglish samjho. Ek hi split par score luck par depend karta hai. K-fold mein har hissa ek baar exam deta hai, to average zyada bharosemand hota hai.

Q22 INTERMEDIATE What does "i.i.d." mean and why does it matter?

Interview answer. Independent and identically distributed: samples are drawn independently from the same distribution. Most ML guarantees and random train/test splits assume it; time series and grouped data violate it.

Hinglish samjho. Matlab har sample alag aur ek hi source se. Time series mein aaj ka data kal se jura hota hai, isliye random split galat hai; time-based split use karo.

Q23 SCENARIO Your model has 99% training accuracy and 70% test accuracy. What do you do?

Interview answer. It is overfitting. I would check for leakage in the test pipeline, then add regularisation or simplify the model, get more data, and use cross-validation to tune.

Hinglish samjho. Bada gap = overfit. Pehle dekho ki split theek hai ya nahi, phir model simple karo (max_depth, regularisation), data badhao, features kam karo.

Q24 INTERMEDIATE Why is accuracy a poor metric for imbalanced data?

Interview answer. A model that always predicts the majority class gets high accuracy while missing every minority case. Precision, recall, F1 and PR-AUC reflect minority performance.

Hinglish samjho. 99% healthy log hain to sabko healthy bol do, 99% accuracy! Par disease wala ek bhi nahi pakda. Isliye recall/precision dekho.

Q25 INTERMEDIATE What is regularisation and why does it work?

Interview answer. Regularisation adds a penalty on model complexity (such as large weights) to the loss, trading a little bias for a larger drop in variance and reducing overfitting.

Hinglish samjho. Model ko "bahut bade weights mat lena" ki saza dete hain, isse woh simple rehta hai aur noise ko ratta nahi marta.

Q26 INTERMEDIATE What is the No Free Lunch theorem in practice?

Interview answer. No single algorithm is best on every problem, so you must compare several models with the same validation protocol on your own data.

Hinglish samjho. Har problem ke liye ek best model nahi hota. Isliye baseline se shuru karo aur kai models cross-validation se compare karo.

CHAPTER 36

Interview Q&A · Linear and Logistic Regression

Linear and logistic regression are the most common first questions: expect equations, assumptions and a small coding or interpretation task.

Q27 BEGINNER What is linear regression?

Interview answer. Linear regression models a numeric target as a weighted sum of features plus an intercept, fitted by minimising the sum of squared errors.

Hinglish samjho. Data ke beech se ek seedhi line (ya plane) nikalna jo errors ke squares ka total sabse chhota kare. Prediction = intercept + weights x features.

Q28 BEGINNER What is the equation of simple linear regression?

Interview answer. $\hat{y} = \beta_0 + \beta_1 x$, where β_1 is the slope and β_0 the intercept.

Hinglish samjho. Slope batata hai x ek unit badhne par y kitna badhega; intercept woh y hai jab x zero ho.

Example. With $\beta_0 = 0.6, \beta_1 = 2.2$ the prediction for $x = 6$ is 13.8.

Q29 BEGINNER What does OLS minimise?

Interview answer. Ordinary Least Squares minimises the sum of squared residuals, $\sum (y_i - \hat{y}_i)^2$.

Hinglish samjho. Har point ki galti ka square karke jod do; jo line ye total sabse chhota kare wahi best line hai.

Q30 INTERMEDIATE Why squared errors and not absolute errors?

Interview answer. Squared error is smooth and differentiable, has a closed-form solution, corresponds to maximum likelihood under Gaussian noise, and penalises large errors more. Absolute error is more robust to outliers.

Hinglish samjho. Square se math aasan hota hai (derivative nikalna) aur ek seedha formula milta hai. Nuksan ye hai ki outlier ka asar zyada hota hai.

Q31 INTERMEDIATE What are the assumptions of linear regression?

Interview answer. Linearity, independence of errors, constant error variance (homoscedasticity), approximately normal errors (for inference) and no severe multicollinearity.

Hinglish samjho. LINE yaad rakho: Linear relation, Independent errors, Normal errors, Equal variance. Features aapas mein perfectly correlated na hon.

Q32 INTERMEDIATE What is multicollinearity and how do you detect it?

Interview answer. It is strong correlation among features, making coefficients unstable. Detect with the correlation matrix or the Variance Inflation Factor (VIF > 5 to 10 is a warning).

Hinglish samjho. Jab do features lagbhag same cheez naapte hon (length aur area), model nahi jaan pata credit kise de. VIF dekho, aur ek feature hatao ya Ridge use karo.

Example. $\text{VIF}_i = 1/(1 - R_i^2)$.

Q33 INTERMEDIATE What is the normal equation?

Interview answer. The closed-form OLS solution $\hat{\boldsymbol{\beta}} = (X^\top X)^{-1} X^\top y$.

Hinglish samjho. Bina loop ke ek formula se best weights mil jate hain. Bahut zyada features par matrix inverse costly ho jata hai, tab gradient descent use hota hai.

Q34 INTERMEDIATE How does gradient descent work for linear regression?

Interview answer. It repeatedly updates the weights in the direction of the negative gradient of the loss: $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla L$, until the loss stops decreasing.

Hinglish samjho. Pahadi se utarne jaisa: har step mein dekho kis taraf dhalan sabse tez hai aur chhota kadam utho. Learning rate η kadam ka size hai.

Q35 INTERMEDIATE What happens if the learning rate is too large or too small?

Interview answer. Too large: the loss oscillates or diverges. Too small: convergence is very slow.

Hinglish samjho. Bada kadam = ghaati ke paar kood jana; chhota kadam = pahunchne mein saalon lagna. Beech ka value chuno ya scheduler lagao.

Q36 BEGINNER What is R^2 and what does it mean?

Interview answer. $R^2 = 1 - \text{SSE}/\text{SST}$ is the fraction of target variance explained by the model compared with predicting the mean.

Hinglish samjho. 1 matlab perfect, 0 matlab sirf average bolne jitna, negative matlab average se bhi bura (test data par ho sakta hai).

Q37 INTERMEDIATE Can R^2 be negative?

Interview answer. Yes, on test data, if the model predicts worse than a constant mean prediction.

Hinglish samjho. Haan. Test set par agar model average se bhi zyada galat hai to $R^2 < 0$ aata hai.

Q38 INTERMEDIATE What is adjusted R^2?

Interview answer. R^2 penalised for the number of features, $1 - (1 - R^2)\frac{n-1}{n-p-1}$, so adding useless features does not increase it.

Hinglish samjho. Normal R^2 naya feature jodne par hamesha badhta hai, chahe feature bekaar ho. Adjusted R^2 bekaar features ko saza deta hai.

Q39 INTERMEDIATE What is the difference between Ridge and Lasso?

Interview answer. Ridge adds an L2 penalty $\lambda \sum w^2$ that shrinks weights; Lasso adds an L1 penalty $\lambda \sum |w|$ that can set weights exactly to zero and so selects features.

Hinglish samjho. Ridge sab weights ko chhota karta hai par zero nahi; Lasso kuch weights ko poora zero kar deta hai (feature selection). ElasticNet dono ka mix hai.

Q40 INTERMEDIATE When would you use Ridge over Lasso?

Interview answer. When many features each contribute a little or are correlated; Lasso arbitrarily picks one of a correlated group.

Hinglish samjho. Correlated features ho aur sab thode useful hon to Ridge. Agar lagta hai ki sirf kuch features asli mein kaam ke hain to Lasso.

Q41 INTERMEDIATE Why must features be scaled before Ridge or Lasso?

Interview answer. The penalty depends on weight size, which depends on feature scale; unscaled features are penalised unevenly.

Hinglish samjho. Chhoti scale wale feature ka weight bada hota hai aur zyada punish hota hai. Scaling se sabke saath barabar vyavhar hota hai.

Q42 INTERMEDIATE How do outliers affect linear regression?

Interview answer. Squared loss gives outliers large influence, tilting the line. Mitigate with removal or capping, robust losses (Huber) or transformations.

Hinglish samjho. Ek bhi door ka point line ko apni taraf kheench leta hai. Outlier check karo, `HuberRegressor` ya log transform try karo.

Q43 INTERMEDIATE How do you handle non-linear relationships with linear regression?

Interview answer. Add transformed or polynomial features (`PolynomialFeatures`), interaction terms or splines. The model stays linear in the parameters.

Hinglish samjho. x ke saath x^2 jod do; model abhi bhi "linear in weights" hai par curve fit kar leta hai.

Q44 BEGINNER What is logistic regression and is it regression or classification?

Interview answer. It is a classification algorithm that models the probability of a class using the sigmoid of a linear score.

Hinglish samjho. Naam mein regression hai par kaam classification ka hai. Linear score ko sigmoid se 0-1 probability bana deta hai.

Q45 BEGINNER What is the sigmoid function?

Interview answer. $\sigma(z) = 1/(1 + e^{-z})$, an S-shaped curve that maps any real number to the range (0, 1).

Hinglish samjho. Kisi bhi number ko 0 aur 1 ke beech dabata hai, isliye probability ke liye perfect hai. $\sigma(0) = 0.5$.

Q46 INTERMEDIATE What loss does logistic regression minimise?

Interview answer. Log loss (binary cross-entropy): $-\frac{1}{n}\sum[y \ln p + (1-y)\ln(1-p)]$.

Hinglish samjho. Galat aur confident predictions ko bahut bada penalty. Sahi aur confident ko almost zero.

Q47 INTERMEDIATE Why not use mean squared error for logistic regression?

Interview answer. With a sigmoid, MSE gives a non-convex loss with flat regions, while log loss is convex and gives clean gradients.

Hinglish samjho. MSE + sigmoid se loss surface ghumavdaar ban jati hai aur gradient descent atak sakta hai. Log loss convex hai.

Q48 INTERMEDIATE How do you interpret a logistic regression coefficient?

Interview answer. A coefficient w is the change in log-odds per unit increase of the feature; e^{w} is the odds ratio.

Hinglish samjho. $w = 0.8$ matlab feature ek unit badhne par odds $e^{0.8} \approx 2.2$ guna ho jate hain.

Q49 INTERMEDIATE What is the decision boundary of logistic regression?

Interview answer. The set of points where $\mathbf{w}^{\top}\mathbf{x} + b = 0$ (probability 0.5), a straight line or hyperplane.

Hinglish samjho. Jahan model ko 50-50 lagta hai wahi seema hai, aur wo seedhi hoti hai. Curved boundary ke liye features badlo ya doosra model lo.

Q50 INTERMEDIATE How do you change the classification threshold and why?

Interview answer. Predict class 1 when $p \geq t$ with a chosen t instead of 0.5, trading precision against recall according to business costs.

Hinglish samjho. Cancer screening mein threshold kam rakho taaki koi marij na chhoote; spam filter mein badhao taaki achhi mail spam na ho.

Q51 INTERMEDIATE How does logistic regression handle multiple classes?

Interview answer. Through one-vs-rest (one binary model per class) or multinomial (softmax) regression, which scikit-learn uses by default with the `lbfgs` solver.

Hinglish samjho. Ya to har class ke liye alag binary model, ya softmax jo sab classes ki probability ek saath deta hai.

Q52 INTERMEDIATE What does `C` mean in scikit-learn's LogisticRegression?

Interview answer. `C` is the inverse of regularisation strength: smaller `C` means stronger regularisation.

Hinglish samjho. Yahan C chhota = zyada penalty = simple model. Ulta lagta hai isliye log galti karte hain.

Q53 INTERMEDIATE **Can logistic regression suffer from perfect separation?**

Interview answer. Yes: if classes are perfectly separable, unregularised weights grow without bound. Regularisation prevents this.

Hinglish samjho. Agar ek line se classes bilkul alag ho jayein to weights infinity ki taraf bhagte hain. L2 penalty (default) isse rokti hai.

CHAPTER 37

Interview Q&A · KNN, Decision Tree and Naive Bayes

Distance-based, rule-based and probabilistic classifiers. Interviewers like to ask "how does it work", "how do you choose its main knob" and "when does it fail".

Q54 BEGINNER How does KNN work?

Interview answer. For a new point, KNN finds the k nearest training points by distance and predicts the majority class (or the average value for regression).

Hinglish samjho. Naye point ke k sabse paas wale padosiyon se poocho aur majority ka jawab le lo. Training mein kuch seekhta nahi, sirf data yaad rakhta hai.

Q55 BEGINNER Why is KNN called a lazy learner?

Interview answer. It does no work at training time except storing the data; all computation happens at prediction time.

Hinglish samjho. Padhai ke time aalas, exam ke time saara kaam. Isliye prediction slow hota hai.

Q56 INTERMEDIATE How do you choose k in KNN?

Interview answer. Treat k as a hyper-parameter and choose it by cross-validation; use an odd k for binary problems to avoid ties.

Hinglish samjho. Chhota k = noisy, overfit; bada k = over-smooth, underfit. Cross-validation se best k chuno.

Q57 BEGINNER Why does KNN need feature scaling?

Interview answer. It uses distances, so a feature with a large range dominates. Standardise features first.

Hinglish samjho. Salary (lakhs) aur age (20-60) ko seedha milaoge to salary hi distance decide karegi. StandardScaler lagao.

Q58 INTERMEDIATE What distance metrics can KNN use?

Interview answer. Euclidean, Manhattan, Minkowski, cosine and others; the best one depends on the data (cosine for text, Manhattan for high dimensions).

Hinglish samjho. Default Euclidean hai. Text jaise sparse data mein cosine, aur outliers ya high dimensions mein Manhattan achha ho sakta hai.

Q59 INTERMEDIATE What is distance-weighted KNN?

Interview answer. Neighbours vote with weight $1/d$, so closer points count more (`weights="distance"`).

Hinglish samjho. Paas wala padosi ki raay zyada bhaari, door wale ki kam. Tie ki problem bhi kam hoti hai.

Q60 INTERMEDIATE Why does KNN struggle in high dimensions?

Interview answer. In many dimensions all points become almost equally far apart, so "nearest" loses meaning; also prediction is slow.

Hinglish samjho. Curse of dimensionality: space itna bada ki sab door lagte hain. PCA ya feature selection se dimensions kam karo.

Q61 INTERMEDIATE What are the time and memory costs of KNN?

Interview answer. Training is $O(1)$ (store data); prediction is $O(nd)$ per query by brute force, less with KD-tree or Ball-tree. Memory holds the whole training set.

Hinglish samjho. Har prediction par saare training points se distance nikalna padta hai, isliye badi dataset par slow. KD-tree isse thoda tez karta hai.

Q62 BEGINNER What is a decision tree?

Interview answer. A decision tree is a flowchart-like model that repeatedly splits data on feature thresholds so that each leaf is as pure as possible, and predicts the majority class or mean of the leaf.

Hinglish samjho. Haan-na sawaalon ka tree: "age > 30?" "income > 5L?". Aakhir mein leaf par answer milta hai.

Q63 BEGINNER What is Gini impurity?

Interview answer. $\text{Gini} = 1 - \sum_k p_k^2$: the probability that two randomly drawn samples from a node have different classes. It is 0 for a pure node.

Hinglish samjho. Node kitna "mixed" hai uska number. Pure node = 0; 50-50 binary node = 0.5.

Example. A node with 6 yes and 4 no has Gini $= 1 - (0.36 + 0.16) = 0.48$.

Q64 BEGINNER What is entropy and information gain?

Interview answer. Entropy $= -\sum p_k \log_2 p_k$ measures disorder; information gain is the entropy of the parent minus the weighted entropy of the children.

Hinglish samjho. Gain jitna bada, split utna accha. Tree wahi split chunta hai jo disorder sabse zyada ghataye.

Q65 INTERMEDIATE Gini or entropy: which should you use?

Interview answer. They usually give very similar trees. Gini is slightly cheaper (no logarithm); entropy can favour more balanced splits. Treat it as a hyper-parameter.

Hinglish samjho. Practice mein farq bahut kam hota hai. Default Gini chalao, chaho to cross-validation se compare kar lo.

Q66 INTERMEDIATE How does a regression tree choose splits?

Interview answer. It picks the split minimising the weighted variance (MSE) of the targets in the child nodes; a leaf predicts the mean target of its samples.

Hinglish samjho. Classification mein Gini, regression mein MSE. Leaf ka answer us leaf ke saare samples ka average hota hai.

Q67 INTERMEDIATE Why do decision trees overfit and how do you prevent it?

Interview answer. Unrestricted trees grow until every leaf is pure, memorising noise. Limit `max_depth`, set `min_samples_leaf` or `min_samples_split`, or prune with `ccp_alpha`.

Hinglish samjho. Tree ko khula chhod doge to har sample ke liye alag leaf ban jayega. Depth ya leaf size limit karo ya pruning lagao.

Q68 INTERMEDIATE What is pruning?

Interview answer. Removing branches that add little predictive value: pre-pruning stops growth early; post-pruning (cost-complexity, `ccp_alpha`) grows fully then cuts back.

Hinglish samjho. Bagiche ki tarah chhantai: zaroorat se zyada daaliyan kaat do taaki tree simple aur general bane.

Q69 INTERMEDIATE Do decision trees need feature scaling?

Interview answer. No. Splits depend on feature order, not scale.

Hinglish samjho. Tree sirf "x kisi threshold se bada hai ya nahi" dekhta hai, isliye scaling zaroori nahi.

Q70 INTERMEDIATE Can decision trees handle missing values and categorical features?

Interview answer. scikit-learn's `DecisionTreeClassifier` supports missing values natively in recent versions and expects numeric features, so categories should be encoded; some other libraries handle categories directly.

Hinglish samjho. Categories ko encode karna padta hai (one-hot ya ordinal). Missing values ke liye ya to imputer lagao ya naye scikit-learn version ka native support use karo.

Q71 INTERMEDIATE What are the weaknesses of a single decision tree?

Interview answer. High variance (small data changes give different trees), overfitting, step-like boundaries, poor extrapolation and bias toward features with many split points.

Hinglish samjho. Thoda data badla to tree poora badal jata hai. Isiliye hum ensembles (Random Forest, Boosting) use karte hain.

Q72 INTERMEDIATE How do you read feature importance from a tree, and what is the caveat?

Interview answer. `feature_importances_` sums impurity reduction per feature, but it is biased toward high-cardinality features and unreliable with correlated ones; permutation importance is safer.

Hinglish samjho. Importance bata deta hai kaun sa feature zyada split mein kaam aaya, par ye bharosemand nahi hamesha. Permutation importance se cross-check karo.

Q73 BEGINNER What is Naive Bayes?

Interview answer. A classifier that applies Bayes' theorem assuming features are conditionally independent given the class.

Hinglish samjho. Har class ke liye "prior x har feature ki likelihood" multiply karo, jiska score zyada wahi class. "Naive" kyunki features ko independent maan liya.

Q74 BEGINNER State Bayes' theorem.

Interview answer. $P(c \mid x) = \dfrac{P(x \mid c)P(c)}{P(x)}$.

Hinglish samjho. Posterior = likelihood x prior / evidence. Hum $P(x)$ ko ignore kar sakte hain kyunki sab classes ke liye same hota hai.

Q75 INTERMEDIATE Why is Naive Bayes called "naive" and does it still work?

Interview answer. Because it assumes features are independent given the class, which is rarely true; it often still ranks classes correctly, though its probabilities are over-confident.

Hinglish samjho. Assumption galat hone par bhi classification accha chalta hai kyunki sirf sabse bada score chahiye, exact probability nahi.

Q76 INTERMEDIATE What is the difference between GaussianNB, MultinomialNB and BernoulliNB?

Interview answer. GaussianNB models continuous features with a normal distribution per class, MultinomialNB uses counts (text), and BernoulliNB uses binary presence/absence features.

Hinglish samjho. Continuous numbers = Gaussian, word counts = Multinomial, 0/1 features = Bernoulli.

Q77 INTERMEDIATE What is Laplace (additive) smoothing?

Interview answer. Adding a small constant α to counts so unseen feature-class combinations do not get zero probability that wipes out the product.

Hinglish samjho. Agar koi word class mein kabhi nahi aaya to probability 0 aur poora product 0. Smoothing har count mein thoda jod deti hai. GaussianNB mein `var_smoothing` similar role nibhata hai.

Q78 INTERMEDIATE Why use log-probabilities in Naive Bayes?

Interview answer. Multiplying many small probabilities underflows; summing logs is stable and gives the same ranking.

Hinglish samjho. Chhote numbers ko multiply karte jaoge to zero ho jayega. Log lekar jodna safe hai.

Q79 INTERMEDIATE **When is Naive Bayes a good choice?**

Interview answer. For small datasets, very high-dimensional sparse data such as text, as a fast baseline, and when training or prediction speed matters.

Hinglish samjho. Spam filtering, document classification, ya jab data kam ho aur jaldi baseline chahiye.

CHAPTER 38

Interview Q&A · Support Vector Machine

SVM questions test geometric intuition (margin, support vectors) and practical knowledge of C, gamma and scaling.

Q80 BEGINNER **What is an SVM?**

Interview answer. A Support Vector Machine finds the hyperplane that separates classes with the largest margin; with kernels it can create non-linear boundaries.

Hinglish samjho. Do groups ke beech sabse chaudi sadak banao. Sadak jitni chaudi, utna accha generalisation.

Q81 BEGINNER **What are support vectors?**

Interview answer. The training points closest to the decision boundary (on or inside the margin); only they determine the boundary.

Hinglish samjho. Baaki points hata do to boundary nahi hilti, par support vector hata do to hil jati hai.

Q82 INTERMEDIATE **What is the margin and why maximise it?**

Interview answer. The margin is the distance between the boundary and the nearest points of each class, $2/\|w\|$. A larger margin generally means better generalisation.

Hinglish samjho. Boundary jitni door sab points se, utna kam chance ki naya point galat side jaye.

Q83 INTERMEDIATE **What is the role of the `C` parameter?**

Interview answer. `C` controls the trade-off between a wide margin and training errors: large `C` penalises mistakes heavily (narrow margin, overfit risk), small `C` allows more violations (smoother boundary).

Hinglish samjho. Bada C = "galti bilkul nahi", chhota C = "thodi galti chalegi, boundary smooth rakho".

Q84 INTERMEDIATE **What is the kernel trick?**

Interview answer. It computes inner products in a high-dimensional feature space through a kernel function without explicitly mapping the data, so a linear separator in that space becomes a non-linear boundary in the original space.

Hinglish samjho. Data ko upar ke dimension mein bhejne ka kaam shortcut formula se ho jata hai, mapping banani nahi padti.

Q85 INTERMEDIATE **Name common kernels and when to use them.**

Interview answer. Linear (many features, text), polynomial, RBF (a good default for non-linear data) and sigmoid.

Hinglish samjho. Pehle linear try karo, phir RBF. RBF ke saath `C` aur `gamma` tune karna padta hai.

Q86 INTERMEDIATE What does `gamma` do in the RBF kernel?

Interview answer. `gamma` sets how far the influence of one training point reaches: high gamma gives wiggly, local boundaries (overfit); low gamma gives smooth ones.

Hinglish samjho. Bada gamma = har point ka asar chhota area mein, boundary ghumavdaar. Chhota gamma = smooth boundary.

Q87 INTERMEDIATE Why must you scale features for SVM?

Interview answer. Margins and RBF distances depend on feature scale; unscaled features dominate the kernel.

Hinglish samjho. SVM distance par chalta hai, to StandardScaler zaroori hai, aur Pipeline ke andar.

Q88 INTERMEDIATE Hard margin vs soft margin?

Interview answer. Hard margin requires perfect separation (only possible and sensible for clean separable data); soft margin allows violations with slack variables controlled by `C`.

Hinglish samjho. Real data noisy hota hai, isliye soft margin hi use hota hai.

Q89 INTERMEDIATE How does SVM give probabilities?

Interview answer. It does not natively; `SVC(probability=True)` fits a Platt-scaling model through internal cross-validation, which is slower. Otherwise use `decision_function` scores.

Hinglish samjho. SVM sirf side batata hai. Probability chahiye to `probability=True` lagao (slow) ya CalibratedClassifierCV.

Q90 INTERMEDIATE What are the complexity and limitations of kernel SVM?

Interview answer. Training is roughly between $O(n^2)$ and $O(n^3)$ and memory grows with the kernel matrix, so it suits small to medium datasets; for large data use `LinearSVC` or SGD.

Hinglish samjho. Lakhon rows par RBF SVM bahut slow hai. Tab linear SVM ya tree ensembles lo.

Q91 SCENARIO SVM accuracy is poor on a non-linear dataset. What do you try?

Interview answer. Scale the features, switch to an RBF kernel, tune `C` and `gamma` with grid or random search using cross-validation inside a Pipeline.

Hinglish samjho. Scaling, RBF kernel, phir C aur gamma ko log-scale par search karo. Test set ko haath mat lagao.

Q92 BEGINNER Can SVM be used for regression?

Interview answer. Yes, Support Vector Regression (`SVR`) fits a tube of width ϵ around the data and penalises points outside it.

Hinglish samjho. SVR mein ek "tube" hota hai; tube ke andar wale points ki koi galti count nahi hoti.

CHAPTER 39

Interview Q&A · Evaluation Metrics

Be ready to define each metric, compute it from a confusion matrix, and say which one fits a business situation.

Q93 BEGINNER **What is MAE?**

Interview answer. Mean Absolute Error is the average of $|y - \hat{y}|$, in the same units as the target.

Hinglish samjho. Har galti ko positive banao aur average lo. "Average hum 4 lakh galat hain."

Q94 BEGINNER **What is MSE and RMSE?**

Interview answer. MSE is the mean of squared errors; RMSE is its square root, returning to target units and penalising large errors more than MAE.

Hinglish samjho. Square karne se badi galtiyan zyada chubti hain. RMSE ka unit wahi hota hai jo target ka.

Q95 INTERMEDIATE **When would you prefer MAE over RMSE?**

Interview answer. When all errors cost the same and outliers are noise; RMSE when large errors are disproportionately costly.

Hinglish samjho. Outlier se bachna ho to MAE; badi galti bahut mehangi ho to RMSE.

Q96 INTERMEDIATE **Is RMSE always larger than MAE?**

Interview answer. RMSE is greater than or equal to MAE; they are equal only when all absolute errors are equal.

Hinglish samjho. Squares ka average hamesha absolute average se zyada ya barabar hota hai (Jensen). Gap jitna bada, utne outliers.

Q97 BEGINNER **What is R^2 score?**

Interview answer. The fraction of variance in the target explained by the model, $1 - \mathrm{SSE}/\mathrm{SST}$.

Hinglish samjho. 0.9 matlab model ne 90% variation samjha liya. Baseline average-model se compare karta hai.

Q98 INTERMEDIATE **Why can a high R^2 still be misleading?**

Interview answer. It can come from overfitting, a few extreme points, or non-linear patterns hidden in residuals; it also never decreases when features are added.

Hinglish samjho. Residual plot zaroor dekho. High R^2 ke saath bhi errors me pattern ho sakta hai.

Q99 BEGINNER **What is accuracy?**

Interview answer. The share of all predictions that are correct, $(TP + TN)/N$.

Hinglish samjho. Kitne predictions sahi hue, total mein se. Imbalanced data mein dhoka de sakta hai.

Q100 BEGINNER What is precision?

Interview answer. Of everything predicted positive, the fraction that is truly positive: $TP/(TP+FP)$.

Hinglish samjho. Jisko model ne "haan" bola, usme se kitne sachchi haan the. False alarm kam karna ho to precision dekho.

Q101 BEGINNER What is recall?

Interview answer. Of all real positives, the fraction found: $TP/(TP+FN)$ (also sensitivity or TPR).

Hinglish samjho. Jitne asli positive the, unme se kitne pakde. Kuch miss nahi karna ho (cancer, fraud) to recall dekho.

Q102 BEGINNER What is the F1-score?

Interview answer. The harmonic mean of precision and recall, $2PR/(P+R)$.

Hinglish samjho. Dono ka balance. Harmonic mean isliye kyunki agar ek bhi bahut low ho to F1 gir jata hai.

Q103 INTERMEDIATE Why harmonic mean and not arithmetic mean for F1?

Interview answer. The harmonic mean is dominated by the smaller value, so a model cannot hide terrible recall behind great precision.

Hinglish samjho. Precision 1.0 aur recall 0.01 ka arithmetic mean 0.5 dikhata hai, par F1 sirf 0.02 aata hai: sach yahi hai.

Example. $P=1, R=0.01 \Rightarrow F_1 = 2(0.01)/1.01 \approx 0.02$.

Q104 INTERMEDIATE Precision or recall: which matters more for spam filtering and for cancer screening?

Interview answer. Spam filter: precision (do not send real mail to spam). Cancer screening: recall (do not miss sick patients).

Hinglish samjho. Jis galti ki keemat zyada hai uske hisaab se metric chuno. Spam mein false positive mehenga, cancer mein false negative.

Q105 BEGINNER What is a confusion matrix?

Interview answer. A table of true versus predicted classes. For binary tasks it holds TN, FP, FN and TP.

Hinglish samjho. Model ki galtiyon ka naksha. scikit-learn mein rows asli class, columns predicted class hote hain.

Q106 INTERMEDIATE What are Type I and Type II errors?

Interview answer. Type I error is a false positive; Type II error is a false negative.

Hinglish samjho. Type I = galat alarm (FP). Type II = miss ho gaya (FN).

Q107 INTERMEDIATE What does the classification report show?

Interview answer. Precision, recall, F1 and support for each class, plus accuracy, macro average and weighted average.

Hinglish samjho. Har class ka report card. Support matlab us class ke kitne asli samples hain.

Q108 INTERMEDIATE What is the difference between macro, micro and weighted averages?

Interview answer. Macro averages per-class scores equally, weighted averages them by class size, and micro pools all TP/FP/FN counts before computing.

Hinglish samjho. Chhoti class ka bhi barabar vote chahiye to macro; badi class ko zyada weightage chahiye to weighted.

Q109 INTERMEDIATE What are ROC curve and AUC?

Interview answer. The ROC curve plots true positive rate against false positive rate across thresholds; AUC is the area under it and equals the probability that a random positive is scored above a random negative.

Hinglish samjho. AUC 0.5 = random guess, 1.0 = perfect. Ye threshold par depend nahi karta, isliye models compare karne ke liye accha hai.

Q110 INTERMEDIATE When is PR-AUC better than ROC-AUC?

Interview answer. With heavy class imbalance, because ROC can look optimistic when negatives are plentiful; precision-recall focuses on the minority class.

Hinglish samjho. Fraud jaise rare cases mein ROC-AUC bahut acha dikha sakta hai jabki precision kharab ho. PR curve sachchai dikhata hai.

Q111 INTERMEDIATE What is specificity?

Interview answer. The fraction of real negatives correctly identified, $TN/(TN+FP)$.

Hinglish samjho. Recall positives ke liye, specificity negatives ke liye. FPR $= 1-$ specificity.

Q112 INTERMEDIATE What is log loss?

Interview answer. The negative mean log-probability assigned to the true class; it punishes confident wrong predictions heavily and evaluates probability quality.

Hinglish samjho. Sirf sahi/galat nahi, ye dekhta hai ki model kitna confident tha. 0.99 confidence se galat hona sabse bura hai.

Q113 SCENARIO A fraud model has 99.5% accuracy but fraud is 0.5% of data. Is it good?

Interview answer. Not necessarily: predicting "no fraud" always also gives 99.5%. Check recall and precision on the fraud class, PR-AUC and the confusion matrix.

Hinglish samjho. Pehle dekho ki kitne fraud pakde (recall). Agar zero, to model bekaar hai chahe accuracy 99.5% ho.

Q114 SCENARIO How do you choose the threshold for a classifier?

Interview answer. Pick it on validation data using the business cost of FP versus FN or a target recall/precision, via the PR or ROC curve, never on the test set.

Hinglish samjho. Cost batao: ek FN ki keemat kitni, ek FP ki kitni. Uske hisaab se threshold validation set par chuno.

Q115 INTERMEDIATE What is balanced accuracy?

Interview answer. The average of recall over classes; for binary data, $(\mathrm{TPR} + \mathrm{TNR})/2$. It is not fooled by imbalance.

Hinglish samjho. Har class ka recall nikalo aur average lo; majority-class ka dabdaba khatam.

Q116 INTERMEDIATE What is MAPE and when does it fail?

Interview answer. Mean Absolute Percentage Error is the average of $|y - \hat{y}|/|y|$; it explodes or is undefined when true values are near or equal to zero and penalises over-prediction less than under-prediction.

Hinglish samjho. Percent mein samajhna aasan hai par jab actual value 0 ke paas ho to bekaar ho jata hai.

CHAPTER 40

Interview Q&A · Ensemble Learning

Ensembles are interview favourites: know bagging vs boosting cold, and be able to explain why Random Forest and gradient boosting work.

Q117 BEGINNER **What is ensemble learning?**

Interview answer. Combining several models so that the combined prediction is more accurate and robust than any single model.

Hinglish samjho. Akela insaan galat ho sakta hai par bahut logon ki raay aksar sahi hoti hai. Ensemble yahi karta hai models ke saath.

Q118 BEGINNER **Name the main types of ensemble methods.**

Interview answer. Bagging (parallel, averaging), boosting (sequential, error correcting) and stacking (a meta-model combines base models); voting is a simple version of combining.

Hinglish samjho. Bagging = alag-alag models average, boosting = ek ke baad ek galti sudhaarte models, stacking = ek model jo doosre models ke answers ko combine kare.

Q119 BEGINNER **What is bagging?**

Interview answer. Bootstrap aggregating: train many models on bootstrap samples (random samples with replacement) and average or vote their predictions.

Hinglish samjho. Data ki random copies banao (kuch rows repeat, kuch gayab), har copy par model train karo, aur sabka average lo. Variance kam hota hai.

Q120 INTERMEDIATE **Why does bagging reduce variance but not bias?**

Interview answer. Averaging models with independent errors cancels noise (variance falls), while each model has the same bias as the base learner, so the average keeps that bias.

Hinglish samjho. Galtiyan alag-alag jagah hoti hain to average mein kat jati hain. Par agar sab models ek hi tarah se simple hain (bias), to average bhi wahi bias rakhta hai.

Q121 INTERMEDIATE **What is a bootstrap sample and how much of the data does it contain?**

Interview answer. A sample of size n drawn with replacement; it contains about 63.2% of the unique rows, leaving about 36.8% out-of-bag.

Hinglish samjho. $1-(1-1/n)^n \to 1-e^{-1} \approx 0.632$. Jo rows chhoot gaye (OOB) unse free validation mil jata hai.

Q122 INTERMEDIATE **What is the out-of-bag (OOB) score?**

Interview answer. An internal validation estimate: each row is predicted only by the trees that did not see it in their bootstrap sample.

Hinglish samjho. Alag validation set ki zaroorat nahi. `oob_score=True` lagao aur free mein generalisation estimate pao.

Q123 BEGINNER What is a Random Forest?

Interview answer. A bagged ensemble of decision trees where each split considers only a random subset of features, which decorrelates the trees.

Hinglish samjho. Bagging + har split par random features. Trees ek doosre se alag sochte hain, to average aur behtar hota hai.

Q124 INTERMEDIATE How does Random Forest differ from plain bagging of trees?

Interview answer. Bagging uses all features at every split; Random Forest samples a subset (`max_features`), making trees less correlated and the ensemble stronger.

Hinglish samjho. Agar ek feature bahut strong hai to bagging ke saare trees usi se start karenge aur ek jaise ho jayenge. RF ye roka deta hai.

Q125 INTERMEDIATE Which Random Forest hyper-parameters matter most?

Interview answer. `n_estimators` , `max_features` , `max_depth` , `min_samples_leaf` and `class_weight` for imbalanced data.

Hinglish samjho. Trees ki sankhya, har split par kitne features, aur tree ki depth/leaf size. Zyada trees kabhi overfit nahi karte, bas slow hote hain.

Q126 INTERMEDIATE Can Random Forest overfit if you add more trees?

Interview answer. Adding trees does not overfit in the usual sense; error plateaus. Overfitting comes from very deep, unconstrained trees on noisy data.

Hinglish samjho. Trees badhane se performance stable hoti hai. Overfit tab hota hai jab har tree khud bahut gehra aur noisy data par ho.

Q127 INTERMEDIATE How do you get feature importance from a Random Forest and what are the limits?

Interview answer. Use impurity-based `feature_importances_` (biased toward high-cardinality features) or permutation importance on held-out data (more reliable).

Hinglish samjho. Impurity importance fast hai par bias hai. Permutation importance: feature ko shuffle karo aur dekho score kitna girta hai.

Q128 INTERMEDIATE Does Random Forest need feature scaling?

Interview answer. No, trees split on order, not scale.

Hinglish samjho. Nahi. Trees ko scaling ki zaroorat nahi hoti.

Q129 INTERMEDIATE Can Random Forest extrapolate?

Interview answer. No. Predictions are averages of training targets, so they stay within the training range.

Hinglish samjho. Agar training mein price max 150 lakh tha, forest 200 lakh predict nahi kar sakta. Trend extrapolation ke liye linear ya boosting+linear socho.

Q130 BEGINNER What is boosting?

Interview answer. Training weak models sequentially, each focusing on the mistakes of the previous ones, and combining them into a strong model.

Hinglish samjho. Teacher jo har test ke baad kamzor sawaalon par zyada dhyan de. Models line mein train hote hain.

Q131 INTERMEDIATE Bagging vs boosting: key differences?

Interview answer. Bagging trains models independently in parallel to cut variance; boosting trains them sequentially to cut bias. Boosting is more prone to overfitting noise and cannot be parallelised across rounds.

Hinglish samjho. Bagging = parallel + variance kam; boosting = sequential + bias kam. Noise zyada ho to boosting dhyan se tune karo.

Q132 BEGINNER What is AdaBoost?

Interview answer. A boosting algorithm that reweights training samples so that misclassified ones count more, and combines weak learners (often stumps) with weights based on their accuracy.

Hinglish samjho. Galat samples ka weight badhta hai, agla stump unhe sahi karne par focus karta hai. Final vote mein accurate stump ka weight zyada.

Q133 INTERMEDIATE How is a learner's weight computed in AdaBoost?

Interview answer. $\alpha_t = \frac{1}{2}\ln\frac{1-\varepsilon_t}{\varepsilon_t}$, so a more accurate learner (small error ε_t) gets a larger weight.

Hinglish samjho. Error 0.2 par $\alpha = 0.693$; error 0.5 (random) par $\alpha = 0$. Random learner ko koi voice nahi.

Q134 INTERMEDIATE Why is AdaBoost sensitive to noise and outliers?

Interview answer. Mislabelled or outlying samples are repeatedly misclassified, so their weights keep growing and later learners overfit them.

Hinglish samjho. Galat label wala sample baar-baar "mushkil sawaal" ban jata hai aur model uske peeche padh jata hai.

Q135 BEGINNER What is gradient boosting?

Interview answer. An additive model built stage by stage, where each new tree is fit to the negative gradient (the residuals for squared error) of the current model and added with a small learning rate.

Hinglish samjho. Pehle model ki galtiyan nikalo, ek chhota tree un galtiyon ko predict kare, usko thoda sa jodo. Dohrate raho.

Q136 INTERMEDIATE What is the role of the learning rate in gradient boosting?

Interview answer. It scales each tree's contribution; a smaller rate needs more trees but usually generalises better (shrinkage).

Hinglish samjho. Chhote kadam, zyada kadam: `learning_rate` kam aur `n_estimators` zyada aksar behtar results deta hai.

Q137 INTERMEDIATE Which gradient boosting hyper-parameters do you tune first?

Interview answer. `n_estimators` with `learning_rate`, `max_depth` (3 to 5), `subsample`, and `min_samples_leaf`; use early stopping to choose the number of trees.

Hinglish samjho. Learning rate + trees ki sankhya ek saath, phir depth. Early stopping validation score dekhkar khud ruk jata hai.

Q138 INTERMEDIATE How does gradient boosting differ from AdaBoost?

Interview answer. AdaBoost reweights samples and uses exponential loss; gradient boosting fits residuals (gradients) of any differentiable loss, so it is more flexible.

Hinglish samjho. AdaBoost sample weights badalta hai, gradient boosting loss ka gradient follow karta hai, isliye alag-alag loss lag sakte hain.

Q139 INTERMEDIATE What is the difference between GradientBoosting, XGBoost and LightGBM?

Interview answer. All implement gradient-boosted trees; XGBoost and LightGBM add regularisation, faster histogram or leaf-wise algorithms and engineering optimisations. scikit-learn's `HistGradientBoosting*` is its fast counterpart.

Hinglish samjho. Idea ek hi hai; XGBoost/LightGBM zyada tez aur feature-rich hain. Large data par inhe prefer karte hain.

Q140 BEGINNER What is stacking?

Interview answer. Training a meta-model on the predictions of several base models to learn how best to combine them.

Hinglish samjho. Teen doctors ki raay ko ek senior doctor combine karta hai. Base models alag type ke hone chahiye.

Q141 INTERMEDIATE Why must stacking use out-of-fold predictions for the meta-model?

Interview answer. In-sample base predictions are over-optimistic; training the meta-model on them leaks target information and overfits. Out-of-fold predictions mimic unseen data.

Hinglish samjho. Base model apne hi training data par bahut accha dikhta hai. Meta-model ko "unseen jaise" predictions chahiye, isliye cross-validation wali predictions.

Q142 INTERMEDIATE Voting vs stacking: what is the difference?

Interview answer. Voting combines models with a fixed rule (majority or average); stacking learns the combination with a meta-model.

Hinglish samjho. Voting mein sabka barabar vote; stacking mein model seekhta hai kiski baat kitni maanni hai.

Q143 INTERMEDIATE Hard voting vs soft voting?

Interview answer. Hard voting counts predicted classes; soft voting averages predicted probabilities, which usually performs better when probabilities are reasonable.

Hinglish samjho. Hard = class ka vote, soft = probability ka average. Soft mein confident models ka asar zyada.

Q144 INTERMEDIATE What makes an ensemble work well?

Interview answer. The members must be individually better than random and diverse: their errors should be as uncorrelated as possible.

Hinglish samjho. Sab models ek jaise galtiyan karein to ensemble ka koi fayda nahi. Diversity zaroori hai (alag algorithms, alag features, alag data samples).

Q145 INTERMEDIATE What are the downsides of ensembles?

Interview answer. More compute and memory, slower prediction, harder interpretation and a more complex deployment.

Hinglish samjho. Accuracy badhti hai par model bada, slow aur explain karna mushkil ho jata hai.

Q146 SCENARIO Your single decision tree overfits. What would you try next?

Interview answer. Try Random Forest to reduce variance, or a shallow gradient-boosted model with early stopping, tuned by cross-validation; also constrain depth and leaf size.

Hinglish samjho. Ek tree ratta maar raha hai to forest lagao (variance kam). Agar accuracy aur chahiye to gradient boosting with early stopping.

Q147 SCENARIO You need the best accuracy on a tabular competition. What is your approach?

Interview answer. Strong preprocessing, a tuned gradient-boosting model (XGBoost, LightGBM or `HistGradientBoosting`), several diverse models, then blending or stacking validated with consistent cross-validation.

Hinglish samjho. Features accha banao, boosting tune karo, kuch alag models banao aur unko stack/blend karo. Sab kuch same CV folds par validate karo.

CHAPTER 41

Interview Q&A · Model Tuning and Cross-Validation

Questions on how to pick settings without cheating: cross-validation, grid and random search, nested CV.

Q148 BEGINNER **What is k-fold cross-validation?**

Interview answer. Split the training data into k folds; train on $k-1$ and validate on the remaining fold, rotating so each fold is validated once, then average the scores.

Hinglish samjho. Data ke k hisse, har baar ek hissa exam aur baaki padhai. k scores ka average final estimate hota hai.

Q149 INTERMEDIATE **How do you choose the number of folds k in cross-validation?**

Interview answer. 5 or 10 is typical: larger k lowers bias of the estimate but increases cost and variance; small datasets may use repeated k-fold.

Hinglish samjho. 5 ya 10 standard hai. Bahut chhota data ho to repeated k-fold ya Leave-One-Out socho.

Q150 INTERMEDIATE **What is stratified k-fold and when do you use it?**

Interview answer. Each fold keeps the class proportions of the full data; use it for classification, especially with imbalance.

Hinglish samjho. Har fold mein same class ratio, taaki kisi fold mein minority class gayab na ho. scikit-learn classifier ke liye default yahi use karta hai.

Q151 INTERMEDIATE **What is Leave-One-Out CV?**

Interview answer. $k = n$: each sample is the validation set once. It is nearly unbiased but expensive and high variance.

Hinglish samjho. Har ek row baari-baari test banti hai. Bahut kam data ho tab hi use karo.

Q152 INTERMEDIATE **When should you use GroupKFold or TimeSeriesSplit?**

Interview answer. GroupKFold when samples are grouped (patients, users) and a group must not appear in both train and validation; TimeSeriesSplit for temporal data so training always precedes validation.

Hinglish samjho. Ek hi patient ke rows train aur test dono mein ho to leakage. Time series mein future ko past predict karne mat do.

Q153 INTERMEDIATE **Why must preprocessing be inside the cross-validation loop?**

Interview answer. Fitting scalers or imputers on all data lets validation folds influence training. A Pipeline refits preprocessing on each training fold.

Hinglish samjho. Pipeline ke andar scaler har fold ke train hisse se hi seekhta hai. Bahar karoge to leakage.

Q154 BEGINNER What is hyper-parameter tuning?

Interview answer. Searching for the hyper-parameter values (such as `max_depth`, `C`) that give the best validation performance.

Hinglish samjho. Model ki settings ka best combination dhoondhna, cross-validation score ke basis par.

Q155 BEGINNER What does GridSearchCV do?

Interview answer. It evaluates every combination in a parameter grid with cross-validation and keeps (and refits) the best one.

Hinglish samjho. Sab combinations try karta hai: 4 values x 3 values = 12 combinations, aur har ek par k-fold CV.

Q156 INTERMEDIATE What does RandomizedSearchCV do and why use it?

Interview answer. It samples `n_iter` random combinations from lists or distributions; it covers many parameters with a fixed budget and often finds a near-best setting faster than a grid.

Hinglish samjho. Budget fix: sirf 30 random combinations. Jab parameters bahut hon ya continuous hon, grid se behtar.

Q157 INTERMEDIATE Grid search vs random search: when to use which?

Interview answer. Grid for few parameters with a few candidate values; random search for many parameters or continuous ranges, or when each fit is expensive.

Hinglish samjho. Chhota search space = grid; bada ya continuous = random, phir best ke aas-paas chhota grid.

Q158 INTERMEDIATE How many model fits does GridSearchCV perform?

Interview answer. (number of combinations) $\times$ (number of folds), plus one final refit if `refit=True`.

Hinglish samjho. 12 combinations aur 5-fold = 60 fits (+1 refit). Isliye grid jaldi mehnga ho jata hai.

Q159 INTERMEDIATE What is nested cross-validation?

Interview answer. An outer CV loop estimates performance while an inner loop tunes hyper-parameters, so the reported score is not optimistically biased by tuning.

Hinglish samjho. Tuning aur evaluation ko alag loops mein rakho. Warna best CV score "tuning-ke-baad ka lucky score" ban jata hai.

Q160 INTERMEDIATE Why use `loguniform` for parameters like `C` and `gamma`?

Interview answer. Their useful values span orders of magnitude, so uniform sampling would waste almost all samples on the largest decade.

Hinglish samjho. C ke liye 0.01 se 1000 tak range ho to log scale par sample karo, tabhi har decade barabar try hoga.

Q161 INTERMEDIATE **What should you do if the best parameter sits at the edge of your grid?**

Interview answer. Extend the grid in that direction; the true optimum may lie outside.

Hinglish samjho. Agar best value kinare par hai to aur aage bhi dekho, shayad wahan behtar value ho.

Q162 SCENARIO **Your grid search takes too long. How do you speed it up?**

Interview answer. Use RandomizedSearchCV or successive halving, a coarser grid first, fewer folds, a subset of data for exploration, `n_jobs` for parallelism and a faster model.

Hinglish samjho. Pehle random search ya halving, phir chhota fine grid. Parallel jobs lagao aur pehle chhote data par check karo.

Q163 SCENARIO **After tuning, how do you report final performance?**

Interview answer. Refit the best model on the full training data and evaluate once on the untouched test set (or use nested CV), reporting mean and spread.

Hinglish samjho. Test set sirf ek baar. Agar tuning ke baad test dekh kar phir tune karte raho to test bhi training ban jata hai.

CHAPTER 42

Interview Q&A · Unsupervised Learning: K-Means, DBSCAN, PCA

Expect "how does it work", "how do you choose its parameter" and "when does it fail" for each algorithm.

Q164 BEGINNER **What is clustering?**

Interview answer. Grouping data points so that points in the same group are more similar to each other than to those in other groups, without labels.

Hinglish samjho. Bina label ke similar cheezon ko saath rakhna, jaise customers ke groups.

Q165 BEGINNER **How does K-Means work?**

Interview answer. Choose k centroids, assign each point to the nearest centroid, move each centroid to the mean of its points, and repeat until assignments stop changing.

Hinglish samjho. Assign karo, average nikalo, centroid khiskao; yahi loop chalta rehta hai jab tak centroid hilna band na karein.

Q166 INTERMEDIATE **What objective does K-Means minimise?**

Interview answer. Inertia: the sum of squared distances from each point to its assigned centroid (within-cluster sum of squares).

Hinglish samjho. Har point apne kendra se kitna door hai uske squares ka jod. Kam inertia = tight clusters.

Q167 INTERMEDIATE **How do you choose the number of clusters k in K-Means?**

Interview answer. Use the elbow method on inertia, silhouette score, domain knowledge and stability across seeds; there is no single correct method.

Hinglish samjho. Elbow par curve mudta hai wahi k. Silhouette zyada ho. Aur business ke liye kitne groups kaam ke hain, ye bhi dekho.

Q168 INTERMEDIATE **What is the silhouette score?**

Interview answer. For each point $(b-a)/\max(a,b)$, where a is mean distance to its own cluster and b to the nearest other cluster; values near 1 are well clustered, near 0 are on a border, negative are likely misassigned.

Hinglish samjho. 1 ke paas = sahi cluster mein, 0 ke paas = do clusters ke beech, negative = shayad galat cluster mein.

Q169 INTERMEDIATE **Why does K-Means need feature scaling?**

Interview answer. It uses Euclidean distance, so features with larger ranges dominate; standardise first.

Hinglish samjho. Salary ka bada number saari clustering ko apni taraf kheench lega. StandardScaler lagao.

Q170 INTERMEDIATE What is k-means++ initialisation?

Interview answer. A smarter start that picks initial centroids far apart with probability proportional to squared distance, leading to better and more stable solutions.

Hinglish samjho. Random centroid kharab jagah pad sakte hain. k-means++ unhe door-door chunta hai. scikit-learn ka default yahi hai.

Q171 INTERMEDIATE Does K-Means always find the global optimum?

Interview answer. No, it converges to a local minimum that depends on initialisation, so run it several times (`n_init`) and keep the best.

Hinglish samjho. Alag random start, alag answer. `n_init=10` karke sabse kam inertia wala le lo.

Q172 INTERMEDIATE What are the limitations of K-Means?

Interview answer. You must pick k; it assumes round, similar-sized clusters; it is sensitive to outliers and scale; it gives hard assignments and cannot find arbitrary shapes.

Hinglish samjho. Tedhe-medhe ya alag size/density ke clusters par fail. Outlier centroid ko kheench lete hain.

Q173 BEGINNER What is DBSCAN?

Interview answer. A density-based clustering algorithm that groups points with enough close neighbours and labels isolated points as noise.

Hinglish samjho. Ghane ilaake = cluster, akele points = noise. K nahi dena padta.

Q174 INTERMEDIATE What are `eps` and `min_samples` in DBSCAN?

Interview answer. `eps` is the neighbourhood radius; `min_samples` is the number of points (including itself) within `eps` needed for a core point.

Hinglish samjho. eps = padosi kitni door tak, min_samples = ghana kehne ke liye kam se kam kitne points.

Q175 INTERMEDIATE What are core, border and noise points?

Interview answer. A core point has at least `min_samples` points within `eps` ; a border point is within `eps` of a core point but is not core; the rest are noise (label -1).

Hinglish samjho. Core = ghane ke andar, border = kinare par, noise = akela.

Q176 INTERMEDIATE How do you choose `eps` ?

Interview answer. Plot the sorted distance to each point's k-th nearest neighbour (with $k =$ `min_samples`) and pick the "knee" of the curve.

Hinglish samjho. k-distance graph banao; jahan curve achanak upar jata hai wahi eps ke aas-paas hai.

Q177 INTERMEDIATE When is DBSCAN better than K-Means?

Interview answer. For clusters of arbitrary shape, unknown number of clusters, and when outliers must be identified.

Hinglish samjho. Chand jaise shape, nested clusters, ya noise detection chahiye to DBSCAN.

Q178 INTERMEDIATE What are DBSCAN's weaknesses?

Interview answer. Sensitive to `eps`, struggles with clusters of very different density and with high-dimensional data, and cannot predict labels for new points directly.

Hinglish samjho. Ek hi eps sab density ke liye nahi chalta. Varying density ke liye HDBSCAN dekho.

Q179 BEGINNER What is PCA?

Interview answer. Principal Component Analysis finds orthogonal directions (components) of maximum variance and projects data onto the top few to reduce dimensions.

Hinglish samjho. Naye axes jo data ka sabse zyada phailav pakadte hain. Pehle kuch axes rakh lo, baaki feature hata do.

Q180 INTERMEDIATE How does PCA work mathematically?

Interview answer. Centre (and usually scale) the data, compute the covariance matrix, take its eigenvectors sorted by eigenvalue, and project onto the top k; scikit-learn uses SVD.

Hinglish samjho. Eigenvector = direction, eigenvalue = us direction mein kitna variance. Bade eigenvalue wale directions rakhte hain.

Q181 INTERMEDIATE What is explained variance ratio?

Interview answer. The fraction of total variance captured by each component, $\lambda_j / \sum \lambda$; its cumulative sum guides how many components to keep.

Hinglish samjho. 95% variance chahiye to jitne components par cumulative 0.95 cross ho utne rakho (`PCA(0.95)`).

Q182 INTERMEDIATE Why scale data before PCA?

Interview answer. PCA maximises variance, so unscaled features with large units dominate the first components.

Hinglish samjho. Bina scaling ke bade unit wala feature PC1 par kabza kar leta hai.

Q183 INTERMEDIATE Is PCA supervised? Does it help prediction?

Interview answer. PCA is unsupervised and ignores labels; it can reduce noise and speed models, but the top-variance directions are not guaranteed to be the most predictive.

Hinglish samjho. PCA y ko dekhta hi nahi. Kabhi kam-variance direction mein hi asli signal hota hai.

Q184 INTERMEDIATE **What are the drawbacks of PCA?**

Interview answer. Components are linear mixtures that are hard to interpret, it captures only linear structure and it is sensitive to outliers and scaling.

Hinglish samjho. PC1 ka matlab samjhana mushkil hai. Curved structure ke liye kernel PCA, t-SNE ya UMAP.

Q185 SCENARIO **You have 200 correlated features and a slow model. How can PCA help?**

Interview answer. Standardise, fit PCA on the training set inside a Pipeline, keep enough components for about 95% variance and compare cross-validated performance and speed with and without it.

Hinglish samjho. Pipeline mein scaler + PCA(0.95) + model, aur CV se check karo ki accuracy bani rahi aur speed badhi ya nahi.

Q186 SCENARIO **How would you evaluate a clustering when there are no labels?**

Interview answer. Use silhouette and Davies-Bouldin scores, stability across seeds and subsamples, profile each cluster for business meaning, and test whether actions based on clusters improve outcomes.

Hinglish samjho. Internal metrics + stability + cluster ka profile table + business impact. Labels hote to ARI/NMI bhi lagate.

CHAPTER 43

Interview Q&A · Data Preprocessing

Preprocessing questions separate people who have trained real models from those who only know algorithms. Always mention leakage and Pipelines.

Q187 BEGINNER Why is data preprocessing needed?

Interview answer. Raw data has missing values, inconsistent scales, text categories and outliers, while most algorithms need clean numeric input; good preprocessing also improves accuracy and training speed.

Hinglish samjho. Kachha data seedha model ko dene se error ya kharab results aate hain. Safai, encoding aur scaling model ko saaf input dete hain.

Q188 BEGINNER What is an outlier?

Interview answer. A data point that is very different from the rest of the observations.

Hinglish samjho. Jaise 10 logon ki salary 40 hazaar hai aur ek ki 10 lakh; wo outlier hai. Wo error bhi ho sakta hai ya asli rare event.

Q189 BEGINNER How does the IQR method detect outliers?

Interview answer. Compute Q_1, Q_3 and $\text{IQR} = Q_3 - Q_1$; values below $Q_1 - 1.5\,\text{IQR}$ or above $Q_3 + 1.5\,\text{IQR}$ are flagged.

Hinglish samjho. Beech ke 50% data ka spread (IQR) nikalo aur uske 1.5 guna se bahar wale points outlier maano. Boxplot ki whiskers yahi dikhati hain.

Example. $Q_1 = 35.75$, $Q_3 = 47.25$, IQR $= 11.5$ gives fences 18.5 and 64.5.

Q190 INTERMEDIATE Why is IQR more robust than the z-score method?

Interview answer. Quartiles are barely affected by extreme values, whereas mean and standard deviation (used by z-scores) are pulled by the outliers themselves.

Hinglish samjho. Outlier khud mean aur std ko badal deta hai, isliye z-score uska pata kam lagata hai. Quartile pe asar kam hota hai.

Q191 INTERMEDIATE What can you do with outliers?

Interview answer. Investigate first, then remove them if they are errors, cap (winsorise) them, transform the feature (log), use robust models or scalers, or keep them if they are genuine and important.

Hinglish samjho. Pehle samjho kyun hain. Galti hai to hatao, asli hai to rakho ya cap karo. Fraud detection mein outlier hi asli maal hai.

Q192 INTERMEDIATE Do all models need outlier treatment?

Interview answer. No. Linear models, KNN, K-Means and SVM are sensitive; tree-based models are fairly robust to outliers in features.

Hinglish samjho. Distance ya squared-loss wale models ko dikkat; trees ko kam.

Q193 BEGINNER What is a scikit-learn Pipeline?

Interview answer. An object that chains preprocessing transformers and a final estimator so `fit` and `predict` run all steps in order.

Hinglish samjho. Assembly line: imputer -> scaler -> model. Ek hi `fit()`, ek hi `predict()`.

Q194 INTERMEDIATE Why use a Pipeline instead of preprocessing manually?

Interview answer. It prevents data leakage in cross-validation, keeps code reproducible, makes deployment a single object and allows tuning preprocessing parameters together with the model.

Hinglish samjho. Sabse bada fayda leakage se bachna: har fold mein scaler sirf train hisse se seekhta hai.

Q195 INTERMEDIATE How do you name parameters of Pipeline steps in GridSearchCV?

Interview answer. Use `stepname__parametername`, for example `svc__C` or `preprocessor__num__imputer__strategy` for nested steps.

Hinglish samjho. Step ke naam ke baad double underscore aur parameter ka naam. Nested ho to har level par `__`.

Q196 INTERMEDIATE What is the difference between `fit`, `transform` and `fit_transform`?

Interview answer. `fit` learns parameters from data (mean, categories); `transform` applies them; `fit_transform` does both on the same data. Use `fit_transform` on train only and `transform` on validation/test.

Hinglish samjho. Train par `fit_transform`, test par sirf `transform`. Test par fit karna = leakage.

Q197 BEGINNER What is ColumnTransformer?

Interview answer. A transformer that applies different preprocessing to different columns (for example scale numeric, one-hot encode categorical) and concatenates the results.

Hinglish samjho. Numeric columns ko scaler, categorical ko encoder; sabko jodkar ek matrix. Pipeline ke andar rakho.

Q198 INTERMEDIATE What does `remainder` do in ColumnTransformer?

Interview answer. It controls columns not listed: `"drop"` (default) discards them, `"passthrough"` keeps them unchanged.

Hinglish samjho. Default mein bacha hua column chupchaap hat jata hai. Dhyan rakho.

Q199 INTERMEDIATE How can you select columns automatically in a ColumnTransformer?

Interview answer. With `make_column_selector`, for example by dtype (`dtype_include="number"`) or a name pattern.

Hinglish samjho. Har column ka naam likhne ki jagah dtype se choose karwa lo.

Q200 BEGINNER What is label encoding?

Interview answer. Converting target class labels to integers with `LabelEncoder`.

Hinglish samjho. "cat/dog/bird" ko 0/1/2 banana. Sirf target y ke liye, features ke liye nahi.

Q201 INTERMEDIATE Why should LabelEncoder not be used on input features?

Interview answer. It assigns arbitrary integers that imply an order and distances (dog=2 is "twice" cat=1), which can mislead linear and distance-based models.

Hinglish samjho. Model sochega dog > cat. Features ke liye OneHot ya Ordinal use karo.

Q202 BEGINNER What is ordinal encoding and when is it appropriate?

Interview answer. Mapping categories with a natural order to integers that preserve that order (small < medium < large); appropriate for ordered categories, and acceptable for tree models.

Hinglish samjho. Jahan "kam se zyada" ka order hota hai: education, size, grade. Order khud `categories=` mein do.

Q203 INTERMEDIATE What is the default category order in OrdinalEncoder and why is it risky?

Interview answer. Alphabetical (sorted), which may not match the real order (for example "high" < "low" < "medium"); pass `categories` explicitly.

Hinglish samjho. Default alphabetical hai, to order galat ho sakta hai. Hamesha explicitly batao.

Q204 BEGINNER What is one-hot encoding?

Interview answer. Replacing a categorical column with one 0/1 column per category so that no order is implied.

Hinglish samjho. Har category ka apna column; sirf ek column 1 hota hai. City=Pune ho to `city_Pune=1`.

Q205 INTERMEDIATE What is the dummy variable trap?

Interview answer. With all K one-hot columns plus an intercept, the columns sum to one and are perfectly collinear. Drop one column (`drop="first"`) for unregularised linear models.

Hinglish samjho. K-1 columns kaafi hain, kyunki K-th ka pata baaki se chal jata hai. Regularised models mein itna fark nahi padta.

Q206 INTERMEDIATE How do you handle unseen categories at prediction time?

Interview answer. Use `OneHotEncoder(handle_unknown="ignore")` (all-zero row) or `OrdinalEncoder(handle_unknown="use_encoded_value", unknown_value=-1)`.

Hinglish samjho. Production mein naya city aa gaya to crash na ho, isliye `handle_unknown="ignore"`.

Q207 INTERMEDIATE How do you encode a high-cardinality categorical feature?

Interview answer. Group rare categories (`min_frequency`), use target or frequency encoding with proper cross-fitting, hashing or embeddings, or tree models with ordinal codes; avoid naive one-hot of thousands of levels.

Hinglish samjho. 5000 pin codes ko one-hot karoge to columns phat jayenge. Rare ko "other" mein daalo ya target encoding (leakage se bachke).

Q208 BEGINNER What is feature scaling and why is it needed?

Interview answer. Rescaling features to comparable ranges so that no feature dominates distance or gradient calculations.

Hinglish samjho. Age (20-60) aur salary (lakhs) ko barabar tolna zaroori hai KNN, SVM, K-Means, PCA aur gradient descent ke liye.

Q209 BEGINNER What does StandardScaler do?

Interview answer. It transforms each feature to $z = (x - \mu)/\sigma$, giving mean 0 and standard deviation 1.

Hinglish samjho. Average ghatao, std se bhaag do. Ab har feature ka center 0 aur spread 1.

Q210 INTERMEDIATE StandardScaler vs MinMaxScaler vs RobustScaler?

Interview answer. Standard: mean/std (unbounded, outlier-sensitive). MinMax: rescales to a range such as [0,1] (outlier-sensitive). Robust: uses median and IQR, so it is resistant to outliers.

Hinglish samjho. Outliers hon to RobustScaler; bounded range chahiye (images, neural nets) to MinMax; baaki mein Standard.

Q211 INTERMEDIATE Which models do not need feature scaling?

Interview answer. Tree-based models (decision tree, random forest, gradient boosting) and Naive Bayes.

Hinglish samjho. Jo models sirf order ya per-feature statistics dekhte hain unhe scaling nahi chahiye.

Q212 INTERMEDIATE Should you scale the target variable?

Interview answer. Usually not for tree models; for some regression or neural models scaling or log-transforming the target helps, but predictions must be inverted (`TransformedTargetRegressor`).

Hinglish samjho. Target ko scale karo to prediction ko wapas original scale mein laana mat bhoolo.

Q213 BEGINNER How can you detect missing values?

Interview answer. With `df.isna().sum()` or `df.isna().mean()` to see counts and percentages, and by checking for placeholders such as "?" or -999.

Hinglish samjho. Sirf NaN nahi, "NA", "?", -999 jaise placeholders bhi missing hote hain.

Q214 BEGINNER What are the common ways to handle missing values?

Interview answer. Drop rows or columns, impute with mean/median/most-frequent or a constant, use KNN or iterative imputation, add a missing-indicator, or use models with native NaN support.

Hinglish samjho. Chhota missing = fill karo, bahut zyada missing = column hatao. Missing hona khud ek information ho sakta hai.

Q215 INTERMEDIATE Mean or median imputation?

Interview answer. Median for skewed data or data with outliers; mean for roughly symmetric data. Median is the safer default.

Hinglish samjho. Salary jaise skewed data mein mean outlier se bhaag jata hai, median stable rehta hai.

Q216 INTERMEDIATE What is the risk of imputing before the train-test split?

Interview answer. Imputation statistics would include test data, leaking information. Fit the imputer on the training set only (inside a Pipeline).

Hinglish samjho. Test ka average training mein ghus jata hai. Imputer ko Pipeline mein rakho.

Q217 INTERMEDIATE What are MCAR, MAR and MNAR?

Interview answer. Missing completely at random (no pattern), at random (depends on other observed columns), and not at random (depends on the missing value itself). The mechanism affects which imputation is safe.

Hinglish samjho. MNAR sabse khatarnak: jaise high-income log salary batate hi nahi. Tab simple imputation bias laata hai.

Q218 INTERMEDIATE What does `add_indicator=True` in SimpleImputer do?

Interview answer. It appends binary columns showing which values were missing, preserving the "was missing" signal.

Hinglish samjho. "Value missing thi" khud ek feature ban jata hai; kabhi kabhi wahi predictive hota hai.

Q219 INTERMEDIATE What is KNNImputer?

Interview answer. It fills a missing value with the mean of that feature among the k nearest rows, measured on the other features; it should be applied after scaling and inside the Pipeline.

Hinglish samjho. Milte-julte rows se value udhaar leta hai. Distance par chalta hai to scaling pehle karo.

Q220 SCENARIO A column has 70% missing values. What do you do?

Interview answer. Investigate why; if it is not informative and cannot be collected, drop it, or keep it as a binary "is present" indicator; imputing 70% of a column mostly invents data.

Hinglish samjho. Itna zyada missing ho to imputation bas andaaza hoga. Ya to hatao ya sirf "present/absent" flag banao.

Q221 SCENARIO **Your model scores 0.97 in cross-validation but 0.80 in production. What preprocessing issue would you check first?**

Interview answer. Data leakage or train-serving skew: preprocessing fitted on all data, features unavailable at prediction time, or different transformations in production. Move everything into one Pipeline and audit features.

Hinglish samjho. Pehle dekho ki kahin leakage to nahi, aur production mein wahi transformation lag raha hai ya nahi. Ek saved Pipeline use karo.

CHAPTER 44

Interview Q&A · Scenario-Based Questions

Scenario questions test judgement. Structure your answer: clarify the goal, check the data, choose a baseline, pick a metric from the business cost, validate properly, then iterate.

Q222 SCENARIO You must predict house prices with 1,000 rows and 15 features. How do you start?

Interview answer. Split off a test set, build a Pipeline with imputation and scaling, start with a mean baseline and Ridge regression, then try Random Forest and gradient boosting, compare by cross-validated RMSE/MAE and inspect residuals.

Hinglish samjho. Pehle baseline, phir linear, phir ensembles. Sab cross-validation se compare karo aur residual plot dekho. Test set ko aakhir mein ek baar.

Q223 SCENARIO A bank wants a loan-approval model. Which metric do you optimise?

Interview answer. It depends on costs: approving a bad loan (false positive) is usually more expensive than rejecting a good one, so I would track precision, recall, F1 and ROC-AUC and tune the threshold to the cost ratio.

Hinglish samjho. Business se poocho ki galat approval kitna mehenga hai. Usi ke hisaab se threshold aur metric chuno. Fairness bhi check karo.

Q224 SCENARIO Only 1% of transactions are fraud. How do you build a model?

Interview answer. Use a stratified split, class weights or resampling inside cross-validation, evaluate with PR-AUC, recall at a fixed precision and the confusion matrix, and tune the threshold on validation data.

Hinglish samjho. Accuracy bekaar hai. `class_weight="balanced"`, stratified CV, aur PR-AUC dekho. Resampling sirf train fold par karo.

Q225 SCENARIO Is it fine to oversample the minority class before splitting the data?

Interview answer. No. Duplicated or synthetic samples would leak into validation or test sets. Resample only inside each training fold.

Hinglish samjho. SMOTE ya oversampling split ke baad aur sirf train par. Warna test mein training ki copies aa jati hain.

Q226 SCENARIO Your classifier is great for one customer group and bad for another. What do you do?

Interview answer. Evaluate metrics per group, check data coverage and label quality for the weak group, add features or more data, consider group-aware thresholds or models, and monitor fairness metrics.

Hinglish samjho. Overall score chhupa leta hai. Group-wise report banao aur kamzor group ke liye data ya features sudharo.

Q227 SCENARIO **Your model performance degrades a few months after deployment. Why and what do you do?**

Interview answer. Likely data drift or concept drift. Monitor input distributions and performance, investigate the change, and retrain or recalibrate on recent data with a scheduled pipeline.

Hinglish samjho. Duniya badal jati hai: customers ka behaviour naya ho gaya. Monitoring lagao aur naye data par retrain karo.

Q228 SCENARIO **You have 50,000 features and 500 rows. What is the risk and what do you try?**

Interview answer. Severe overfitting. Use strong regularisation (Lasso, ElasticNet), feature selection or PCA inside the Pipeline, simple models and rigorous cross-validation.

Hinglish samjho. Features rows se bahut zyada. L1 regularisation ya dimensionality reduction lagao aur simple model rakho.

Q229 SCENARIO **Two models have CV accuracy 0.91 +/- 0.03 and 0.92 +/- 0.03. Which do you pick?**

Interview answer. The difference is within the noise, so I would prefer the simpler, faster or more interpretable model, and compare using repeated CV or a paired test if the decision matters.

Hinglish samjho. 0.01 ka farq std ke andar hai, matlab faraq nahi. Simple model chuno.

Q230 SCENARIO **A stakeholder asks why the model rejected an applicant. How do you answer?**

Interview answer. Use an interpretable model or explanation tools: coefficients for linear models, permutation importance globally, and SHAP or LIME for individual predictions, while checking for proxies of protected attributes.

Hinglish samjho. Logistic regression ke coefficients ya SHAP values se bata sakte ho ki kaun sa feature kitna zimmedar tha.

Q231 SCENARIO **Features include "account closed date" in a churn model. What is wrong?**

Interview answer. It is target leakage: the date is only known after the customer has churned, so the model cannot use it at prediction time. Remove features that are not available when predicting.

Hinglish samjho. Jo cheez prediction ke time pata hi nahi hogi wo feature mat do. Warna offline score jhootha.

Q232 SCENARIO **Predicting next month's sales from daily data. How do you split and validate?**

Interview answer. Use a time-based split and `TimeSeriesSplit`, never shuffle, build lag and rolling features using only past data, and compare against a naive "same as last period" baseline.

Hinglish samjho. Past se train, future par test. Random split se future ki jaankari leak ho jati hai.

Q233 SCENARIO **The target is skewed (a few very expensive houses). What can help?**

Interview answer. Log-transform the target (`TransformedTargetRegressor`), use a robust loss or metric (MAE, Huber), and check whether errors should be judged in relative terms.

Hinglish samjho. Log lene se skew kam hota hai aur model stable hota hai. Prediction ko `exp` se wapas laao.

Q234 SCENARIO **You must cluster customers but features are mixed (numeric and categorical). What do you do?**

Interview answer. Encode and scale carefully (one-hot plus scaling) and use K-Means with caution, or use K-Prototypes or Gower distance with hierarchical clustering; validate with stability and business interpretability.

Hinglish samjho. K-Means numeric ke liye bana hai. Mixed data ke liye K-Prototypes ya Gower distance, aur segments ko profile karke samjho.

Q235 SCENARIO **K-Means gives different clusters each run. How do you stabilise it?**

Interview answer. Use k-means++ with a larger `n_init` and fixed `random_state` , scale features, check the silhouette and ARI between runs, and consider whether k matches the data structure.

Hinglish samjho. Zyada `n_init` aur fixed seed. Agar phir bhi stable nahi to shayad k galat hai ya clusters natural hain hi nahi.

Q236 SCENARIO **Training accuracy is 100% for a Random Forest. Is something wrong?**

Interview answer. Not by itself: a forest with deep trees can fit training data perfectly. Judge by OOB or cross-validated score; if validation is much lower, constrain depth and leaf size.

Hinglish samjho. Forest ke liye training score 100% normal hai. OOB score ya CV score dekho.

Q237 SCENARIO **Your logistic regression does not converge. What do you try?**

Interview answer. Scale the features, increase `max_iter` , try a different solver, strengthen regularisation (smaller `C`) and check for perfect separation or collinearity.

Hinglish samjho. Pehle scaling, phir max_iter badhao, solver badlo. Perfect separation ho to regularisation lagao.

Q238 SCENARIO **How would you choose between KNN, Random Forest and SVM for a new tabular problem?**

Interview answer. I would compare all with the same preprocessing and cross-validation folds, using scaling for KNN and SVM; Random Forest is a strong default, SVM suits small/medium data, KNN suits small data with meaningful distances.

Hinglish samjho. Teeno ko same CV par chalao. Aksar Random Forest ya boosting jeetta hai tabular par, par data hi faisla karta hai.

Q239 SCENARIO **You need predictions in under 5 ms. What changes?**

Interview answer. Prefer a lighter model (linear model, shallow trees, small boosted ensemble), reduce features, avoid KNN and huge ensembles, precompute features and benchmark latency.

Hinglish samjho. KNN aur 1000-tree forest slow hote hain. Chhota model, kam features, aur latency measure karo.

Q240 SCENARIO **How would you explain overfitting to a non-technical manager?**

Interview answer. "The model memorised the past examples, like a student who memorised last year's paper; it looks perfect on old questions but struggles with new ones. We test on unseen data to catch this."

Hinglish samjho. Ratta marne wale student ki tarah: purane paper mein 100%, naye mein fail. Isliye hum unseen data par test karte hain.

Q241 SCENARIO **A colleague tuned hyper-parameters on the test set. What is the problem?**

Interview answer. The test score is no longer an unbiased estimate: the test set influenced model choice. Collect or hold out a new test set, and tune with cross-validation on training data.

Hinglish samjho. Test set ab "padhai ka hissa" ban gaya. Naya test set chahiye ya nested CV use karo.

Q242 SCENARIO **How do you decide the number of trees and stopping point in gradient boosting?**

Interview answer. Use a validation set or `n_iter_no_change` early stopping with a small learning rate, rather than tuning `n_estimators` on the test set.

Hinglish samjho. Validation loss rukne par training band (early stopping). Test set se mat choose karo.

Q243 SCENARIO **How do you make a model's results reproducible?**

Interview answer. Fix `random_state` everywhere, version the data and code, save the whole Pipeline with `joblib`, record library versions and use the same CV splits.

Hinglish samjho. Seed fix, versions note karo, Pipeline save karo. Tabhi kal wahi score dubara aayega.

Q244 SCENARIO **A feature has a correlation of 0.99 with the target in training. Should you celebrate?**

Interview answer. First suspect leakage: check how the feature is produced and whether it is available at prediction time. A near-perfect single feature is rarely real.

Hinglish samjho. Itna perfect hona shaq ki baat hai. Kahin ye feature target se hi to nahi bana?

Q245 SCENARIO **You have to compare a linear model and gradient boosting fairly. What protocol do you use?**

Interview answer. Same train/test split and CV folds, each model inside a Pipeline with appropriate preprocessing and tuned with equal effort, several metrics, uncertainty (std) and a final single test evaluation.

Hinglish samjho. Dono ko barabar mauka do: same folds, same tuning budget, aur aakhir mein test set ek baar.

Q246 SCENARIO **The business wants "the best model" but has not defined success. What do you do first?**

Interview answer. Clarify the objective, the decision the model supports, the cost of each error type, data availability at decision time and a measurable target metric, plus a baseline to beat.

Hinglish samjho. Pehle samjho "best" ka matlab kya hai: kaunsi galti kitni mehengi, kaunsa metric. Bina iske model banana andhere mein teer chalana hai.

Q247 SCENARIO **How do you handle a categorical feature with a rare category that only appears in the test data?**

Interview answer. Use encoders that handle unknown categories (`handle_unknown="ignore"`) or group rare categories into "other" so production data never breaks the pipeline.

Hinglish samjho. Naye category ke liye pipeline crash na ho, isliye unknown handle karna zaroori hai.

Q248 SCENARIO **After PCA your accuracy dropped. Why might that happen and what do you do?**

Interview answer. Discarded low-variance components may hold predictive signal, or PCA was fitted without scaling. Keep more components, tune `n_components` by cross-validation, or skip PCA.

Hinglish samjho. PCA variance dekhta hai, prediction power nahi. Components badhao ya PCA hata ke dekho.

Q249 SCENARIO **You need to deploy a model that uses scaling, one-hot encoding and a classifier. What do you ship?**

Interview answer. A single scikit-learn Pipeline (with a ColumnTransformer) saved via `joblib`, with input schema validation and monitoring, so training and serving transformations are identical.

Hinglish samjho. Poora Pipeline ek file mein save karo. Alag-alag scaler aur model save karoge to galti hogi.

CHAPTER 45

Interview Q&A · Mixed Rapid-Fire Round

Short questions that interviewers use to switch topics quickly. Give a one-line answer, then offer an example if asked.

Q250 BEGINNER **What is the difference between a parameter and an attribute like `coef_` in scikit-learn?**

Interview answer. Hyper-parameters are constructor arguments you choose (`C=1.0`); attributes ending with an underscore (`coef_` , `classes_`) are learned from data during `fit` .

Hinglish samjho. Underscore wale attributes fit ke baad hi milte hain. Fit se pehle unhe access karoge to error aayega.

Q251 INTERMEDIATE **What is ElasticNet?**

Interview answer. Linear regression with a mix of L1 and L2 penalties, controlled by `alpha` and `l1_ratio` ; it keeps Lasso's sparsity while handling correlated features better.

Hinglish samjho. Ridge aur Lasso ka mix: kuch features zero karta hai aur correlated features ko saath handle bhi karta hai.

Q252 INTERMEDIATE **What is class imbalance and name three ways to handle it?**

Interview answer. When one class is much rarer than another. Options: class weights, resampling (inside training folds), threshold tuning, and suitable metrics such as PR-AUC.

Hinglish samjho. Rare class ko zyada weight do (`class_weight="balanced"`), ya resample karo, aur threshold ko validation par tune karo.

Q253 INTERMEDIATE **What is probability calibration?**

Interview answer. Adjusting predicted probabilities so that they match observed frequencies (of the samples predicted 0.8, about 80% should be positive), using Platt scaling or isotonic regression (`CalibratedClassifierCV`).

Hinglish samjho. Model "80% sure" bole to sach mein 80% baar sahi hona chahiye. Naive Bayes aur SVM ko aksar calibration chahiye.

Q254 INTERMEDIATE **What is feature selection and name one filter, wrapper and embedded method?**

Interview answer. Choosing a useful subset of features. Filter: `SelectKBest` with a statistic; wrapper: recursive feature elimination (`RFE`); embedded: Lasso or tree importances.

Hinglish samjho. Bekaar features hatane se model tez, simple aur kam overfit hota hai. Selection bhi Pipeline ke andar karo.

Q255 INTERMEDIATE What is the difference between normalisation and standardisation?

Interview answer. Normalisation (min-max) rescales to a fixed range such as [0,1]; standardisation rescales to mean 0 and standard deviation 1. People also use "normalisation" loosely, so state the formula.

Hinglish samjho. Min-Max = range 0 se 1, Standard = mean 0 std 1. Outliers hon to dono par asar padta hai.

Q256 INTERMEDIATE Why might you log-transform a feature?

Interview answer. To reduce right skew and the influence of extreme values, and to make multiplicative relationships additive; use `np.log1p` when zeros occur.

Hinglish samjho. Salary ya price jaise skewed features par log lene se distribution normal ke kareeb aati hai.

Q257 INTERMEDIATE What is a learning curve and what does it tell you?

Interview answer. A plot of training and validation scores versus training-set size. A large persistent gap suggests overfitting (more data may help); both low and close suggests underfitting (need a richer model).

Hinglish samjho. Data badhane se validation score upar jaye aur gap kam ho to aur data kaam aayega. Dono low hain to model ko taakatwar banao.

Q258 INTERMEDIATE What is a validation curve?

Interview answer. A plot of training and validation scores versus one hyper-parameter (for example `max_depth`) that shows where underfitting turns into overfitting.

Hinglish samjho. Ek hyper-parameter badalte hue score ka graph. Jahan validation score peak kare wahi sweet spot.

Q259 BEGINNER What is the difference between `predict` and `predict_proba`?

Interview answer. `predict` returns class labels; `predict_proba` returns class probabilities, which allow threshold tuning and ranking.

Hinglish samjho. predict direct label deta hai, predict_proba confidence. Threshold badalna ho to proba chahiye.

Q260 INTERMEDIATE Why is `random_state` important?

Interview answer. It fixes the randomness in splits, initialisation and sampling, making results reproducible and comparisons fair.

Hinglish samjho. Seed fix nahi to har run mein alag score, aur do models ki tulna ka koi matlab nahi.

Q261 INTERMEDIATE What is early stopping?

Interview answer. Stopping training when validation performance stops improving for a set number of rounds, to prevent overfitting and save time (gradient boosting, neural networks).

Hinglish samjho. Validation loss badhna shuru ho to training rok do, aur best point par wapas aa jao.

Q262 INTERMEDIATE **What is the Local Outlier Factor or Isolation Forest used for?**

Interview answer. Unsupervised anomaly detection: Isolation Forest isolates anomalies with fewer random splits; LOF compares a point's local density with its neighbours'.

Hinglish samjho. Bina label ke anomalies dhundne ke tarike. Isolation Forest tez hai aur high-dimension mein achha chalta hai.

Q263 BEGINNER **What is the difference between a model's bias and statistical bias in data?**

Interview answer. Model bias is error from simplifying assumptions (underfitting); data bias means the sample or labels systematically misrepresent the population or reflect unfair patterns.

Hinglish samjho. Model bias = model simple hai. Data bias = data hi ek taraf jhuka hua hai. Dono alag problems hain.

PART J

Reference

46. Formula Cheat Sheet

47. References and Further Reading

CHAPTER 46

Formula Cheat Sheet

How to use this page

One-page revision of every formula in the handbook. Symbols: y true value, $\hat{y}$ prediction, n samples, p features, $\bar{y}$ mean of y.

Regression

Concept	Formula
Linear model	$\hat{y} = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p$
Simple OLS	$\beta_1 = \dfrac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2}, \quad \beta_0 = \bar{y} - \beta_1 \bar{x}$
Normal equation	$\hat{\boldsymbol{\beta}} = (X^\top X)^{-1} X^\top \mathbf{y}$
Ridge / Lasso	$\text{SSE} + \lambda \sum \beta_j^2$ / $\text{SSE} + \lambda \sum \lvert\beta_j\rvert$
MAE / MSE / RMSE	$\frac{1}{n}\sum \lvert y_i - \hat{y}_i\rvert$ / $\frac{1}{n}\sum (y_i - \hat{y}_i)^2$ / $\sqrt{\text{MSE}}$
R^2 and adjusted R^2	$1 - \dfrac{\text{SSE}}{\text{SST}}$ / $1 - (1 - R^2)\dfrac{n-1}{n-p-1}$
VIF	$\dfrac{1}{1 - R_j^2}$

Classification

Concept	Formula
Sigmoid, odds	$\sigma(z) = \dfrac{1}{1+e^{-z}}, \quad \text{odds} = \dfrac{p}{1-p}, \quad \ln \text{odds} = z$
Log loss	$-\frac{1}{n}\sum [y \ln p + (1-y)\ln(1-p)]$
Accuracy	$\dfrac{TP + TN}{TP + TN + FP + FN}$
Precision / Recall	$\dfrac{TP}{TP + FP}$ / $\dfrac{TP}{TP + FN}$
Specificity / FPR	$\dfrac{TN}{TN + FP}$ / $\dfrac{FP}{FP + TN}$
F_1 and F_β	$\dfrac{2PR}{P+R}$ / $(1+\beta^2)\dfrac{PR}{\beta^2 P + R}$
Bayes theorem	$P(c \mid x) = \dfrac{P(x \mid c) P(c)}{P(x)}$
Gaussian NB likelihood	$\dfrac{1}{\sqrt{2\pi\sigma^2}} \exp\!\left(-\dfrac{(x-\mu)^2}{2\sigma^2}\right)$
Gini / Entropy	$1 - \sum p_k^2$ / $-\sum p_k \log_2 p_k$
Information gain	$I(\text{parent}) - \sum \frac{n_c}{n} I(c)$
Euclidean / Manhattan	$\sqrt{\sum (a_j - b_j)^2}$ / $\sum \lvert a_j - b_j\rvert$
SVM margin, RBF kernel	$\dfrac{2}{\lVert w\rVert}, \quad \exp(-\gamma \lVert a - b\rVert^2)$

Ensembles

Concept	Formula
Bagging prediction	$\frac{1}{M}\sum_m f_m(x)$ (vote for classes)
Bootstrap share left out	$(1-1/n)^n \to e^{-1} \approx 0.368$
Variance of average	$\rho\sigma^2 + \frac{1-\rho}{M}\sigma^2$
AdaBoost learner weight	$\alpha_t = \frac{1}{2}\ln\frac{1-\varepsilon_t}{\varepsilon_t}$
AdaBoost sample update	$w_i \leftarrow w_i e^{-\alpha_t y_i h_t(x_i)}/Z$
Gradient boosting	$F_m = F_{m-1} + \eta h_m$, $h_m \approx$ residuals $y - F_{m-1}$
Random forest features per split	$\approx \sqrt{p}$ (classification)

Tuning, Unsupervised and Preprocessing

Concept	Formula
CV score	$\frac{1}{k}\sum_{i=1}^{k} \text{score}_i$
Grid search fits	(product of value counts) $\times k$
Random search trials for top q with confidence c	$n \geq \ln(1-c)/\ln(1-q)$
K-Means inertia	$\sum_j \sum_{x \in C_j} \lVert x - \mu_j \rVert^2$
Silhouette	$(b-a)/\max(a,b)$
DBSCAN core point	$\lvert N_\varepsilon(p) \rvert \geq \text{min_samples}$
PCA	$\Sigma v = \lambda v$; $\text{ratio}_j = \lambda_j / \sum \lambda$
IQR fences	$Q_1 - 1.5\,\text{IQR}$, $Q_3 + 1.5\,\text{IQR}$
Standardisation	$z = (x-\mu)/\sigma$
Min-Max	$(x - x_{\min})/(x_{\max} - x_{\min})$
One-hot	$x_c = 1[\text{value} = c]$

Decision Guide

Situation	Reach for
Need a quick interpretable baseline	Linear / Logistic Regression, shallow Decision Tree
Small data, many features, clear margin	SVM (scaled)
Tabular data, want top accuracy	Gradient Boosting, Random Forest
Many correlated features	Ridge, PCA
Unknown number of clusters, noisy shapes	DBSCAN
Round, similar-size groups	K-Means (scaled)
Imbalanced classes	Class weights, PR-AUC, threshold tuning
Any preprocessing	Pipeline + ColumnTransformer, fitted on train only

Five-second leakage check

Did any statistic (mean, category list, imputation value, importance, threshold) come from data that includes the test set? If yes, move that step into a Pipeline and refit.

CHAPTER 47

References and Further Reading

How these sources were checked

Sources were found with web search during the build (October 2026). ✔ **= the exact URL appeared in search results** for the stated topic. ○ **= the page is a standard section of the official scikit-learn User Guide and is listed in the verified User Guide index, but the direct page could not be fetched** from the build environment (outbound access to scikit-learn.org was blocked), so click-test it once. All explanations, formulas, code and the 263 interview questions in this handbook are written in original wording; external pages were used only to check topic coverage and terminology, nothing was copied. No company-specific interview claims are made.

Official scikit-learn documentation

Topic	Link	Status
User Guide (index)	scikit-learn.org/stable/user_guide.html	✔
Linear models (Linear, Ridge, Lasso, Logistic)	modules/linear_model.html	○
Nearest neighbors (KNN)	modules/neighbors.html	○
Decision trees	modules/tree.html	○
Naive Bayes	modules/naive_bayes.html	○
Support vector machines	modules/svm.html	○
Ensembles (bagging, forests, AdaBoost, gradient boosting, stacking)	modules/ensemble.html	✔
Cross-validation	modules/cross_validation.html	✔
Tuning hyper-parameters (GridSearchCV, RandomizedSearchCV)	modules/grid_search.html	○
Metrics and scoring (R^2, MAE, MSE, accuracy, F1, confusion matrix)	modules/model_evaluation.html	✔
`classification_report` API	sklearn.metrics.classification_report	✔
Clustering (K-Means, DBSCAN)	modules/clustering.html	✔
PCA and decomposition	modules/decomposition.html	○
Pipelines and ColumnTransformer	modules/compose.html	✔
Example: ColumnTransformer with mixed types	plot_column_transformer_mixed_types	✔
Imputation of missing values	modules/impute.html	✔
Preprocessing (scalers, encoders)	modules/preprocessing.html	○
API: `sklearn.ensemble`	api/sklearn.ensemble.html	✔
API: `sklearn.cluster`	api/sklearn.cluster.html	✔
API: `sklearn.metrics`	api/sklearn.metrics.html	✔
Clustering examples gallery	auto_examples/cluster	✔

Interview-preparation resources

Resource	Link	Status and honest note
GeeksforGeeks: Machine Learning Interview Questions and Answers	geeksforgeeks.org/machine-learning/machine-learning-interview-questions/	✔ Broad topic list (regularisation, metrics, bias-variance, KNN, SVM, bagging vs boosting). Good for cross-checking coverage.
GeeksforGeeks: AI / ML / DS Interview Series	geeksforgeeks.org/interview-experiences/ai-ml-ds-interview/	✔ Hub page linking ML, deep learning and data-science question sets.
GeeksforGeeks: Machine Learning Quiz	geeksforgeeks.org/quizzes/machine-learning-quiz-questions-and-answers/	✔ Multiple-choice self-test.
Naukri Code360 (Coding Ninjas): Interview Questions library	naukri.com/code360/library/interview-questions	✔ Library of interview articles; it includes a "What is Machine Learning?" style entry among many general topics.
Naukri Code360: Guided Paths	naukri.com/code360/guided-paths	✔ Lists a "Data science / machine learning" track alongside DSA and interview paths.
LeetCode community post: problem sets for AI/ML interviews	leetcode.com/discuss/.../LeetCode-Problem-sets-to-prepare-for-AIML-interview	✔ A user post (not an official statement) noting that apart from SQL, LeetCode has no dedicated AI/ML problem set.
LeetCode: "LeetCode 75" study plan announcement	leetcode.com/discuss/post/2163754/...	✔ Data-structures-and-algorithms plan. Use it for the *coding* rounds that often accompany ML interviews, not for ML theory.
Educative: How to prepare for ML coding interviews	educative.io/blog/machine-learning-coding-interview-questions	✔ Describes typical rounds (implement k-means or logistic regression from scratch, take-home tasks).

Classic papers (cited by name; not link-checked)

- Breiman, L. (2001). *Random Forests.* Machine Learning, 45, 5-32.
- Freund, Y., and Schapire, R. (1997). *A decision-theoretic generalization of on-line learning and an application to boosting.* Journal of Computer and System Sciences, 55.
- Friedman, J. (2001). *Greedy function approximation: a gradient boosting machine.* Annals of Statistics, 29(5).
- Ester, M., Kriegel, H.-P., Sander, J., and Xu, X. (1996). *A density-based algorithm for discovering clusters in large spatial databases with noise (DBSCAN).* KDD-96.
- Cortes, C., and Vapnik, V. (1995). *Support-vector networks.* Machine Learning, 20, 273-297.
- Bergstra, J., and Bengio, Y. (2012). *Random search for hyper-parameter optimization.* Journal of Machine Learning Research, 13.

Suggested study plan

Four-week plan

- **Week 1:** Volume 1, Part B and C. Re-derive the worked examples by hand; answer the first 100 interview questions aloud.

- **Week 2:** Volume 1, Parts D and E, plus Volume 3 (preprocessing). Rebuild one Pipeline from memory.
- **Week 3:** Volume 2 (ensembles) and Project 1 and 2. Change one thing at a time and record what happens to the metrics.
- **Week 4:** Projects 3 and 4, the scenario questions and the rapid-fire round. Practise explaining each answer in one minute, in English and in your own words.

Honest limitations of this handbook

- The datasets in Projects 1-3 are synthetic; real data is messier. Scores will differ on your data.
- Printed outputs come from one run with fixed seeds on scikit-learn 1.9.1; decimals may differ slightly on other versions.
- Deep learning, time-series models, recommender systems and MLOps are outside this handbook's scope.